ספר פרויקט | Bike Fleet

מערכת לניטור ותפעול צי אופניים שיתופיים

אשר פירוב ופסח באגד

ספר פרויקט להגשה ולבחינה

C# · ASP.NET Core · Razor Views · GBFS

MySQL · MongoDB · Kafka · Redis

מהדורת תיעוד: אוקטובר 2026

הספר מתאר את הקוד לאחר הניקוי, את המערכת המקומית ואת גבולות האימות. עברית נקראת מימין לשמאל; קוד, נתיבים ונוסחאות מוצגים משמאל לימין.

מפת הספר והנחיות קריאה

הספר בנוי מן הבעיה אל המימוש. החלק הראשון מסביר את הצורך העסקי, קהלי היעד, הדרישות והארכיטקטורה. אחריו מופיע מדריך מסכים מצולם המחבר בין פעולת המשתמש לבין ה־View, ה־Controller והשירות שמבצע אותה. החלק הטכני מפרק את שירותי הקליטה, עיבוד ההודעות, האחסון, הניתוח, המטמון והאבטחה. בסוף מופיעים בדיקות, הפעלה, מגבלות ומפת קבצים.

כל קטע קוד נלקח מהעותק המקומי של הפרויקט ומלווה בשם הקובץ ובמספרי שורות. הקטע הוא מבחר לצורך הסבר; תפקיד שאר המתודות מוצג בתיאור הרכיב ובמפת הקבצים. שורה ארוכה עשויה להישבר חזותית בספר, בלי לשנות את הקוד המקורי.

נתוני המסכים הם תמונת מצב בזמן הצילום ולא תוצאה מובטחת. מספרי תחנות, כמות אופניים, תאריכים ומדדים משתנים. דוגמאות מספריות בפרקי האלגוריתם הן דוגמאות לימודיות ואינן נתוני ניסוי.

הכותרות המובנות במסמך מאפשרות ניווט באמצעות חלונית מתאר המסמך ב־Google Docs. לפני הגשה יש לעבור גם על פרק מגבלות האימות: בנייה שעברה אינה מוכיחה שכל תרחיש מול מסד נתונים אמיתי נבדק.

תוכן עניינים

1. הבעיה שהמערכת פותרת
2. מטרות, הצלחה וגבולות
3. משתמשים והרשאות
4. תרחישי שימוש עיקריים
5. מבנה הפתרון וגבולות האחריות
6. זרימת נתונים מקצה לקצה
7. חוזה GBFS ומשמעות המידע
8. זמן, טריות ואיכות נתונים
9. מודל אחסון יחסי
10. MongoDB והיסטוריית מדידות
11. Redis: מטמון, לא מקור אמת
12. אלגוריתם הסיכון
13. חיזוי מגמה ודוגמה מספרית
14. המלצות להעברת אופניים
15. שלושה מצבים שונים לאותה תחנה
16. אתחול שירות הקליטה
17. תזמון לולאות הקליטה
18. קריאת HTTP וטיפול בכשל
19. פענוח פרטי תחנות
20. פענוח זמינות וסוגי אופניים
21. יצרן Kafka והאיחוד לאחר הניקוי
22. צריכת מטא־דאטה ו־Handlers
23. אתחול המעבד ואחריות מסדי הנתונים
24. אתחול API ו־Dependency Injection
25. איחוד מידע ושליפת תחנה בודדת
26. FleetController וממשקי הקריאה
27. מחזור חיי משימה ושמירה
28. יצירת התראות והמלצות ברקע
29. סגירת דיווחי רוכבים
30. מדידת תועלת ודיוק
31. התחברות, הרשמה ויציאה
32. CSRF, הגבלת קצב ושמירת פרטיות
33. שכבת Razor ולקוח API פנימי
34. עיצוב, רענון ותצוגות משותפות
35. API ותיק ותאימות
מסכים 01 | כניסה למערכת
מסכים 02 | הרשמת רוכב

מסכים 03 | סקירת מצב הצי
מסכים 04 | רשימת תחנות וחיפוש
מסכים 05 | פרטי תחנה והיסטוריה
מסכים 06 | התראות תפעוליות
מסכים 07 | משימות העברת אופניים
מסכים 08 | מדדי השפעה
מסכים 09 | דיווחי רוכבים לצוות
מסכים 10 | ניהול משתמשים
מסכים 11 | יצירת חבר צוות
מסכים 12 | פרופיל משתמש
מסכים 13 | מסך שגיאה מבוקר
מסכים 14 | עריכת משתמש קיים
מסכים 15 | אישור פעולה
מסכים 16 | חיפוש אופניים לרוכב
מסכים 17 | תחנות מועדפות
מסכים 18 | שליחת דיווח ומעקב אישי
מסכים 19 | זהות רוכב והרשאותיו
מסכים 20 | מבט עובד תפעול
36. תהליך שלם: מן הלחיצה לשמירה
37. בדיקות עסקיות דטרמיניסטיות
38. בדיקות מטמון, HTTP ו־Kafka
39. בדיקות Razor ותרחישי קצה
40. הפעלה, הגדרות וסודות
41. ניקוי הקוד והחלטות תחזוקה
42. מגבלות והמשך פיתוח
43. הכנה להצגה ולשאלות בבחינה
44. מילון הישויות התפעוליות
45. חוזי DTO ומיפוי שמות
נספח א | קובצי ממשק, תצורה וכלי עבודה
נספח ב | מונחים ותשובות קצרות
נספח ג | מקורות וגרסת התיעוד

1. הבעיה שהמערכת פותרת

ברשת אופניים שיתופיים, כמות האופניים בכל תחנה משתנה בהתאם לנסיעות ולהחזרות. תחנה ריקה מונעת יציאה לנסיעה; תחנה מלאה מקשה על החזרת אופניים. גם אם כמות האופניים הזמינים ברשת גדולה, הפיזור הגאוגרפי עלול להיות בלתי מתאים לביקוש המקומי. לכן מפעיל צי זקוק לתמונת מצב לפי תחנה, לא רק לסכום כללי.

מעקב ידני אחר מאות או אלפי תחנות מקשה לזהות בזמן מחסור ומידע שאינו אמין. בנוסף, פעולה שבוצעה בשטח צריכה לקבל תיעוד: מי טיפל בה, מתי התחיל, מתי סיים ומה השתנה לאחר מכן. ללא תיעוד קשה להבחין בין בעיית נתונים, תקלה מדווחת ופעולת העברה שלא הושלמה.

המערכת מרכזת מידע חיצוני, בודקת את טריותו, מחשבת סיכון, מציעה העברות ומתעדת טיפול. היא נותנת למפעיל נקודת התחלה לקבלת החלטה. רוכבים יכולים לבדוק זמינות, לשמור תחנות מועדפות ולדווח על תקלות. כך מתקבל חיבור בין מידע מהספק, צוות התפעול ומשתמשי השירות.

גבול הפתרון ברור: המערכת אינה משכירה אופניים, אינה פותחת מנעולים ואינה מנהלת תשלום. היא אינה משנה מרחוק את צי הספק; היא מציגה ומנתחת מידע ציבורי ומתעדת עבודת משתמשים.

2. מטרות, הצלחה וגבולות

מטרה ראשונה היא אמינות: לא להציג חוסר מידע כאילו הוא אפס אופניים. מטרה שנייה היא שימושיות תפעולית: להציג תחנות בסיכון ולהציע העברה אפשרית על בסיס קיבולת, זמינות ומקומות החזרה. מטרה שלישית היא תיעוד: לשמור פעולות ומדדים בלי לייחס לתוכנה יכולת פיזית שאין לה.

הצלחה נבחנת גם ברמת התוכנה וגם ברמת השירות. ברמת התוכנה, בקשת תחנה מחזירה מידע עקבי, כתיבה שומרת סטטוס, רוכב אינו מקבל הרשאת מנהל ונפילת מטמון אינה מבטלת קריאה תקינה מהמסדים. ברמת השירות, אפשר לעקוב אחר דקות תחנה ריקה, זמן תגובה למשימות, שיעור השלמה ושגיאת תחזית.

לא נקבע יעד עסקי שלא נמדד, כגון הפחתת מחסור בשיעור מסוים. כדי לטעון לשיפור כזה נדרשת תקופת תצפית, בסיס להשוואה ובקרה על השפעות חיצוניות. הספר מתאר מדדים שהמערכת מסוגלת לחשב, ולא מבטיח שיפור שכבר הוכח.

הפתרון נשאר בטכנולוגיות הפרויקט: C# ו־ASP.NET Core בצד השרת, Razor ו־CSS לתצוגה, MySQL לנתונים יחסיים, MongoDB למדידות, Kafka להעברת הודעות ו־Redis למטמון. אין קוד JavaScript בממשק.

3. משתמשים והרשאות

למערכת שלושה תפקידים. ADMIN מנהל משתמשים וגם מבצע פעולות תפעול. OPERATOR צופה במידע התפעולי, מטפל בהתראות, במשימות ובדיווחי רוכבים, אך אינו מנהל את צוות המשתמשים. RIDER משתמש באזור ייעודי לזמינות, מועדפים ודיווחי תקלות אישיים.

הרשמה ציבורית אינה מאפשרת לבחור תפקיד: השרת יוצר RIDER. יצירת עובד או מנהל נעשית דרך API שמוגבל למנהל. הסתרת קישור בתפריט משפרת את הממשק אך אינה מנגנון האבטחה: ההרשאה נבדקת גם על הפעולה בשרת באמצעות Authorize.

מצב Enabled מאפשר להשבית חשבון בלי למחוק את ההיסטוריה שאליה הוא קשור. SessionVersion משתנה בניהול משתמש, ובדיקת ה־cookie משווה אותו לגרסה במסד. כך שינוי הרשאה או השבתה יכול לפסול כניסות קיימות. הקוד מונע ממנהל להשבית או להוריד את הרשאת עצמו ומגן על קיומו של מנהל פעיל אחרון.

משתמש לא מזוהה מקבל 401 מה־API. משתמש מזוהה שאין לו הרשאה מקבל 403. דפי Razor מפנים לכניסה כשצריך או מציגים שגיאה. יש להבדיל בין תפקיד המשתמש לבין מצב התחנה: הרשאה קובעת אילו פעולות מותר לבצע, לא מה מצב זמינות האופניים.

מתוך קוד המקור

StationApi/Controllers/UsersController.cs

```
017  [ApiController, Authorize(Roles = "ADMIN"), Route("api/users")]
018  public class UsersController(OperationsDbContext db, IPasswordHasher<FleetUser>
       passwords) : ControllerBase
019  {
020      [HttpGet]
021      public async Task<IActionResult> All() => Ok((await
       db.Users.AsNoTracking().OrderBy(x => x.Name).ToListAsync()).Select(UserView.From));
022      [HttpGet("{id:int}")]
023      public async Task<IActionResult> Get(int id) => await db.Users.FindAsync(id) is { }
       user ? Ok(UserView.From(user)) : NotFound();
024      [HttpPost]
025      public async Task<IActionResult> Create(UserRequest request)
026      {
027          if (string.IsNullOrWhiteSpace(request.Password) || request.Password.Length <
       10)
028              return BadRequest(new { message = "A password of at least 10 characters is
       required." });
029          var email = request.Email.Trim().ToLowerInvariant();
030          if (await db.Users.AnyAsync(x => x.Email == email)) return Conflict(new {
       message = "This email is already registered." });
031          var user = new FleetUser { Name = request.Name.Trim(), Email = email, Role =
       request.Role, Enabled = request.Enabled };
032          user.PasswordHash = passwords.HashPassword(user, request.Password);
033          db.Users.Add(user); await db.SaveChangesAsync();
```

4. תרחישי שימוש עיקריים

תרחיש מפעיל: כניסה למערכת, עיון בסיכום, מעבר לתחנה בסיכון, בדיקת טריות וקיבולת, פתיחת משימת העברה, התחלת עבודה, השלמה וחזרה לרשימה עם הסטטוס המעודכן. הנתון החיצוני עשוי להמשיך להצביע על מחסור גם לאחר השלמה; זו אינה הוכחה שהשמירה נכשלה.

תרחיש רוכב: יצירת חשבון, כניסה, חיפוש תחנה לפי שם, סימון מועדפת וצפייה במידע העדכני שלה. כאשר מתגלה תקלה, הרוכב בוחר תחנה וכותב תיאור. השרת מקשר את הדיווח למזהה המשתמש מתוך ה־cookie ולא למזהה שרירותי מהטופס.

תרחיש מנהל: פתיחת Team management, יצירת עובד, עריכת פרטים או השבתת חשבון. שדות הסיסמה אינם מוצגים בחזרה, והסיסמה אינה נשמרת כטקסט גלוי. התוצאה עוברת אימות גם בצד השרת.

תרחיש כשל: מקור הנתונים אינו זמין או מחזיר דיווח ישן. השרת אינו מייצר זמינות חלופית. המדידה האחרונה נשמרת עם הזמן המקורי, והניתוח מסמן שהמצב הנוכחי אינו ידוע. כשל Redis מוביל לקריאת מסד, בעוד כשל מסד מוצג כשגיאה ולא כהצלחה ריקה.

5. מבנה הפתרון וגבולות האחריות

ה־Solution מכיל שלושה שירותי ייצור ופרויקט בדיקות. שירות הקליטה קורא מסמכי GBFS ושולח הודעות Kafka. ProcessingService צורך אותן ושומר במאגרים. StationApi מחבר בין מאגרי הנתונים, האלגוריתם, API ומסכי Razor. FleetTests מריץ בדיקות ללא צורך להפעיל את כל סביבת Docker.

ההפרדה מאפשרת לתזמן קליטה בנפרד מבקשת משתמש. כאשר אדם פותח דף, הדף אינו נדרש להמתין לקריאה ישירה של הספק החיצוני; הוא משתמש במידע שכבר נקלט. Kafka מפריד בין קצב הקליטה לקצב השמירה ומאפשר לצרכן לחזור על הודעה אחרי כשל.

Shared מכיל חוזה מטמון קטן שנכלל בבנייה של ה־API והמעבד. הוא אינו שירות רשת נוסף. Views מחולקים לאזור Workspace ולאזור Rider, עם תצוגות משותפות לטבלאות ומשימות. Controllers של Razor מתווכים ל־API דרך FleetApiClient, וה־API מבצע הרשאות וגישת נתונים.

ההפרדה אינה מבטלת תלות תפעולית: המערכת עדיין זקוקה למאגרים ולתשתיות כדי לעבוד. יש כאן ארכיטקטורה מרובת תהליכים, אך אין לטעון שהפרויקט כבר כולל איזון עומסים, אשכול זמין־תמיד או מערכת ניטור ייצור מלאה.

6. זרימת נתונים מקצה לקצה

השלב הראשון הוא בקשת HTTP למקור Citi Bike. שירותי הקליטה קוראים פרטי תחנות, מצב תחנות וסוגי כלי רכב. הם מפענחים JSON ל־DTO, מוסיפים תחילית מקור למזהה תחנה ומעבירים הודעות אל topics ייעודיים ב־Kafka.

בשלב השני צרכן המטא־דאטה מבצע הוספה או עדכון ב־MySQL. צרכן הסטטוסים שומר מסמך נוכחי ב־StationsStatus ומדידה היסטורית ב־StationHistory. הודעת Kafka מאושרת לאחר הטיפול. כששמירה נכשלת, הצרכן חוזר לאותו offset ומנסה שוב; טיפול ב־JSON לא תקין מתועד בנפרד.

בשלב השלישי FleetDataService מאחד את רשומת התחנה היחסית, המצב האחרון וההיסטוריה. SmartFleetEngine מחשב טריות, תפוסה, קצב שינוי וסיכון. קריאות מתאימות נשמרות במטמון Redis למשך 60 שניות, וגרסת המטמון משתנה כשנכתב מידע חדש.

בשלב הרביעי FleetController מוסיף מצב תפעולי מתוך תקלות פתוחות, ו־WorkspaceController מקבל JSON דרך FleetApiClient ומעביר אותו ל־View. דפי מעקב מתרעננים במחזור באמצעות HTML. זהו תהליך אסינכרוני של עדכונים תקופתיים, ולכן ייתכן פער בין האירוע בשטח לזמן הופעתו במסך.

7. חוזה GBFS ומשמעות המידע

GBFS הוא מבנה ציבורי להפצת זמינות של מערכות ניידות משותפת. בפרויקט נבחר feed של Citi Bike, עם מסמכי station_information, station_status ו־vehicle_types. המקור כולל אופניים רגילים וחשמליים. כתובת הבסיס ותחילית התחנות נשמרות בהגדרות ולא מפוזרות בין המסכים.

station_information מספק מזהה, שם, מיקום וקיבולת. station_status מספק כמות אופניים זמינים, מקומות החזרה, דגלי שירות וזמן דיווח. ההבדל חשוב: קיבולת היא מאפיין התחנה, בעוד זמינות משתנה לעיתים קרובות. אין להסיק שכמות האופניים הזמינים שווה לגודל הצי, משום שכלים בנסיעה או בתחזוקה עשויים לא להיכלל.

ערך null מייצג שדה חסר, ואפס מפורש מייצג מדידה של אפס. ממיר הדגלים מטפל בספק שמחזיר true/false או 0/1 ושומר את חוזה ההודעות המספרי הקיים. דגל שאינו תקין גורם לשגיאת פענוח, במקום להפוך בשקט ל־false.

המזהה הפנימי מקבל citibike: כדי להבדיל בין מקורות. החלפת מקור אינה מוחקת נתונים קודמים, אבל השאילתות התפעוליות מסננות למקור המוגדר. בחיבור מקור אחר יש לבדוק התאמה לשדות, לזמנים ולמשמעות הקיבולת.

מתוך קוד המקור

BikeFleetOperationPlatform/Services/GbfsFlagConverter.cs

```
001  using System.Text.Json;
002  using System.Text.Json.Serialization;
003
004  namespace BikeFleetOperationPlatform.Services;
005
006  // GBFS flags may be booleans or 0/1, while our existing Kafka contract uses 0/1.
007  public class GbfsFlagConverter : JsonConverter<int?>
008  {
009      public override int? Read(ref Utf8JsonReader reader, Type type,
       JsonSerializerOptions options) => reader.TokenType switch
010      {
011          JsonTokenType.True => 1, JsonTokenType.False => 0, JsonTokenType.Null => null,
012          JsonTokenType.Number when reader.TryGetInt32(out var value) && value is 0 or 1
       => value,
013          _ => throw new JsonException("Expected a GBFS boolean or 0/1 flag.")
014      };
015      public override void Write(Utf8JsonWriter writer, int? value, JsonSerializerOptions
       options)
016      { if (value.HasValue) writer.WriteNumberValue(value.Value); else
       writer.WriteNullValue(); }
017  }
```

8. זמן, טריות ואיכות נתונים

המערכת משתמשת בשני זמנים שונים. ReportedAt הוא זמן המדידה שהספק דיווח, ואילו ReceivedAt הוא זמן קבלתה במערכת. רענון דף משנה את זמן יצירת הדף, אך אינו הופך מדידה ישנה למדידה חדשה. לכן יש להציג את הזמנים במשמעות הנכונה.

הניתוח מסמן MISSING כאשר חסרים שדות נדרשים, INVALID כאשר יש ערכים שליליים, חריגה מקיבולת או זמן רחוק בעתיד, ו־OUTDATED כאשר זמן הדיווח ישן מעבר לסף. ברירת המחדל לטריות היא עשר דקות. זמן עד שתי דקות בעתיד נסבל כפער שעון; מעבר לכך נחשב לא תקין.

כאשר נתונים חסרים או ישנים, תחזית אינה מחושבת. הצבע יכול להיות Warning כדי להפנות תשומת לב, אבל אין משמעותו שהתחנה בהכרח דלה באופניים. קיבולת אפס משמשת כמצב שבו לא פורסמה קיבולת שימושית: לא מחושבים אחוז תפוסה, זמן מילוי והמלצת העברה.

לצורך חישוב מגמה ההיסטוריה מוגבלת לחלון זמן, מסודרת ומנוכה ממדידות כפולות. אין להשלים פערי מידע באופן שממציא נסיעות. אמינות כזו חשובה יותר מהצגת מספר בכל שדה, מפני שמספר שגוי עלול להוביל לפעולה תפעולית שגויה.

מתוך קוד המקור

```
StationApi/Services/SmartFleetEngine.cs
038          bool stale = station.ReportedAt is null || station.ReportedAt <
       now.AddMinutes(-options.StaleMinutes)
039              || station.ReportedAt > now.AddMinutes(2);
040          bool missing = station.Count is null || station.AvailableDocks is null ||
       station.ReportedAt is null
041              || station.Renting is null || station.Returning is null;
042          bool invalid = station.Capacity < 0 || station.Count < 0 || (station.Capacity >
       0 && station.Count > station.Capacity)
043              || station.AvailableDocks < 0 || (station.Capacity > 0 &&
       station.AvailableDocks > station.Capacity)
044              || station.ReportedAt > now.AddMinutes(2);
045          if (missing || invalid || stale)
046          {
047              string state = invalid ? "INVALID" : missing ? "MISSING" : "OUTDATED";
048              string message = invalid ? "Station data is invalid. Verify the source
       before moving bikes."
049                  : missing ? "Required station data is missing. This does not mean the
       station is empty."
050                  : "The last reading is outdated. Current availability is unknown.";
051              return new(station, null, 70, "HIGH", "WARNING", null, null, null, true,
       message, state);
052          }
053          double? ratio = station.Capacity > 0 ? (double)station.Count!.Value /
       station.Capacity : null;
054          var history = station.History.Where(x => x.At >=
       now.AddMinutes(-options.HistoryMinutes)
055              && x.At <= now && x.Count >= 0 && (station.Capacity == 0 || x.Count <=
       station.Capacity))
```

9. מודל אחסון יחסי

MySQL משמש לשתי קבוצות נתונים. stationDb מחזיק את פרטי התחנות וסוגי הכלים. fleetOperationsDb מחזיק משתמשים, התראות, המלצות, מועדפים, דיווחים ומדידות תפעוליות. שני DbContext נפרדים מבהירים מי אחראי לכל סכימה ומונעים יצירה מחדש של סכימת הקליטה בזמן אתחול אזור התפעול.

ב־Users נשמרת כתובת דוא״ל ייחודית יחד עם שם, גיבוב סיסמה, תפקיד, סטטוס פעילות וגרסת session. ב־Recommendations נשמרים מזהי מקור ויעד, כמות, סיבה, עדיפות ומועדי שינוי. המזהים מקשרים מושגית לתחנות גם כאשר ההקשר היחסי האחר או MongoDB מחזיקים את המידע עצמו.

מועדף מזוהה באמצעות מפתח מורכב של משתמש ותחנה. TaskMeasurements משתמש במזהה ההמלצה כמפתח שאינו נוצר אוטומטית. אינדקסים על תחנה וסטטוס מסייעים לחיפוש בעיות פתוחות; אינדקס על תחנה וזמן תחזית מסייע לאיסוף מדדים.

אין להציג את כל הקשרים כאילו קיימים עבורם אילוצי Foreign Key במסד: בקוד רבים מן הקשרים נשמרים כמזהים ונבדקים בשירותים. ההפרדה בין מאגרים מחייבת אימות אפליקטיבי והבנה שאין עסקה יחידה שמכסה אוטומטית את כל שירותי המערכת.

מתוך קוד המקור

```
StationApi/DataDbContext/OperationsDbContext.cs
016      protected override void OnModelCreating(ModelBuilder model)
017      {
018          model.Entity<RiderFavorite>().HasKey(x => new { x.UserId, x.StationId });
019          model.Entity<RiderIssue>().HasIndex(x => new { x.StationId, x.Status });
020          model.Entity<TaskMeasurement>().HasKey(x => x.RecommendationId);
021          model.Entity<TaskMeasurement>().Property(x =>
       x.RecommendationId).ValueGeneratedNever();
022          model.Entity<ForecastSample>().HasIndex(x => new { x.StationId, x.IssuedAt });
023          model.Entity<FleetUser>().HasIndex(x => x.Email).IsUnique();
024          model.Entity<FleetAlert>().HasIndex(x => new { x.StationId, x.Status });
025          model.Entity<Recommendation>().HasIndex(x => x.Status);
026      }
027  }
```

10. MongoDB והיסטוריית מדידות

MongoDB שומר את המידע המשתנה בתדירות גבוהה. StationsStatus הוא אוסף המצב האחרון לכל תחנה; StationHistory מחזיק דגימות של כמות אופניים לפי זמן. צרכן הסטטוס בונה מזהה היסטוריה משילוב מזהה תחנה ו־last_reported, ולכן חזרה על אותה הודעה מעדכנת אותה דגימה במקום להוסיף כפילות.

רק זמן תקין וכמות שאינה שלילית הופכים לדגימה היסטורית. מידע חסר עדיין יכול להופיע במצב האחרון כדי שהאתר יוכל להתריע על איכות הנתונים. המנגנון מבחין בין שמירת הראיה שהתקבלה לבין שימוש בה לצורך חישוב מגמה.

קיים אינדקס משולב על StationId ו־RecordedAt, ואינדקס TTL לשבעה ימים. TTL של MongoDB הוא מדיניות מחיקה ברקע, ולא הבטחה שהמסמך יימחק בשנייה המדויקת שבה חלפו שבעה ימים. ה־API מבקש בדרך כלל חלון היסטוריה קצר יותר, לפי HistoryMinutes.

בעת קבלת דיווח ישן יותר, הקוד בודק אם הוא רשאי להחליף את המצב האחרון. לאחר הכתיבה מתבצעת פסילת מטמון. כתיבת היסטוריה וכתיבת latest אינן עטופות כאן בעסקת MongoDB משותפת; Idempotency מאפשרת לצרכן לחזור על ההודעה לאחר כשל חלקי.

מתוך קוד המקור

```
ProcessingService/Service/MongoConsumerService.cs
060                         var history = new BsonDocument { ["_id"] =
       $"{state.Station_id}:{state.Last_reported}",
061                             ["StationId"] = state.StationId, ["RecordedAt"] = at,
       ["Count"] = state.NumBikesAvailable.Value };
062                         // Idempotent writes make replays safe when one write succeeds
       and another fails.
063                         await _history.ReplaceOneAsync(Builders<BsonDocument>.Filter.Eq
       ("_id", history["_id"]), history,
064                             new ReplaceOptions { IsUpsert = true }, token);
065                     }
066                     var filter = Builders<BsonDocument>.Filter.Eq("Station_id",
       state.StationId);
067                     var previous = await
       _latest.Find(filter).FirstOrDefaultAsync(token);
068                     var previousTime = previous?.GetValue("Last_reported",
       BsonNull.Value) ?? BsonNull.Value;
069                     long previousTimestamp = previousTime.IsInt32 ||
       previousTime.IsInt64 ? previousTime.ToInt64() : 0;
070                     if (previous is null || !validTime || previousTimestamp <=
       state.LastReported
071                         || previousTimestamp >
       DateTimeOffset.UtcNow.AddMinutes(2).ToUnixTimeSeconds())
072                     {
073                         if (previous is not null && previous.Contains("_id"))
       document["_id"] = previous["_id"];
074                         await _latest.ReplaceOneAsync(filter, document, new
       ReplaceOptions { IsUpsert = true }, token);
075                         // Cache only after durable writes. Redis failure must not lose
       telemetry.
```

11. Redis: מטמון, לא מקור אמת

Redis נועד לצמצם עבודה חוזרת בקריאות של ניתוח תחנות. כאשר יש cache hit, FleetDataService טוען את נתוני התחנה השמורים ומחשב מחדש את הניתוח מול השעה הנוכחית. כך גיל המדידה אינו מתקבע רק מפני שהתשובה נמצאת במטמון.

מפתח המטמון כולל scope שנגזר מכתובת המקור, תחילית המזהים, אפשרויות הניתוח, גרסה מקומית וגרסת Redis משותפת. Dashboard ותחנה בודדת משתמשים במשאבים נפרדים. התוצאה נשמרת ל־60 שניות בדיוק. תוצאה ריקה בתחילת הקליטה אינה נשמרת, וכשל קריאת מסד אינו הופך לתוצאה מוצלחת במטמון.

בעת עדכון נתונים המעבד משנה RevisionKey. לאחר שינוי תפעולי ה־API משנה גם דור מקומי. אם Redis אינו זמין, הדור המקומי מסייע למנוע שימוש חוזר במפתח שהיה ידוע לפני השינוי; בקריאה אפשר לחזור למאגרים. זה אינו מנגנון עקביות טרנזקציוני כלל־מערכתי, וה־TTL מגביל את משך החיים של רשומות ישנות.

SemaphoreSlim מונע מספר מילויים מקבילים בתוך תהליך API אחד. הוא אינו נעילה מבוזרת. הגדלת מספר מופעי API דורשת לבחון מחדש עומס, תיאום ופסילות, ולא להניח שהנעילה המקומית פועלת בין שרתים.

מתוך קוד המקור

```
StationApi/Services/FleetDataService.cs
034      private async Task<List<StationAnalysis>> GetCached(string resource,
       Func<Task<List<StationAnalysis>>> load, CancellationToken token)
035      {
036          await cacheState.Gate.WaitAsync(token);
037          try
038          {
039              var cache = redis.GetDatabase();
040              string? key = null;
041              try
042              {
043                  var revision = await
       cache.StringGetAsync(FleetCacheContract.RevisionKey).WaitAsync(token);
044                  var scope = $"{config["DataSource:BaseUrl"]}|{config["DataSource:Statio
       nPrefix"]}|{JsonSerializer.Serialize(options)}|{cacheState.Generation}|{revision}";
045                  key = FleetCacheContract.KeyPrefix +
       Convert.ToHexString(SHA256.HashData(Encoding.UTF8.GetBytes(scope))) + ":" + resource;
046                  var value = await cache.StringGetAsync(key).WaitAsync(token);
047                  if (value.HasValue)
048                  {
049                      var saved =
       JsonSerializer.Deserialize<List<StationAnalysis>>(value.ToString());
050                      if (saved is not null && saved.All(x => x is not null && x.Station
       is not null && x.Station.History is not null))
051                          return saved.Select(x => engine.Analyze(x.Station,
       DateTime.UtcNow))
```

12. אלגוריתם הסיכון

SmartFleetEngine הוא רכיב חישוב ללא גישה למסדים או לרשת. הוא מקבל StationInput ואת הזמן הנוכחי כפרמטר. בחירה זו מאפשרת לבדוק מקרי גבול באופן דטרמיניסטי בלי להיות תלויים בשעון אמיתי או בנתוני הספק ברגע הבדיקה.

לאחר בדיקת תקינות, מחושב יחס זמינות לקיבולת. ערך נמוך מאוד או גבוה מאוד מעלה את הסיכון; תחנה ריקה מקבלת סיכון גבוה. ברירות המחדל מגדירות אזור נמוך מתחת ל־25%, אזור גבוה מעל 75% ויעד של 50%. זמני תחזית קצרים מעלים את הציון גם כאשר הכמות הנוכחית נראית סבירה.

הציון מתורגם ל־HEALTHY, WARNING או CRITICAL, ולרמת סיכון נוספת. דגלי השכרה או החזרה סגורים מעלים תשומת לב. הציון אינו הסתברות סטטיסטית: 95 אינו אומר שיש סיכוי של 95% למחסור. זהו סולם כללים שנועד לתעדף תחנות.

יש לשמור על הפרדה בין סיכון זמינות לתחזוקה. רוכב יכול לדווח על תקלה גם בתחנה מאוזנת. לעומת זאת תחנה יכולה להיות Critical ללא תקלה פיזית, רק מפני שאזלו האופניים. השילוב התפעולי נעשה בשירות אחר ולא בתוך החישוב הטהור.

מתוך קוד המקור

```
StationApi/Services/SmartFleetEngine.cs
066          int score = station.Count == 0 ? 85 : ratio <= .1 || ratio >= .9 ? 85 : ratio <
       options.LowRatio || ratio > options.HighRatio ? 55 : 15;
067          double? earliest = shortage ?? full;
068          if (earliest <= options.CriticalMinutes) score = Math.Max(score, 95);
069          else if (earliest <= options.WarningMinutes) score = Math.Max(score, 75);
070          if (station.Renting != true || station.Returning != true) score =
       Math.Max(score, 70);
071          string status = score >= 85 ? "CRITICAL" : score >= 40 ? "WARNING" : "HEALTHY";
072          string level = score >= 85 ? "CRITICAL" : score >= 70 ? "HIGH" : score >= 40 ?
       "MEDIUM" : "LOW";
073          string explanation = shortage is not null ? $"Expected to run out in
       approximately {DurationText.Format(shortage.Value)} at the recent net rate."
074              : full is not null ? $"Expected to reach capacity in approximately
       {DurationText.Format(full.Value)} at the recent net rate."
075              : rate is null ? "Not enough recent history for a prediction; risk uses
       current availability."
076              : "Availability is stable over the recent observation window.";
077          if (station.Renting != true || station.Returning != true) explanation += "
       Station rental or return service is unavailable.";
078          if (station.Capacity == 0) explanation += " Station capacity is not published.
       Utilization, full-capacity predictions and transfer recommendations are
       unavailable.";
```

13. חיזוי מגמה ודוגמה מספרית

התחזית מבוססת על קצב שינוי נטו בין הדגימה הראשונה לאחרונה בחלון ההיסטוריה. נדרשות לפחות שתי דגימות, מרחק זמן של חמש דקות לפחות ודגימה אחרונה עדכנית. הקצב מחושב כאופניים לשעה; הוא מבטא את ההפרש הכולל, לא מספר נסיעות נפרד ומספר החזרות נפרד.

לדוגמה לימודית: בתחנה היו 18 אופניים לפני שעה ו־12 עכשיו. הקצב הוא מינוס 6 אופניים לשעה. אם המגמה תימשך, 12 חלקי 6 הם שעתיים, כלומר 120 דקות עד ריקון. כאשר הקצב חיובי, זמן המילוי מחושב מן המקומות שנותרו בקיבולת חלקי הקצב.

כאשר אין מספיק היסטוריה או שהקצב קרוב לאפס, זמן התחזית הוא null. תחנה ריקה מקבלת זמן ריקון אפס גם ללא חיזוי מגמה. אין לבלבל בין מצב נוכחי שכבר התרחש לבין תחזית עתידית.

זו אקסטרפולציה ליניארית פשוטה, לא מודל למידת מכונה. היא אינה כוללת מזג אוויר, שעות שיא, אירועים או לוח חופשות. היתרון הוא שקיפות והסבר קצר בבחינה; המגבלה היא שמגמה יכולה להשתנות מיד. הפרויקט מודד שגיאת תחזיות כדי להעריך את התועלת בפועל.

מתוך קוד המקור

```
StationApi/Services/SmartFleetEngine.cs
057          double? rate = null;
058          if (history.Count >= 2)
059          {
060              double hours = (history[^1].At - history[0].At).TotalHours;
061              if (hours >= 5d / 60 && history[^1].At >=
       now.AddMinutes(-options.StaleMinutes))
062                  rate = (history[^1].Count - history[0].Count) / hours;
063          }
064          double? shortage = station.Count == 0 ? 0 : rate < -.01 ? station.Count / -rate
       * 60 : null;
065          double? full = station.Capacity == 0 ? null : station.Count == station.Capacity
       ? 0 : rate > .01 ? (station.Capacity - station.Count) / rate * 60 : null;
066          int score = station.Count == 0 ? 85 : ratio <= .1 || ratio >= .9 ? 85 : ratio <
       options.LowRatio || ratio > options.HighRatio ? 55 : 15;
067          double? earliest = shortage ?? full;
068          if (earliest <= options.CriticalMinutes) score = Math.Max(score, 95);
069          else if (earliest <= options.WarningMinutes) score = Math.Max(score, 75);
```

14. המלצות להעברת אופניים

Recommend מקבלת את ניתוח התחנות ואת קבוצת התחנות שכבר שמורות לפעולות פעילות או אחרונות. רק תחנות עם מידע עדכני, קיבולת חיובית ושירותי השכרה והחזרה פתוחים כשירות להמלצה. לכל תחנה מחושבים עודף מעל יעד ואופניים חסרים עד יעד.

התחנות המקבלות מטופלות לפי סיכון יורד. המועמדות לתרומה מסודרות לפי סיכון ואחר כך לפי מרחק יחסי. פונקציית המרחק משתמשת בסכום ריבועי הפרשי קווי רוחב ואורך; זהו דירוג גאוגרפי פשוט, לא מרחק כביש או זמן נסיעה.

כמות ההעברה מוגבלת למינימום של MaxMove, עודף התורמת, צורך המקבלת ומקומות ההחזרה הזמינים. ברירת המחדל למהלך היא עד 12 אופניים. תורמת שצפויה להתרוקן בקרוב אינה נבחרת, ומקבלת שצפויה להתמלא אינה מקבלת העברה.

לאחר יצירת המלצה, יתרות התחנות מתאפסות לצורך המחזור כדי לא ליצור הוראות חופפות. OperationsService מוסיף תקופת המתנה של 15 דקות בהתאם להגדרות. האלגוריתם חמדני ואינו אופטימיזציה גלובלית של מסלולי משאיות; יתרונו הוא פשטות ובקרה ברורה.

מתוך קוד המקור

```
StationApi/Services/SmartFleetEngine.cs
104                 int quantity = Math.Min(options.MaxMove,
       Math.Min(available[donor.Station.Id], needs[receiver.Station.Id]));
105                 quantity = Math.Min(quantity, Math.Max(0,
       receiver.Station.AvailableDocks!.Value));
106                 if (quantity <= 0) continue;
107                 result.Add(new(donor.Station.Id, receiver.Station.Id, quantity,
108                     Math.Max(receiver.RiskScore, donor.RiskScore) >= 85 ? "CRITICAL" :
       "HIGH",
109                     $"Move {quantity} bikes from {donor.Station.Name} to
       {receiver.Station.Name}. "
110                     + (shortage ? receiver.Explanation : donor.Explanation)));
111                 available[donor.Station.Id] -= quantity;
112                 needs[receiver.Station.Id] -= quantity;
113                 // One active operation per station keeps operator instructions
       unambiguous.
114                 available[receiver.Station.Id] = 0;
115                 needs[donor.Station.Id] = 0;
116                 available[donor.Station.Id] = 0;
117                 needs[receiver.Station.Id] = 0;
118                 break;
119             }
```

15. שלושה מצבים שונים לאותה תחנה

Status בניתוח מתאר את סיכון הזמינות שמגיע מהמדידות. MaintenanceStatus מציין אם קיימים דיווחי רוכבים פתוחים. OperationalStatus משלב את המידע: תחזוקה כאשר יש דיווחים, מצב לא ידוע כאשר המידע ישן, שירות לא זמין כאשר דגלי ההפעלה סגורים, תשומת לב כאשר נותרו התראות או סיכון, ו־ACTIVE כאשר אין בעיות והמידע תקין.

למשימה עצמה יש מחזור חיים נפרד: PENDING, IN_PROGRESS, COMPLETED או DISMISSED. סגירת משימה משמרת את הפעולה שביצע המשתמש. היא אינה מוסיפה או מורידה אופניים במסמך הספק. לכן ייתכן שמשימה הושלמה ובמקביל התחנה עדיין דורשת תשומת לב.

Apply היא פונקציה טהורה יחסית שמקבלת את ניתוח התחנה ומספר הבעיות. Enrich קוראת את מספר ההתראות והדיווחים הפתוחים ממסד התפעול ומשלבת אותם בתוצאות. ReconcileProviderAlerts מסיימת התראות ספק מוכרות רק כאשר המידע כבר עדכני ובריא.

ההפרדה מונעת תיקון קוסמטי שצובע תחנה בירוק רק משום שנלחץ כפתור. בבחינה כדאי להדגים שני תרחישים: סגירת הבעיה האחרונה בתחנה תקינה, וסגירת בעיה אחת כאשר קיימת בעיה אחרת. התוצאה התפעולית אמורה להיות שונה.

מתוך קוד המקור

```
StationApi/Services/StationStateService.cs
008       public static StationAnalysis Apply(StationAnalysis analysis, int openAlerts, int
        openReports) => analysis with {
009           OpenIssueCount = openAlerts + openReports,
010           MaintenanceStatus = openReports > 0 ? "OPEN_ISSUES" : "CLEAR",
011           OperationalStatus = openReports > 0 ? "MAINTENANCE" : analysis.Stale ?
        "UNKNOWN"
012               : analysis.Station.Renting != true || analysis.Station.Returning != true ?
        "SERVICE_UNAVAILABLE"
013               : openAlerts > 0 || analysis.Status != "HEALTHY" ? "ATTENTION" : "ACTIVE"
014       };
015       public async Task ReconcileProviderAlerts(StationAnalysis station,
        CancellationToken token)
016       {
```

16. אתחול שירות הקליטה

Program של שירות הקליטה יוצר HostApplicationBuilder וטוען הגדרות. לפני הפעלת ה־worker נבדקת כתובת המקור: עליה להיות HTTPS ולהסתיים בקו נטוי כדי שבניית כתובות יחסיות תעבוד. נבדקים גם תחילית מקור, timeout ומרווחי קליטה. כשל בהגדרות נעצר מוקדם עם הודעה מפורשת במקום ליצור מידע חלקי.

AddHttpClient מגדיר לקוח בעל שם עם כתובת בסיס, timeout ו־IHttpClientFactory. User-Agent מאפשר לשירותים לבקש לקוח עם אותן הגדרות. KafkaProducerServices נרשם כ־Singleton, ושירותי הפענוח נרשמים כ־Transient. HostedService נרשם פעם אחת ומנהל את התזמון.

הקובץ אינו מבצע קריאות MySQL או Redis. בניקוי הוסרו ממנו תלויות מסד שלא שימשו את הקליטה ושלושה משתני topic שלא נקראו. הגדרות ה־topic הפעילות עדיין נקראות על ידי ה־worker. בכך צומצם הקוד בלי לשנות את ההודעות או את מסלולי הנתונים.

בעת תחזוקה, הגדרות סביבת הרצה יכולות לדרוס ערכי appsettings באמצעות שמות עם שני קווים תחתונים. אין להעתיק סיסמאות למסמך או לקוד; קובץ .env המקומי משמש את Compose ואינו מקור שיש לפרסם.

מתוך קוד המקור

```
BikeFleetOperationPlatform/Program.cs
024  builder.Services.AddHttpClient(
025      httpClientName,
026      client =>
027      {
028          client.BaseAddress = new Uri(builder.Configuration["DataSource:BaseUrl"]!);
029          client.Timeout = TimeSpan.FromSeconds(builder.Configuration.GetValue<int>("Http
       :TimeoutSeconds"));
030          client.DefaultRequestHeaders.UserAgent.ParseAdd("BikeFleet/1.0");
031      });
032
033  builder.Services.AddSingleton<KafkaProducerServices>(sp => new
       KafkaProducerServices(bootstrapServices));
034  builder.Services.AddTransient<StationInformationService>();
035  builder.Services.AddTransient<StationStatusService>();
036  builder.Services.AddTransient<VehicleTypesService>();
037
038  builder.Services.AddHostedService<StationsPollingWorker>();
039
040  using IHost host = builder.Build();
```

17. תזמון לולאות הקליטה

StationsPollingWorker יורש מ־BackgroundService. ExecuteAsync קוראת את ה־topic ואת המרווח לכל סוג feed ופותחת שלוש משימות מחזוריות. פרטי תחנות וסוגי כלים מתעדכנים בתדירות נמוכה יותר ממצב התחנות, משום שהם משתנים פחות. Task.WhenAll משאיר את השירות חי כל עוד הלולאות פועלות.

RunPeriodicJobAsync מבצעת את הפעולה פעם אחת מיד, ואז ממתינה ל־PeriodicTimer. כל מחזור ממתין לסיום הפעולה הקודמת; אין באותה לולאה שתי קריאות חופפות. שלושת סוגי הקליטה כן יכולים לפעול במקביל, וזה מכוון.

ExecuteSafelyAsync מתעדת כשל ומאפשרת למחזור הבא להמשיך. OperationCanceledException אינה נבלעת כשגיאה רגילה, כדי שה־host יוכל להיסגר בצורה מסודרת. השימוש ב־CancellationToken עובר דרך שירותי הקליטה עד בקשות HTTP ושליחת Kafka.

במבחן אפשר להסביר את ההבדל בין תזמון לביצוע: ה־worker יודע מתי לקרוא, אך השירות יודע מה לקרוא ואיך לפרש. הפרדה זו מונעת הכנסת כל פרטי הפענוח ללולאת התזמון ומאפשרת שינוי מרווחים בלי שינוי הלוגיקה העסקית.

מתוך קוד המקור

```
BikeFleetOperationPlatform/Worker/StationsPollingWorker.cs
062      private async Task RunPeriodicJobAsync(
063          string jobName,
064          TimeSpan interval,
065          Func<Task> action,
066          CancellationToken stoppingToken)
067      {
068          _logger.LogInformation("Job {JobName} started (every {Interval}s)", jobName,
       interval.TotalSeconds);
069          using var timer = new PeriodicTimer(interval);
070          try
071          {
072              await ExecuteSafelyAsync(jobName, action);
073
074              while (!stoppingToken.IsCancellationRequested &&
075                     await timer.WaitForNextTickAsync(stoppingToken))
076              {
077                  await ExecuteSafelyAsync(jobName, action);
078              }
079          }
080          catch(OperationCanceledException)
081          {
082              _logger.LogInformation("Job {JobName} gracefully stopped.", jobName);
083          }
084      }
```

18. קריאת HTTP וטיפול בכשל

GbfsHttp מרכז את פעולת GetJsonAsync שבה משתמשים שלושת שירותי הקליטה. הוא מגביל את מספר הניסיונות לטווח אחד עד שלושה, מבצע GET, בודק קוד HTTP ומפענח את זרם התשובה ל־JsonDocument. Dispose של התשובה ושל הזרם משחרר משאבים גם כאשר הפענוח נכשל.

429 ושגיאות 5xx מאפשרות ניסיון חוזר עם השהיה מדורגת לפי מספר הניסיון. שגיאת חיבור ללא קוד HTTP ו־timeout שאינו ביטול יזום יכולים להוביל לניסיון נוסף. ביטול שהגיע מהקורא אינו הופך לרצף ניסיונות חוזרים. שגיאת לקוח כגון 404 אינה מטופלת כהצלחה ריקה.

הבחירה לשמור את הקובץ אחרי הניקוי מבוססת על שימוש ממשי: בלעדיו אותו מנגנון יופיע שלוש פעמים. לעומת זאת, helper לסינון ספקים שכבר לא היה נחוץ הוסר. קובץ עזר מוצדק כאשר הוא מרכז אחריות משותפת ומובנת, ולא רק כאשר הוא מקטין את מספר השורות בקובץ אחר.

הבדיקות מפעילות HttpMessageHandler מדומה שמחזיר כשל זמני ולאחריו feed ריק תקין. ההבחנה בין feed ריק תקין לבין כשל ספק היא חלק מחוזה האמינות.

מתוך קוד המקור

```
BikeFleetOperationPlatform/Services/GbfsHttp.cs
007      public static async Task<JsonDocument> GetJsonAsync(HttpClient client, string path,
       int maxAttempts, CancellationToken token)
008      {
009          if (maxAttempts is < 1 or > 3) throw new
       InvalidOperationException("Http:MaxAttempts must be between 1 and 3.");
010          for (int attempt = 1; ; attempt++)
011          {
012              try
013              {
014                  using var response = await client.GetAsync(path, token);
015                  if ((response.StatusCode == HttpStatusCode.TooManyRequests ||
       (int)response.StatusCode >= 500) && attempt < maxAttempts)
016                  { await Task.Delay(TimeSpan.FromSeconds(attempt), token); continue; }
017                  response.EnsureSuccessStatusCode();
018                  await using var stream = await
       response.Content.ReadAsStreamAsync(token);
019                  return await JsonDocument.ParseAsync(stream, cancellationToken: token);
020              }
021              catch (HttpRequestException ex) when (attempt < maxAttempts &&
       ex.StatusCode is null)
022              { await Task.Delay(TimeSpan.FromSeconds(attempt), token); }
023              catch (OperationCanceledException) when (!token.IsCancellationRequested &&
       attempt < maxAttempts)
024              { await Task.Delay(TimeSpan.FromSeconds(attempt), token); }
025          }
026      }
027  }
```

19. פענוח פרטי תחנות

StationInformationService מבקשת station_information.json, בוחרת את המערך data.stations ומפענחת כל איבר ל־StationInformationDto. האפשרות PropertyNameCaseInsensitive מאפשרת התאמת שמות ללא תלות ברישיות, אך אינה משנה את משמעות השדות.

לפני השליחה נבדק שמזהה התחנה אינו ריק, מתווספת תחילית המקור ומופעל Validator.TryValidateObject על DataAnnotations. שם, מיקום וקיבולת נשמרים לפי ה־DTO. רשומות שנכשלות באימות מתועדות ולא נכללות ברשימת השליחה.

SendBatchAsync מקבלת את הרשומות התקינות ופונקציה שבוחרת StationId כמפתח ההודעה. השירות אינו שומר למסד; הוא מפרסם מידע למעבד. במקרה כשל הבקשה נרשמת בלוג ונזרקת שגיאה שמאפשרת ל־worker לטפל במחזור ככישלון.

המתודה מחזירה את מערך הרשומות שפוענח, אך ה־worker משתמש בה כפעולה אסינכרונית ולא לצורך תצוגה. אין להשתמש בערך המוחזר כהוכחה שכל הרשומות נשמרו ב־MySQL; השמירה מתבצעת בשירות אחר ובזמן אחר.

מתוך קוד המקור

```
BikeFleetOperationPlatform/Services/StationInformationService.cs
039             foreach (var station in stationsInformation)
040             {
041                 if (string.IsNullOrWhiteSpace(station.StationId)) {
      _logger.LogWarning("GBFS station without an ID was skipped."); continue; }
042                 station.StationId = _configuration["DataSource:StationPrefix"] +
      station.StationId;
043                 var validationContext = new ValidationContext(station);
044                 var validationResults = new List<ValidationResult>();
045 
046                 bool isValid = Validator.TryValidateObject(station, validationContext,
      validationResults, validateAllProperties: true);
047 
048                 if (isValid)
049                 {
050                     validStations.Add(station);
051                 }
052                 else
053                 {
054                     var errors = string.Join(", ", validationResults.Select(r =>
      r.ErrorMessage));
055                     _logger.LogWarning("station {Id} failed validation: {errors}",
      station.StationId, errors);
056                 }
057             }
058             await _kafkaProducer.SendBatchAsync(topic, validStations, item =>
      item.StationId, token);
059             return stationsInformation;
```

20. פענוח זמינות וסוגי אופניים

StationStatusService מבקשת station_status.json ומפענחת שדות nullable, כדי לשמר את ההבדל בין חסר לבין אפס. ReceivedAt נקבע פעם אחת למחזור הקליטה ומוצמד לכל רשומה. LastReported נשאר הזמן שדווח על ידי הספק ואינו מוחלף בזמן המקומי.

אותו מנגנון אימות ותחילית מזהה מופעל לפני פרסום. מפתח Kafka הוא מזהה התחנה. JsonPropertyName משמר מיפוי לשמות feed מיוחדים, וממיר הדגלים דואג להתאמה בין boolean לבין מספרים. סימני שאלה בטיפוסים הם החלטה עסקית שמונעת זמינות מדומה.

VehicleTypesService קוראת את data.vehicle_types ומפרסמת לפי VehicleTypeId. היא אינה משתמשת בתחילית תחנה כי מדובר במזהה סוג כלי ולא במזהה תחנה. null במקום מערך נחשב לפענוח לא תקין ומוביל לשגיאה; מערך ריק תקין אינו מחייב יצירת רשומות חלופיות.

שני השירותים משתמשים באותו יצרן Kafka, אך אחריות הפענוח נשארת נפרדת. אין יתרון להכניס את כל ההבדלים האלה למחלקה גנרית גדולה: שלוש פעולות קצרות עם חוזים ברורים קלות יותר להסבר ולשינוי.

מתוך קוד המקור

```
BikeFleetOperationPlatform/Services/StationStatusService.cs
034              var receivedAt = DateTime.UtcNow;
035              StationStatusDto[] stationsStatus =
       document.RootElement.GetProperty("data").GetProperty("stations").EnumerateArray()
036                  .Select(x => x.Deserialize<StationStatusDto>(options)!).ToArray();
037              var validStations = new List<StationStatusDto>();
038              foreach(var station in stationsStatus)
039              {
040                  station.ReceivedAt = receivedAt;
041                  if (string.IsNullOrWhiteSpace(station.StationId)) {
       _logger.LogWarning("GBFS station without an ID was skipped."); continue; }
042                  station.StationId = _configuration["DataSource:StationPrefix"] +
       station.StationId;
043                  var validationContext = new ValidationContext(station);
044                  var validationResults = new List<ValidationResult>();
045
046                  bool isValid = Validator.TryValidateObject(station, validationContext,
       validationResults, validateAllProperties: true);
047
048                  if (isValid)
049                  {
050                      validStations.Add(station);
051                  }
052                  else
```

21. יצרן Kafka והאיחוד לאחר הניקוי

KafkaProducerServices מחזיק IProducer אחד לאורך חיי השירות. הבנאי הציבורי בונה ProducerConfig מכתובת ה־broker. בנאי פנימי מקבל IProducer קיים ומשמש לבדיקה בלי להפעיל broker. Dispose מבצע flush קצר ומשחרר את היצרן.

SendBatchAsync היא הפעולה המשותפת שנוספה בניקוי: היא מקבלת topic, אוסף רשומות ופונקציית בחירת מפתח. כל רשומה מומרת ל־JSON ונשלחת דרך SendToKafkaAsync. Task.WhenAll ממתינה לכל אישורי השליחה, ורק לאחר הצלחה מתבצע flush של עשר שניות.

SendToKafkaAsync מתעדת partition ו־offset שהוחזרו. ProduceException מתועדת ונזרקת מחדש. הצלחת produce אינה מוכיחה שהצרכן כבר שמר למסד; היא שייכת לשלב מסירת ההודעה ל־Kafka. כשל באחת השליחות אינו מוצג כהצלחת האצווה.

הבדיקות בודקות שהמפתח, ה־topic, ה־JSON וה־CancellationToken נשמרים. הן בודקות גם אפס הודעות באצווה ריקה והיעדר flush הצלחה אחרי כשל. אלו בדיקות של המימוש המשותף, ולא רק השוואת מספר השורות שנחסכו.

מתוך קוד המקור

```
BikeFleetOperationPlatform/Services/KafkaProducerService.cs
016       public async Task SendBatchAsync<T>(string topic, IEnumerable<T> items, Func<T,
       string> keySelector,
017           CancellationToken token = default)
018       {
019           var sends = items.Select(item => SendToKafkaAsync(topic, new Message<string,
       string>
020           {
021               Key = keySelector(item),
022               Value = JsonSerializer.Serialize(item)
023           }, token));
024           await Task.WhenAll(sends);
025           _producer.Flush(TimeSpan.FromSeconds(10));
026       }
027
028       public async Task SendToKafkaAsync(string topicName, Message<string,string>
       content, CancellationToken token = default)
029       {
030           try
031           {
032               var result = await _producer.ProduceAsync(topicName, content, token);
033               Console.WriteLine($"delivered station {content.Key} to partition
       {result.Partition.Value} at offset {result.Offset.Value}");
034           }
035           catch (ProduceException<string, string> ex)
036           {
037               Console.ForegroundColor = ConsoleColor.Red;
038               Console.WriteLine($"kafka Error for station {content.Key}:
       {ex.Error.Reason}");
```

22. צריכת מטא־דאטה ו־Handlers

KafkaConsumerBackgroundService אוספת את ה־topics מכל המימושים של IKafkaMessageHandler ונרשמת אליהם. עבור כל הודעה נוצר scope חדש, נבחר handler מתאים לפי topic, ורק אחרי HandleAsync מוצלחת מתבצע Commit. הגדרות AutoCommit ו־AutoOffsetStore כבויות כדי שהקוד ישלוט בנקודת האישור.

StationInformationHandler מפענחת DTO, מחפשת רשומה לפי מזהה ומבצעת Add אם היא חדשה או SetValues אם היא קיימת. לאחר SaveChangesAsync היא משנה את גרסת Redis. כשל Redis מתועד אך אינו מבטל שמירה מוצלחת במאגר העיקרי. VehicleTypesHandler עושה upsert דומה עבור סוג כלי.

כאשר metadata אינו JSON תקין או handler מחזיר false, ההודעה מתועדת כלא תקינה ומאושרת כדי שלא תחסום לנצח את הצרכן. כשל תשתיתי גורם ל־Seek לאותו offset והשהיה לפני ניסיון נוסף. הבחירה לדלג על הודעה לא תקינה היא מגבלה תפעולית: אין בפרויקט תור שגיאות נפרד לשחזור ידני.

IKafkaMessageHandler הוא חוזה קטן של Topic ו־HandleAsync. הוא שימושי כי הצרכן יכול להפעיל מספר סוגי מטא־דאטה בלי להכיר את פרטי השמירה של כל אחד. אין צורך להוסיף framework נוסף לשם כך.

מתוך קוד המקור

```
ProcessingService/Service/KafkaConsumerBackService.cs
027                     message = consumer.Consume(token);
028                     using var scope = scopes.CreateScope();
029                     var handler =
       scope.ServiceProvider.GetServices<IKafkaMessageHandler>().First(x => x.Topic ==
       message.Topic);
030                     if (!await handler.HandleAsync(message.Message.Value)) throw new
       JsonException("Invalid metadata.");
031                     consumer.Commit(message);
032                 }
033                 catch (OperationCanceledException) when (token.IsCancellationRequested)
       { break; }
034                 catch (JsonException ex) { logger.LogError(ex, "Invalid metadata
       discarded."); if (message is not null) consumer.Commit(message); }
035                 catch (Exception ex)
036                 {
037                     logger.LogError(ex, "Metadata processing failed; retrying the same
       message.");
038                     if (message is not null)
       consumer.Seek(message.TopicPartitionOffset);
039                     await Task.Delay(2000, token);
040                 }
041             }
042         }
043         finally { consumer.Close(); }
```

23. אתחול המעבד ואחריות מסדי הנתונים

Program של ProcessingService רושם חיבור Redis משותף, IDatabase, DbContext עבור MySQL, handlers ולקוחות MongoDB. ConsumerConfig של סטטוסים נלקח מההגדרות, ואילו צרכן המטא־דאטה קורא את הגדרותיו בזמן האתחול. שני HostedService פועלים במקביל, כל אחד עם קבוצת צרכנים ואחריות משלו.

StationDbContext מגדיר stationInformation ו־vehicleTypes ומפתחות לפי מזהי הספק. EnsureCreated מופעל לפני RunAsync. קיימים גם קובצי migrations מההתפתחות ההיסטורית של הסכימה; הם נשמרו בניקוי. חשוב להבחין: קיום migrations אינו אומר שהמסלול הנוכחי מפעיל Migrate בכל אתחול.

MongoConsumerService יוצרת אינדקסי היסטוריה בעת עלייה. חיבורי Mongo ו־Redis הם Singleton, בעוד DbContext הוא scoped. כך לא משתפים אובייקט EF שאינו מיועד לגישה מקבילית בין כל ההודעות.

סביבת Compose ממתינה לבריאות התשתיות לפני יצירת המעבד, אבל סדר התנעה אינו הבטחה שלא יתרחש כשל מאוחר יותר. לכן קיימת גם לוגיקת ניסיונות חוזרים בצרכנים. אתחול, צריכה ושמירה הם שלבים נפרדים שיש לבדוק בנפרד כאשר נתונים אינם מגיעים למסך.

מתוך קוד המקור

```
ProcessingService/Program.cs
034  builder.Services.AddHostedService<KafkaConsumerBackgroundService>();
035  builder.Services.AddScoped<IKafkaMessageHandler, StationInformationHandler>();
036  builder.Services.AddScoped<IKafkaMessageHandler, VehicleTypesHandler>();
037
038  var consumerConfig = new ConsumerConfig();
039  builder.Configuration.GetSection("Kafka:Consumer").Bind(consumerConfig);
040  builder.Services.AddSingleton(consumerConfig);
041
042  builder.Services.AddSingleton<IMongoClient>(sp =>
043          new MongoClient(builder.Configuration["MongoDb:ConnectionString"]));
044
045      builder.Services.AddSingleton<IMongoDatabase>(sp =>
046      {
047          var client = sp.GetRequiredService<IMongoClient>();
048          return client.GetDatabase(builder.Configuration["MongoDb:DatabaseName"]);
049      });
050
```

24. אתחול API ו־Dependency Injection

StationApi/Program.cs הוא נקודת הרכבה: חיבורים, DbContext, שירותי ניתוח, מטמון, הרשאות, MVC, rate limiting ו־HTTP client פנימי נרשמים בו. המערכת אינה יוצרת ידנית שירות חדש בכל Controller; ה־container מספק את התלויות בהתאם לאורך החיים שנבחר.

Singleton מתאים לאפשרויות הניתוח, מנוע טהור, חיבור Redis ולנעילות מקומיות. Scoped מתאים ל־DbContext ולשירותים המשתמשים בו. AddHostedService מפעיל ניתוח מחזורי באמצעות scope חדש בכל מחזור. שמירת DbContext בתוך Singleton הייתה מסכנת גישה מקבילית ומעקב ישויות לא נכון.

Middleware מוסיף כותרות אבטחה ומונע מטמון HTTP עבור תוכן אישי. חריגת שמירה ממופה ל־409 ושגיאת שירות אחרת ל־503 כאשר ניתן עדיין לכתוב לתשובה. בנתיבי API מוחזר JSON; במסך עלולה להופיע הודעת שירות כללית. Swagger פעיל בסביבת Development בלבד לפי הקוד.

אתחול המשתמש הראשוני מתבצע רק כאשר Users ריקה. פרטי seed מגיעים מהגדרות, נבדקים ומגובבים. שינוי env. לאחר שכבר קיים חשבון אינו מאפס את סיסמתו. Health בודק MySQL ו־MongoDB, אך אינו בדיקת בריאות מלאה לכל רכיבי Kafka ו־Redis.

מתוך קוד המקור

```
StationApi/Program.cs
060  builder.Services.AddSingleton(fleetOptions);
       builder.Services.AddSingleton<SmartFleetEngine>();
061  builder.Services.AddSingleton<FleetCacheState>();
062  builder.Services.AddScoped<StationStateService>();
063  builder.Services.AddSingleton<OperationsGate>();
       builder.Services.AddScoped<FleetDataService>();
064  builder.Services.AddScoped<MetricsService>();
065  builder.Services.AddScoped<OperationsService>();
       builder.Services.AddHostedService<FleetAnalysisWorker>();
066  builder.Services.AddDataProtection();
067  var app = builder.Build();
068  app.Use(async (ctx, next) =>
069  {
070      ctx.Response.Headers["X-Content-Type-Options"] = "nosniff";
071      ctx.Response.Headers["Referrer-Policy"] = "same-origin";
072      ctx.Response.Headers["Content-Security-Policy"] = "default-src 'self'; script-src
       'none'; style-src 'self'; img-src 'self' data:; connect-src 'self'; frame-ancestors
       'none'; form-action 'self'";
```

25. איחוד מידע ושליפת תחנה בודדת

FleetDataService הוא גבול הקריאה המרכזי בין האחסון לניתוח. GetStations קוראת LoadStations ללא מזהה; GetStation מעבירה מזהה ומחזירה את הרשומה הראשונה או null. השאילתה היחסית מסננת לפי מקור ובמידת הצורך לפי התחנה המבוקשת לפני קריאת MongoDB.

LoadStations בונה רשימת מזהים, מבקשת latest והיסטוריה רק עבורם ומארגנת מילונים לגישה יעילה. עבור כל תחנה היא קוראת ערכים מ־BsonDocument ומייצרת StationInput. פונקציות Value, Number, Flag ו־Text מטפלות בשמות שדות קיימים ובשדות חסרים. מספר שאינו בתחום int אינו הופך לערך שרירותי.

הפרדה זו מתקנת עבודה מיותרת שבה בקשת פרטי תחנה עלולה לחשב את כל הרשת. עם זאת, קריאת כלל התחנות עדיין טוענת היסטוריה לכל הרשימה; בפריסה גדולה אפשר לבחון pagination ושאילתות ממוקדות נוספות. הספר אינו מציג שיפורים עתידיים כקוד שכבר קיים.

Names ושלוש גרסאות WithNames מעשירות התראות, המלצות ודיווחים בשם תחנה מתוך MySQL. הן אינן מפעילות את מנוע הניתוח רק כדי למצוא שם. כאשר שם חסר, מתקבל Station #id במקום כותרת ריקה.

מתוך קוד המקור

```
StationApi/Services/FleetDataService.cs
069      private async Task<List<StationAnalysis>> LoadStations(string? stationId,
       CancellationToken cancellationToken)
070      {
071          var now = DateTime.UtcNow;
072          var prefix = config["DataSource:StationPrefix"]!;
073          var query = sql.stationInformation.AsNoTracking().Where(x =>
       x.StationId.StartsWith(prefix));
074          if (stationId is not null) query = query.Where(x => x.StationId == stationId);
075          var stations = await query.ToListAsync(cancellationToken);
076          if (stations.Count == 0) return [];
077          var ids = stations.Select(x => x.StationId).ToList();
078          var latest = await mongo.GetCollection<BsonDocument>("StationsStatus")
079              .Find(Builders<BsonDocument>.Filter.In("Station_id", ids) |
       Builders<BsonDocument>.Filter.In("station_id", ids))
080              .ToListAsync(cancellationToken);
081          var history = await mongo.GetCollection<BsonDocument>("StationHistory")
082              .Find(Builders<BsonDocument>.Filter.Gte("RecordedAt",
       now.AddMinutes(-options.HistoryMinutes))
083                  & Builders<BsonDocument>.Filter.In("StationId", ids))
084              .ToListAsync(cancellationToken);
085          var currentById = latest.GroupBy(x => Text(x, "Station_id",
       "station_id")).ToDictionary(x => x.Key, x => x.Last());
086          var historyById = history.GroupBy(x => Text(x, "StationId", "station_id"))
087              .ToDictionary(x => x.Key, x => x.Where(d => Number(d, "Count") is >= 0 &&
       d.GetValue("RecordedAt", BsonNull.Value).IsBsonDateTime)
088                  .Select(d => new Reading(d["RecordedAt"].ToUniversalTime(), Number(d,
       "Count")!.Value)).ToList());
089          return stations.Select(s =>
090          {
```

26. FleetController וממשקי הקריאה

FleetController מוגבל למנהל ולעובד. Stations מחזירה ניתוח רשימה דרך המטמון ובתוספת מצב תפעולי. Station מחזירה אובייקט שמכיל analysis, התראות פעילות והמלצות פתוחות הקשורות לתחנה. מזהה שלא נמצא מחזיר 404 עם הודעה ברורה.

History מחזירה מדידות ממוינות בזמן. Status ו־Statistics הם aliases לאותה פעולת קריאה. Dashboard מסכמת תחנות לפי בריאות, כמות זמינה ומקומות החזרה, ובמקביל מחזירה רשימות קצרות של התראות ומשימות. ספירת כמות זמינה משתמשת רק בתחנות עם מידע עדכני.

יש להבדיל בין רשימת התחנות המלאה לבין שש התחנות שנבחרות להצגה בתצוגת העדיפות. ריבוי תחנות אדומות בדשבורד יכול להיות תוצאה של מיון הסיכון, ולא מצב כל הרשת. התצוגה כוללת קישור לרשימה המלאה לצורך בדיקה.

ה־Controller אינו פונה ישירות לספק GBFS. הוא נשען על המידע שנקלט ועל שירותי המערכת. חוזה ה־JSON נשמר בניקוי, כולל aliases קיימים, מפני שלקוח חיצוני עשוי להשתמש בהם גם אם דף מסוים אינו עושה זאת.

מתוך קוד המקור

```
StationApi/Controllers/FleetController.cs
016       public async Task<IActionResult> Station(string id, CancellationToken token)
017       {
018           var station = await data.GetCachedStation(id, token);
019           if (station is null) return NotFound(new { message = "Station not found." });
020           station = (await states.Enrich([station], token))[0];
021           return Ok(new { analysis = station,
022               alerts = await data.WithNames(await db.Alerts.AsNoTracking().Where(x =>
        x.StationId == id && x.Status == "ACTIVE").ToListAsync(token), token),
023               recommendations = await data.WithNames(await
        db.Recommendations.AsNoTracking().Where(x => (x.SourceStationId == id ||
        x.DestinationStationId == id)
024                   && (x.Status == "PENDING" || x.Status ==
        "IN_PROGRESS")).ToListAsync(token), token) });
025       }
026
027       [HttpGet("stations/{id}/history")]
028       public async Task<IActionResult> History(string id, CancellationToken token)
029       {
030           var station = await data.GetCachedStation(id, token);
```

27. מחזור חיי משימה ושמירה

OperationsController מספקת קריאה של התראות והמלצות וכתיבת מצב. שינוי משימה מוגבל ל־IN_PROGRESS, COMPLETED או DISMISSED. מזהה חסר מחזיר 404, ותחנה שאינה שייכת למקור הנוכחי גורמת ל־409. בקשה חוזרת לאותו מצב מחזירה את הרשומה בלי ליצור השלמה נוספת.

לפני תחילת עבודה נבדקת הזמינות שוב מהמאגר: מקור עם מספיק אופניים, יעד עם מקומות החזרה, קיבולת מתאימה ונתונים עדכניים. משימה יכולה להיות אפשרית כשנוצרה וכבר לא אפשרית כאשר המפעיל מתחיל אותה; לכן בדיקה זו אינה כפילות מיותרת.

TaskMeasurements נוצרת או נטענת לפי מזהה ההמלצה. בתחילת עבודה נשמרים StartedAt ומספר האופניים ביעד. בהשלמה נשמר CompletedAt, ובמקביל אפשר ליישב התראות ספק אם המדידה כבר תקינה. UpdatedAt ו־UpdatedBy מתעדים את הפעולה, ואז SaveChangesAsync שומרת ו־Invalidate פוסלת מטמון.

OperationsGate מסדרת כתיבות בתוך מופע API יחיד. היא אינה תחליף לנעילה בין מופעים. השלמה אינה משנה מדידות מקור. אחרי תשובה מוצלחת, מסך Razor מפנה לרשימת המשימות עם סינון לפי הסטטוס החדש ועם הדגשת המזהה שעודכן.

מתוך קוד המקור

```
StationApi/Controllers/OperationsController.cs
077             var measurement = await db.TaskMeasurements.FindAsync([item.Id], token);
078             if (measurement is null) { measurement = new TaskMeasurement {
      RecommendationId = item.Id }; db.TaskMeasurements.Add(measurement); }
079             if (request.Status == "IN_PROGRESS" && measurement.StartedAt is null)
080             {
081                 measurement.StartedAt = DateTime.UtcNow;
082                 var destination = await data.GetStation(item.DestinationStationId,
      token);
083                 measurement.DestinationBefore = destination is { Stale: false } ?
      destination.Station.Count : null;
084             }
085             if (request.Status == "COMPLETED")
086             {
087                 measurement.CompletedAt ??= DateTime.UtcNow;
088                 foreach (var idToRefresh in new[] { item.SourceStationId,
      item.DestinationStationId })
089                     if (await data.GetStation(idToRefresh, token) is { } current) await
      states.ReconcileProviderAlerts(current, token);
090             }
091             item.Status = request.Status; item.UpdatedAt = DateTime.UtcNow;
092             item.UpdatedBy = int.Parse(User.FindFirstValue(ClaimTypes.NameIdentifier)!)
      ;
093             // Completion records the operator's action. Telemetry remains owned by the
      data source.
094             await db.SaveChangesAsync(token);
095             await data.Invalidate(token);
096             return Ok(item);
097         }
098         finally { gate.Semaphore.Release(); }
099     }
```

28. יצירת התראות והמלצות ברקע

OperationsService.Refresh פועלת תחת OperationsGate. היא טוענת ניתוח תחנות, בודקת התראות פעילות ותחנות שטופלו לאחרונה, ויוצרת או מעדכנת התראות לפי מצב הספק. תחנה בריאה יכולה להביא ליישוב התראה, ואילו תקופת המתנה מונעת יצירה חוזרת מיידית אחרי פתרון ידני.

השלב הבא בודק המלצות PENDING מול הזמינות הנוכחית. המלצה שכבר אינה אפשרית מסומנת DISMISSED עם סיבה. משימה IN_PROGRESS אינה מבוטלת אוטומטית באותו מסלול, מפני שעבודה פיזית יכולה כבר להיות בעיצומה. לאחר מכן נבנית קבוצת תחנות שמורות מהפעולות הפתוחות ומהפעולות האחרונות.

מנוע Recommend מקבל את הרשימה ואת התחנות השמורות, והצעות חדשות נשמרות כ־Recommendation. לאחר שמירת הנתונים מופעל MetricsService.Observe לאיסוף תצפיות. FleetAnalysisWorker חוזר על התהליך במחזור של 30 שניות, מתעד כשל ומנסה שוב במחזור הבא.

יש כאן שתי תדירויות שונות: קליטת מצב מהמקור וניתוח תפעולי. ריצה נוספת של הניתוח אינה בהכרח קריאה חיצונית חדשה. בבדיקה של נתון ישן צריך לעקוב אחר ReportedAt ולא רק אחר תדירות ה־worker.

מתוך קוד המקור

```
StationApi/Services/OperationsService.cs
055             var recent = await db.Recommendations.Where(x => x.UpdatedAt >
       now.AddMinutes(-options.RecommendationCooldownMinutes)).ToListAsync(token);
056             var reserved = pending.Where(x => x.Status is "PENDING" or
       "IN_PROGRESS").Concat(recent)
057                 .SelectMany(x => new[] { x.SourceStationId, x.DestinationStationId
       }).ToHashSet();
058             foreach (var move in engine.Recommend(stations, reserved))
059                 db.Recommendations.Add(new Recommendation { SourceStationId =
       move.SourceStationId,
060                     DestinationStationId = move.DestinationStationId, Quantity =
       move.Quantity, Priority = move.Priority, Reason = move.Reason });
061             await db.SaveChangesAsync(token);
062             await metrics.Observe(stations, token);
063         }
064         finally { gate.Semaphore.Release(); }
065     }
```

29. סגירת דיווחי רוכבים

RiderIssuesController מיועדת ל־ADMIN ול־OPERATOR. All מחזירה רשימת דיווחים עם שמות תחנות. Resolve טוענת את הדיווח, מוודאת שהתחנה שייכת למקור המוגדר, ושומרת RESOLVED, זמן סגירה ומזהה המטפל. אם הדיווח כבר סגור, הפעולה נשארת עקבית ואינה משכתבת את מועד הסגירה.

הפעולה מתבצעת תחת אותו שער כתיבה של התפעול. לפני SaveChangesAsync אפשר ליישב התראות ספק ידועות על תחנה שהפכה בריאה. לאחר שמירה מתבצעת פסילת מטמון. חישוב מספר הבעיות הפתוחות מבוצע בנפרד, ולכן סגירת דיווח אחד אינה מעלימה דיווח אחר.

RiderApiController מספקת לרוכב את הדיווחים האישיים בלבד לפי UserId מה־Claims. Report בודקת תיאור, קיום תחנה ומכסת עשרה דיווחים בשעה. התיאור נשמר כטקסט ו־Razor מקודד אותו בהצגה, במקום להתייחס אליו כ־HTML.

במבחן יש להציג את נתיב האחריות: הרוכב יוצר דיווח, העובד פותר, והמסך מציג את המצב המעודכן. אין לרוכב endpoint מורשה לפתרון דיווחים. המערכת אינה מאמתת אוטומטית שתיקון פיזי בוצע; היא מתעדת את הצהרת המטפל.

מתוך קוד המקור

```
StationApi/Controllers/RiderApiController.cs
057      public async Task<IActionResult> Resolve(long id, CancellationToken token)
058      {
059          await gate.Semaphore.WaitAsync(token);
060          try
061          {
062              var issue = await db.RiderIssues.FindAsync([id], token);
063              if (issue is null) return NotFound();
064              var station = await data.GetStation(issue.StationId, token);
065              if (station is null) return Conflict(new { message = "Station is outside
       the configured source." });
066              if (issue.Status != "RESOLVED")
067              { issue.Status = "RESOLVED"; issue.ResolvedAt = DateTime.UtcNow;
       issue.ResolvedBy = int.Parse(User.FindFirstValue(ClaimTypes.NameIdentifier)!); await
       states.ReconcileProviderAlerts(station, token); await db.SaveChangesAsync(token); }
068              await data.Invalidate(token);
069              return Ok(issue);
070          }
071          finally { gate.Semaphore.Release(); }
072      }
073  }
```

30. מדידת תועלת ודיוק

EmptyMinutes מחברת פערים בין קריאות אפס עוקבות רק כאשר הפער חיובי ואינו גדול מסף הטריות. זמן ללא מידע אינו נספר כאילו ידוע שהתחנה הייתה ריקה. הסכום הוא station-minutes: שתי תחנות ריקות במשך חמש דקות תורמות יחד עשר דקות תחנה.

Observe יוצרת תחזית לכמות בעוד 30 דקות מתוך הדיווח בפועל והקצב. כאשר מגיעה היסטוריה מתאימה, היא מחפשת דגימה בחמש הדקות שלאחר זמן היעד ושומרת ActualCount. Summary מחשבת MAE כממוצע הערכים המוחלטים של ההפרשים. המדד נמדד באופניים ואינו אחוז דיוק.

זמן תגובה הוא הזמן בין יצירת המלצה ל־StartedAt. שיעור השלמה מחושב מתוך המלצות שנוצרו בשבעת הימים האחרונים. שיפור יעד מושווה בין קריאה בתחילת משימה לקריאה 15 עד 20 דקות אחרי השלמה. כאשר אין מדגמים, מוצג null או מקף במקום מספר מטעה.

יש להציג גם את מספר המדגמים. ממוצע על שתי משימות אינו שקול לממוצע על מאות. שיפור לאחר משימה אינו הוכחה לסיבתיות: נסיעות רוכבים, החזרות ופעולות אחרות משפיעות על אותן מדידות.

מתוך קוד המקור

StationApi/Services/MetricsService.cs

```
008       public static double EmptyMinutes(IEnumerable<Reading> history, int maxGapMinutes)
009       {
010           var readings = history.OrderBy(x => x.At).DistinctBy(x => x.At).ToList();
011           double total = 0;
012           for (int i = 1; i < readings.Count; i++)
013           {
014               var gap = (readings[i].At - readings[i - 1].At).TotalMinutes;
015               if (gap > 0 && gap <= maxGapMinutes && readings[i - 1].Count == 0 &&
        readings[i].Count == 0) total += gap;
016           }
017           return Math.Round(total, 1);
018       }
019       public async Task Observe(List<StationAnalysis> stations, CancellationToken token)
020       {
```

31. התחברות, הרשמה ויציאה

AuthController מרכזת את מחזור ההזדהות. Login מנרמלת דוא״ל, מחפשת משתמש, בודקת Enabled ומוודאת את הסיסמה באמצעות PasswordHasher. במקרה הצלחה נוצרות Claims של מזהה, שם, תפקיד וגרסת session, ונשלח cookie. משתמש או סיסמה שגויים מקבלים הודעה כללית שאינה חושפת איזה פרט היה שגוי.

RegisterRequest מגדירה אילוצים על שם, דוא״ל וסיסמה. Register מנרמלת דוא״ל, בודקת כפילות וקובעת RIDER בשרת. הסיסמה מגובבת לפני SaveChangesAsync. הרשמה אינה מבצעת כניסה אוטומטית, ולכן המסך מפנה לכניסה לאחר יצירת החשבון.

Me מחזירה UserView, שמכיל פרטים שמותר להציג ללא PasswordHash. Logout מבטלת את ה־cookie ומחזירה 204. טפסי Razor מעבירים את ה־cookie והתשובות דרך FleetApiClient. אין לשמור cookies של משתמשים ב־HttpClient משותף.

הקוד משתמש ב־HttpOnly, SameSite Strict ובתוקף של שמונה שעות ללא sliding expiration. לפריסה ציבורית נדרשת התאמת HTTPS ואבטחת תשתית. אין להציג את סביבת localhost כפריסת ייצור קשיחה שכבר הוקשחה לכל איום.

מתוך קוד המקור

```
StationApi/Controllers/AuthController.cs
047      public async Task<IActionResult> Register(RegisterRequest request)
048      {
049          var email = request.Email.Trim().ToLowerInvariant();
050          if (await db.Users.AnyAsync(x => x.Email == email)) return Conflict(new {
       message = "This email is already registered." });
051          var user = new FleetUser { Name = request.Name.Trim(), Email = email, Role =
       "RIDER" };
052          user.PasswordHash = passwords.HashPassword(user, request.Password);
053          db.Users.Add(user);
054          await db.SaveChangesAsync();
055          return Ok(new { message = "Account created. You can sign in now." });
056      }
057      [HttpGet("me"), Authorize]
058      public async Task<IActionResult> Me() => Ok(UserView.From((await
       db.Users.FindAsync(int.Parse(User.FindFirstValue(ClaimTypes.NameIdentifier)!)))!));
059      [HttpPost("logout"), Authorize]
```

32. CSRF, הגבלת קצב ושמירת פרטיות

AutoValidateAntiforgeryToken מופעל על פעולות MVC. טפסים נוצרים עם token, ו־FleetApiClient מעביר אותו בכותרת X-CSRF-TOKEN יחד עם cookies של הבקשה. לקוח API שאינו טופס נדרש לקבל token דרך api/auth/csrf/ ולהחזיר אותו בכתיבה. כך בקשת POST מאתר זר אינה נחשבת אוטומטית לפעולה מורשית רק בגלל cookie קיים.

Rate limiter מגביל כניסה והרשמה לעשרה ניסיונות בדקה לפי כתובת הבקשה. מכיוון ש־Razor פונה ל־API דרך HTTP פנימי, צריך לבחון את משמעות כתובת הלקוח בפריסה עם proxy או מספר משתמשים; זו אינה הוכחה להגנה מבוזרת מלאה מפני מתקפות.

Content-Security-Policy חוסמת סקריפטים בממשק הרגיל, וכותרת no-store מונעת מטמון HTTP של תוכן אישי. Redis שומר ניתוח תחנות משותף ולא נתוני חשבונות. סיסמאות ופרטי seed אינם מופיעים בספר ובצילומים.

הבדיקות המקומיות בודקות דחיית טופס ללא CSRF, חסימת רוכב מנתיבים תפעוליים וחסימת עובד מניהול משתמשים. בדיקות כאלה משלימות, אך אינן מחליפות, סקירת אבטחה של סביבת ההרצה בפועל.

מתוך קוד המקור

```
StationApi/Program.cs
043  builder.Services.AddRateLimiter(o =>
044  {
045      o.RejectionStatusCode = 429;
046      o.AddPolicy("login", context =>
       RateLimitPartition.GetFixedWindowLimiter(context.Connection.RemoteIpAddress?.ToString
       () ?? "local",
047          _ => new FixedWindowRateLimiterOptions { PermitLimit = 10, Window =
       TimeSpan.FromMinutes(1), QueueLimit = 0 }));
048  });
049  builder.Services.AddControllersWithViews(o => o.Filters.Add(new
       AutoValidateAntiforgeryTokenAttribute()));
050  builder.Services.AddHttpContextAccessor();
051  builder.Services.AddHttpClient("FleetApi", client =>
052  {
053      client.BaseAddress = new Uri(builder.Configuration["ApiBaseUrl"] ??
       "http://localhost:5138");
054      client.Timeout = TimeSpan.FromSeconds(20);
055  }).ConfigurePrimaryHttpMessageHandler(() => new HttpClientHandler { UseCookies = false,
       AllowAutoRedirect = false });
```

33. שכבת Razor ולקוח API פנימי

WorkspaceController ו־RiderController מחזירות Views ולא תשובות JSON עסקיות. לפני פעולות מוגנות הן מבקשות /api/auth/me, מגדירות CurrentUser ומשתמשות בתפקיד להפניה המתאימה. שגיאות ApiFailure מהפעולה הופכות למסך Error עם קוד HTTP מתאים.

FleetApiClient.Send בונה HttpRequestMessage, מעביר cookies של המשתמש הנוכחי ומצרף JSON כאשר נשלח body. לקוח ה־HTTP אינו מנהל CookieContainer משותף. Set-Cookie מתשובת ה־API מועבר לתשובת הדפדפן כדי שכניסה ויציאה יעבדו דרך שכבת ה־Views.

כאשר תשובת ה־API אינה מוצלחת, הלקוח מנסה לקרוא message או errors ולהציג הסבר. בקשה מוצלחת ללא גוף מחזירה JsonElement ברירת מחדל. קוד ה־Views צריך להכיר את מבנה התשובה שהוא מציג ולא לקרוא שדות שאינם קיימים.

לארכיטקטורה זו יש מחיר של בקשת HTTP פנימית נוספת. היא נשמרה כדי לא לשנות את מבנה המערכת במהלך הניקוי. אין להציג אותה כבחירה היחידה האפשרית; חלופה עתידית היא שירות אפליקטיבי משותף, אך מעבר כזה דורש בדיקת הרשאות, cookies וחוזי API מחדש.

מתוך קוד המקור

```
StationApi/Services/FleetApiClient.cs
015       public async Task<JsonElement> Send(string path, HttpMethod? method = null, object?
       body = null)
016       {
017           var context = accessor.HttpContext!;
018           using var request = new HttpRequestMessage(method ?? HttpMethod.Get, path);
019           if (context.Request.Headers.TryGetValue("Cookie", out var cookies))
       request.Headers.TryAddWithoutValidation("Cookie", cookies.ToString());
020           if (context.Request.HasFormContentType)
021           {
022               var form = await context.Request.ReadFormAsync();
023               request.Headers.TryAddWithoutValidation("X-CSRF-TOKEN",
       form["__RequestVerificationToken"].ToString());
024           }
025           if (body is not null) request.Content = JsonContent.Create(body);
026           using var response = await factory.CreateClient("FleetApi").SendAsync(request,
       context.RequestAborted);
027           if (response.Headers.TryGetValues("Set-Cookie", out var setCookies))
028               foreach (var cookie in setCookies)
       context.Response.Headers.Append("Set-Cookie", cookie);
029           string json = await response.Content.ReadAsStringAsync(context.RequestAborted);
030           if (!response.IsSuccessStatusCode)
031           {
032               string message = response.StatusCode switch { HttpStatusCode.Unauthorized
       => "Please sign in to continue.",
033                   HttpStatusCode.Forbidden => "You do not have permission for this
       action.",
```

34. עיצוב, רענון ותצוגות משותפות

_Layout אחראית למעטפת האתר: כותרת מסמך, CSS, תפריט לפי תפקיד, שם משתמש, התנתקות והודעות הצלחה. meta refresh מופעל רק בדפי קריאה מתאימים. פרמטר refresh=off מאפשר להשהות אותו בזמן חיפוש; טפסי הרשמה, עריכה ודיווח אינם מתרעננים אוטומטית.

_StationTable מציגה שמות תחנות, מזהים משניים, זמינות, מצב איכות וזמנים. _Recommendations מציגה כמות, מקור ויעד, סיבה, סטטוס וקישורי פעולה. התצוגה המשותפת מונעת הבדל בין משימה שמופיעה בדשבורד לבין אותה משימה בדף התחנה.

site.css מגדיר צבעי סיכון, כרטיסים, טבלאות ותפריט. Media queries משנים את פריסת העמוד במסכים קטנים. גרף ההיסטוריה הוא SVG שנוצר בצד השרת מרשימת קריאות; אין ספריית JavaScript לגרפים.

ViewText מרכז קריאה בטוחה של JSON, תאריכים, שמות ומצבי שירות. DurationText מעגלת דקות חיוביות כלפי מעלה ומחלקת לשעות ולשארית. כך 61 דקות מוצגות כשעה ודקה, וערך חסר אינו מוצג כאפס. אחידות זו חוסכת תנאים כפולים בכל View.

מתוך קוד המקור

```
StationApi/Models/DurationText.cs
005      public static string Format(double? minutes, string missing = "Not predicted")
006      {
007          if (minutes is null || !double.IsFinite(minutes.Value) || minutes < 0 ||
       minutes > int.MaxValue) return missing;
008          long total = (long)Math.Ceiling(minutes.Value);
009          long hours = total / 60, remainder = total % 60;
010          string part(long value, string unit) => $"{value} {unit}{(value == 1 ? "" :
       "s")}";
011          if (hours == 0) return part(remainder, "minute");
012          return part(hours, "hour") + (remainder == 0 ? "" : " " + part(remainder,
       "minute"));
013      }
014  }
```

35. API ותיק ותאימות

StationController וה־Repository משמרים את הנתיב api/Station/ לצד API התחנות החדש. GetAllFilteredAsync מקבלת מינימום אופניים ודגלי שירות, קוראת MongoDB ומצרפת מטא־דאטה מ־MySQL. GetByIdAsync מחזירה AllDataDto או 404. הנתיב מוגבל גם הוא למנהל ולעובד.

ה־DTO של הרשימה ושל התחנה הבודדת שונים מ־StationAnalysis החדש. חלק משמות השדות ההיסטוריים מכילים איות לא אידאלי, אך שינוי שלהם עלול לשבור JSON, שאילתות או נתונים קיימים. לכן הניקוי אינו שינוי גורף של כל השמות שנראים לא אחידים.

StationRepository משתמשת במיפוי fallback לשמות שדות ומונעת חיפוש תחנה ממקור אחר. Number ו־Timestamp ממירות ערכי BSON בלי להמציא אפס עבור null. המיזוג היחסי מבוצע על רשימת מזהים אחת ולא בשאילתת שם נפרדת לכל רשומה.

הלקח ההנדסי הוא שקוד שאינו מופעל מהמסך הראשי אינו בהכרח קוד מת. לפני מחיקת API יש לוודא שאין צרכנים אחרים או להגדיר תהליך הפסקת תמיכה. בספר מתועד הנתיב כדי שהבוחן יוכל להבין מדוע ה־Controller נשמר.

מתוך קוד המקור

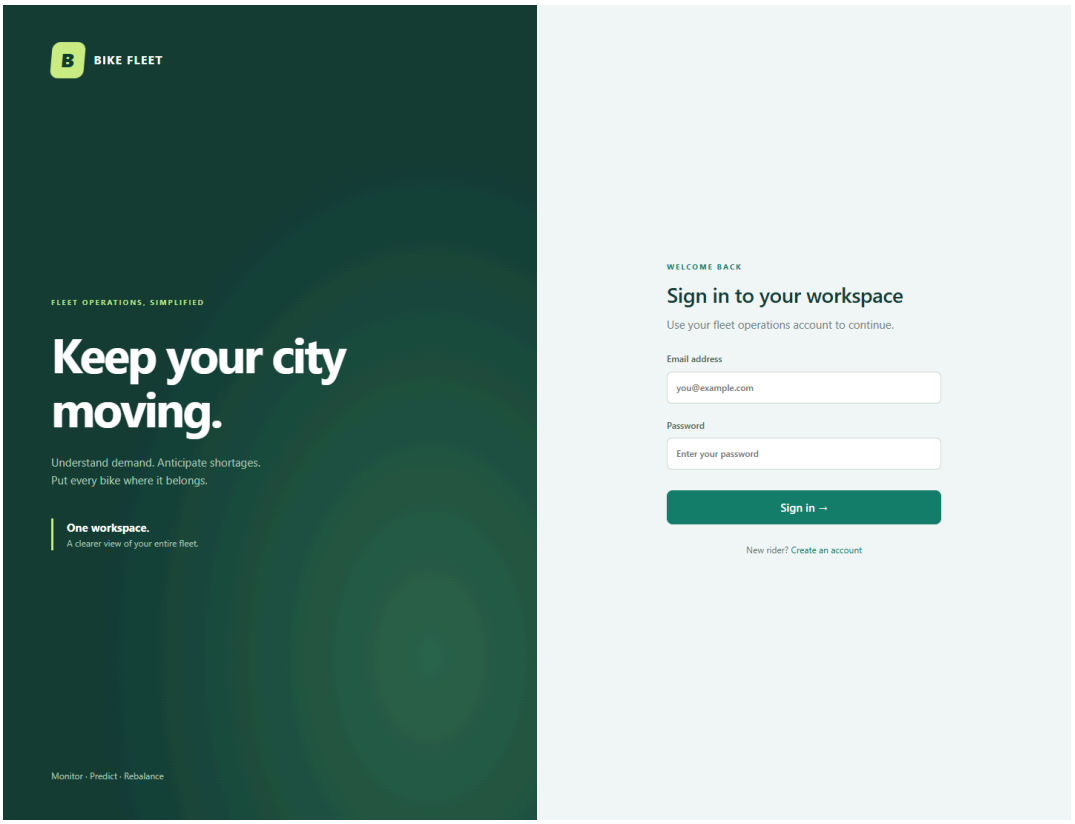
```
StationApi/Controllers/StationController.cs
016       [HttpGet]
017       public async Task<ActionResult<IEnumerable<GetFilterDto>>>
        GetAllFilteredAsync([FromQuery] int? minAvailableBike, [FromQuery] int? inRenting,
        [FromQuery] int? isReturning)
018       {
019           var result = await _repo.GetStationAsync(minAvailableBike, inRenting,
        isReturning);
020           return Ok(result);
021       }
022       [HttpGet("{id}")]
023       public async Task<ActionResult<AllDataDto>> GetByIdAsync(string id)
024       {
025           var result = await _repo.GetAllDataByIdAsync(id);
026           if(result is null)
027           {
028               return NotFound();
029           }
030           return Ok(result);
031       }
032   }
```

מסכים 01 | כניסה למערכת

זהו השער למערכת. המשתמש מקליד דוא״ל וסיסמה ולוחץ Sign in. קישור Create an account מוביל להרשמת רוכב. הודעת כשל מוצגת בתוך הטופס, ושדה הסיסמה מוסווה. אין במסך בחירת תפקיד; התפקיד נקבע מהחשבון שנשמר במסד.

ה־View שולחת POST עם token נגד CSRF. ה־Controller מעביר את הנתונים ל־API ומפנה לדשבורד לאחר הצלחה. רוכב מנותב בהמשך לאזור הרוכבים. Get של Login רק מציג את הטופס; Post מבצע אימות. אין להכניס לספר סיסמת מנהל אמיתית.

מסלול הפעולה: AuthController.Login → WorkspaceController.Login.



מתוך קוד המקור

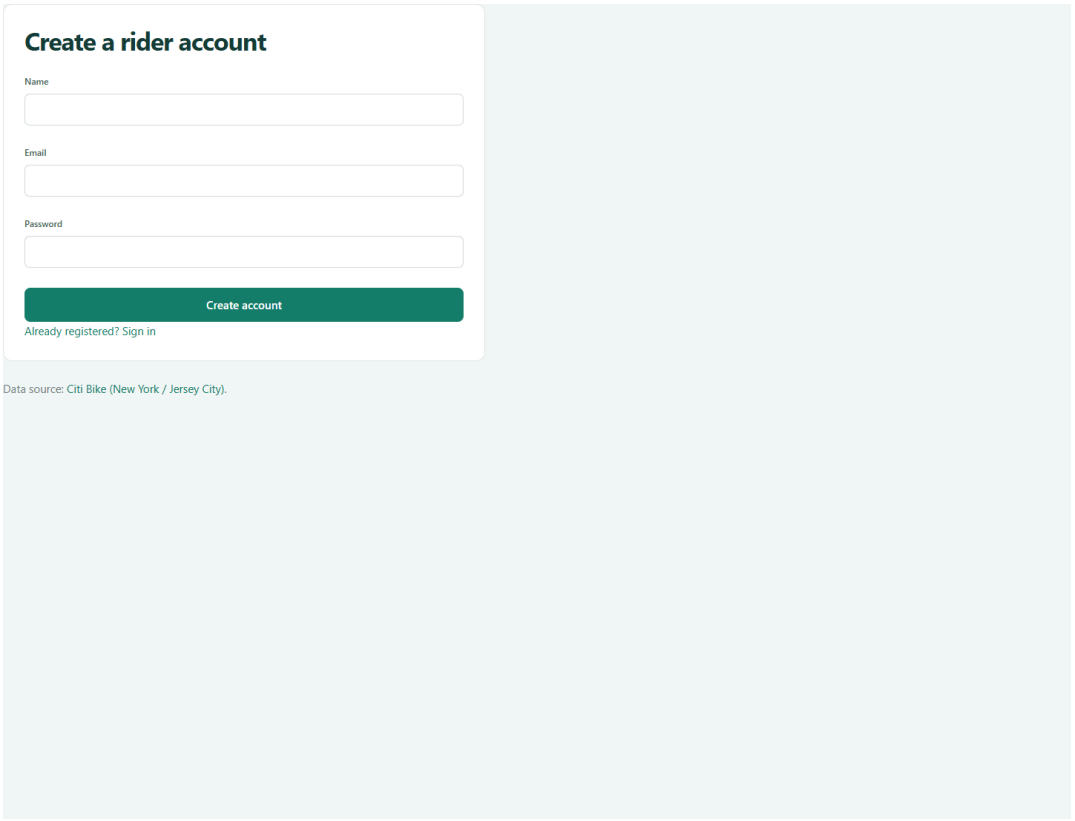
```
StationApi/Views/Workspace/Login.cshtml
001  @{ ViewData["Title"] = "Sign in"; }
002  <div class="login-grid">
003      <section class="login-story"><a class="brand" href="/"><span
       class="brand-icon">B</span>BIKE FLEET</a><div><span class="eyebrow">FLEET OPERATIONS,
       SIMPLIFIED</span><h1>Keep your city<br />moving.</h1><p>Understand demand. Anticipate
       shortages.<br />Put every bike where it belongs.</p><div
       class="login-stat"><strong>One workspace.</strong><span>A clearer view of your entire
       fleet.</span></div></div><small>Monitor · Predict · Rebalance</small></section>
004      <section class="login-panel"><div><span class="eyebrow">WELCOME BACK</span><h2>Sign
       in to your workspace</h2><p class="muted">Use your fleet operations account to
       continue.</p>
```

מסכים 02 | הרשמת רוכב

הטופס כולל שם, דוא״ל וסיסמה. הודעות validation מופיעות לצד השדות וסיכום שגיאות מופיע בראש הטופס. לחיצה על Create account יוצרת חשבון RIDER בלבד, ולא חשבון צוות. לאחר הרשמה מוצלחת מופיעה הפניה לכניסה.

asp-for מחבר את שדות השם והדוא״ל ל־RegisterRequest. הסיסמה אינה מוחזרת לערך השדה אחרי שגיאה. minlength ו־required עוזרים למשתמש בדפדפן, אך השרת מפעיל DataAnnotations גם אם לקוח עוקף את בדיקות ה־HTML.

מסלול הפעולה: AuthController.Register → WorkspaceController.Register.



מתוך קוד המקור

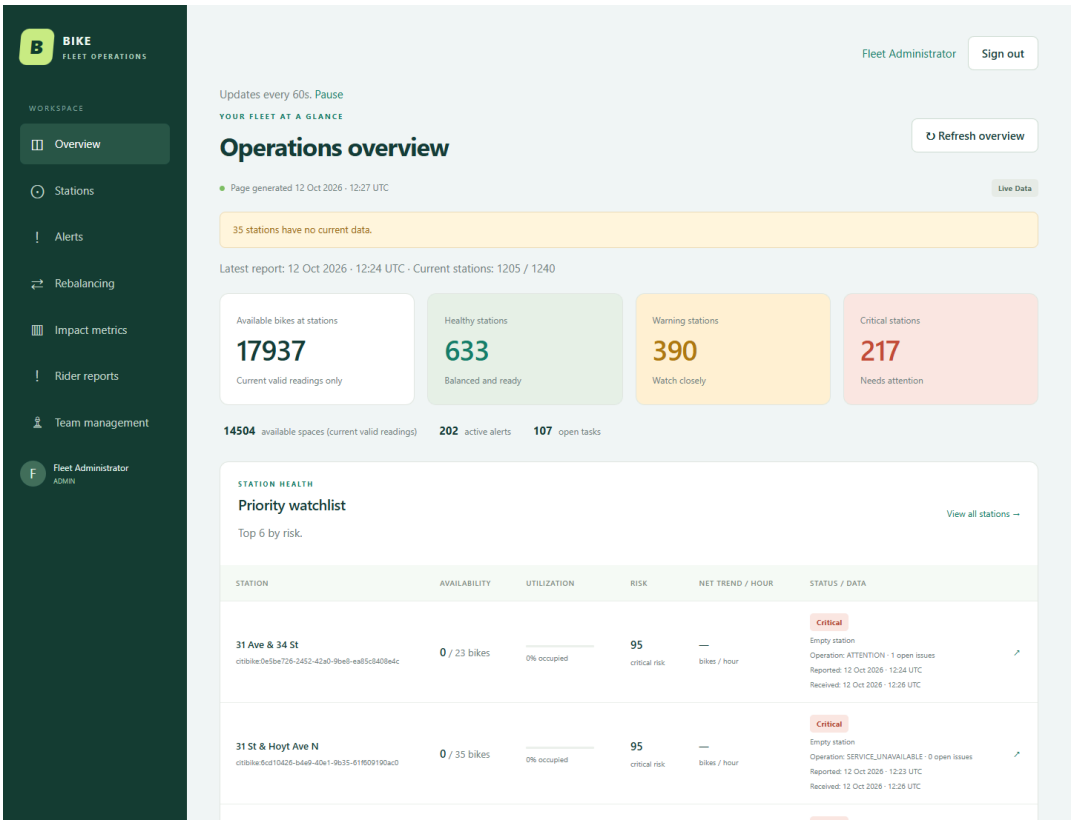
```
StationApi/Views/Workspace/Register.cshtml
001  @model StationApi.Controllers.RegisterRequest
002  @{ ViewData["Title"] = "Rider registration"; }
003  <section class="panel padded narrow"><h1>Create a rider account</h1>
004  <div asp-validation-summary="ModelOnly" class="notice warning" role="alert"></div>
```

מסכים 03 | סקירת מצב הצי

הדשבורד מציג כמות זמינה, תחנות בריאות, אזהרות, תחנות קריטיות, התראות ומשימות. למעלה מופיעים זמן יצירת הדף וזמן דיווח המקור. רשימת Priority watchlist מציגה את שש התחנות בסיכון הגבוה ביותר; Recommended moves מציגה הצעות פעולה.

בתצלום המתועד התקבל מצב של נתונים לא עדכניים בכל התחנות. לכן כמות זמינה מוצגת כמקף והתחנות מסומנות באזהרה. זו תמונת המערכת בזמן הצילום, ולא הוכחה שאין אופניים. ה־View מציגה נתוני JSON ו־partials; החישוב עצמו נמצא בשירותים.

מסלול הפעולה: WorkspaceController.Dashboard → FleetController.Dashboard.



מתוך קוד המקור

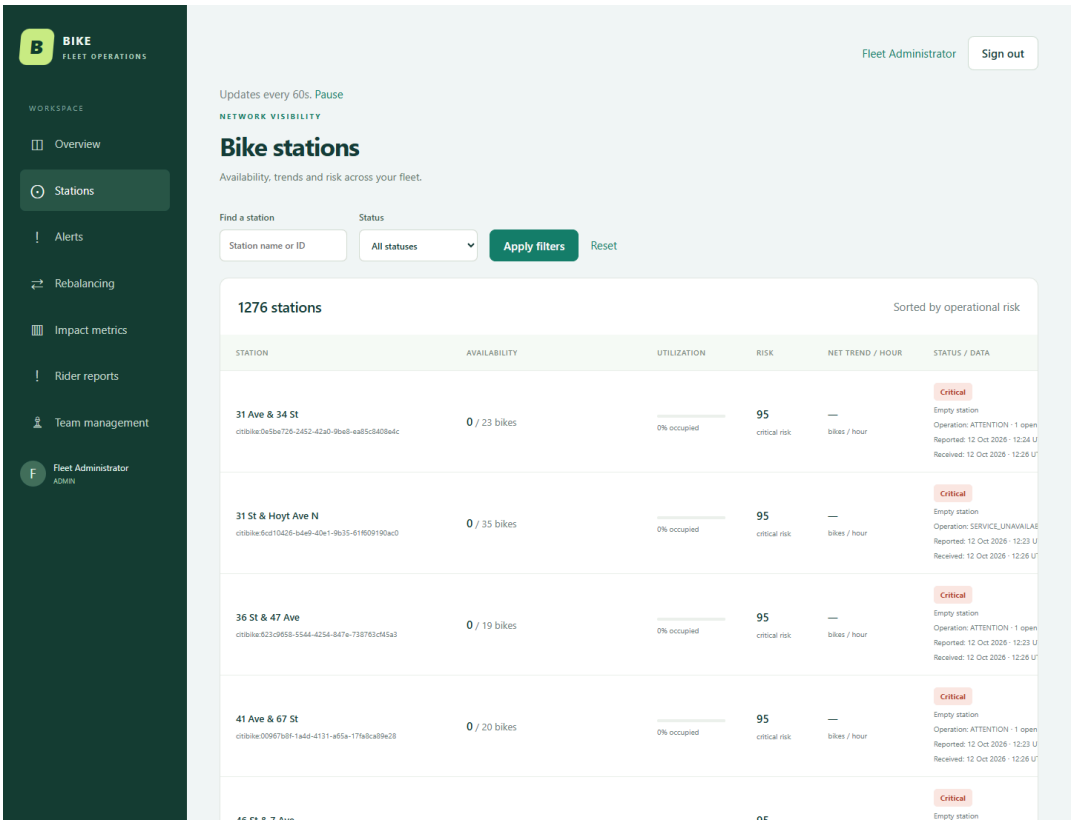
```
StationApi/Views/Workspace/Dashboard.cshtml
001  @model JsonElement
002  @{ ViewData["Title"] = "Overview"; var stations =
       Model.GetProperty("stations").EnumerateArray().ToList(); var moves =
       Model.GetProperty("recommendations").EnumerateArray().Take(2); }
003  <header class="page-header"><div><span class="eyebrow">YOUR FLEET AT A
       GLANCE</span><h1>Operations overview</h1></div><a class="button secondary"
       asp-action="Dashboard">↻ Refresh overview</a></header>
004  <div class="context-bar"><span><span class="live-dot"></span>Page generated
       @Model.Time("updatedAt")</span><span class="badge neutral">@Model.Text("dataMode")
       data</span></div>
```

מסכים 04 | רשימת תחנות וחיפוש

רשימת התחנות מאפשרת חיפוש לפי שם או מזהה וסינון לפי סטטוס. כל שורה מציגה שם, מזהה, זמינות, תפוסה, סיכון, מגמה וזמני דיווח וקבלה. לחיצה על התחנה פותחת פירוט. הבחירה בשמות ראשיים ובמזהים משניים מונעת צורך לזכור UUID.

Stations מקבלת את הרשימה ומסננת אותה לפי _StationTable. query parameters אחראית לרינדור המשותף. אין כאן pagination של השרת; ברשת גדולה כל הנתונים מוחזרים לפני סינון התצוגה. השהיית refresh מאפשרת לעיין בתוצאות בלי החלפת הדף באמצע.

מסלול הפעולה: WorkspaceController.Stations → FleetController.Stations.



מתוך קוד המקור

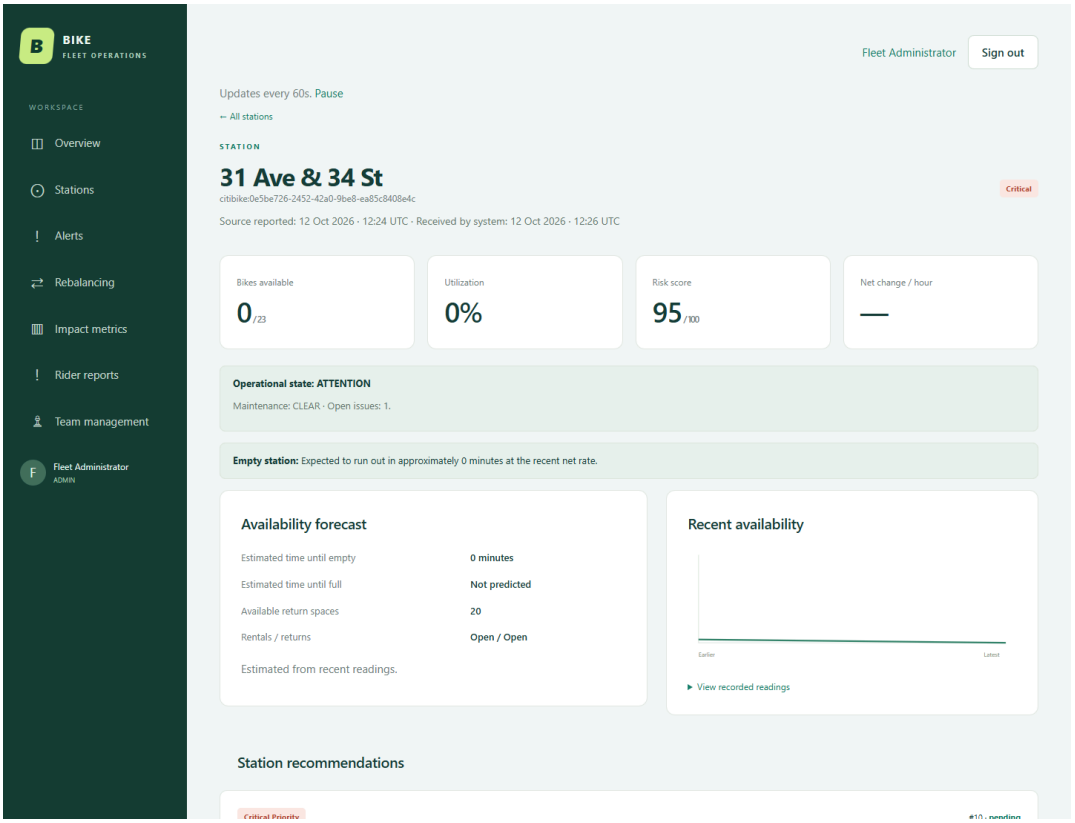
```
StationApi/Views/Workspace/Stations.cshtml
001  @model List<JsonElement>
002  @{ ViewData["Title"] = "Stations"; }
003  <header class="page-header"><div><span class="eyebrow">NETWORK
       VISIBILITY</span><h1>Bike stations</h1><p>Availability, trends and risk across your
       fleet.</p></div></header>
004  <form method="get" class="filters"><label>Find a station<input name="search"
       value="@ViewBag.Search" placeholder="Station name or ID"
       /></label><label>Status<select name="status"><option value="">All
       statuses</option>@foreach(var status in new[]{"HEALTHY","WARNING","CRITICAL"}) {
       <option value="@status" selected="@(ViewBag.Status ==
       status)">@ViewText.Label(status)</option> }</select></label><button
       type="submit">Apply filters</button><a asp-action="Stations">Reset</a></form>
```

מסכים 05 | פרטי תחנה והיסטוריה

הדף מרכז את מצב התחנה: זמינות, תפוסה, ציון סיכון וקצב נטו, מצב תפעולי, תחזית וגרף קריאות. מתחת מופיעות המלצות והתראות הקשורות לתחנה. יש להשוות בין זמן המקור לבין זמן הקבלה לפני החלטה על העברה.

ה־View בונה polyline ב־SVG מתוך ההיסטוריה הממוינת. דף זה משתמש בשאילתה ממוקדת למזהה ולא מחשב תחילה את כל התחנות. נתונים ישנים גורמים לתצוגת Unknown או Not predicted. סגירת התראה עשויה לשנות מצב תפעולי בלי לשנות את המדידה החיצונית.

מסלול הפעולה: WorkspaceController.Station → FleetController.Station.



מתוך קוד המקור

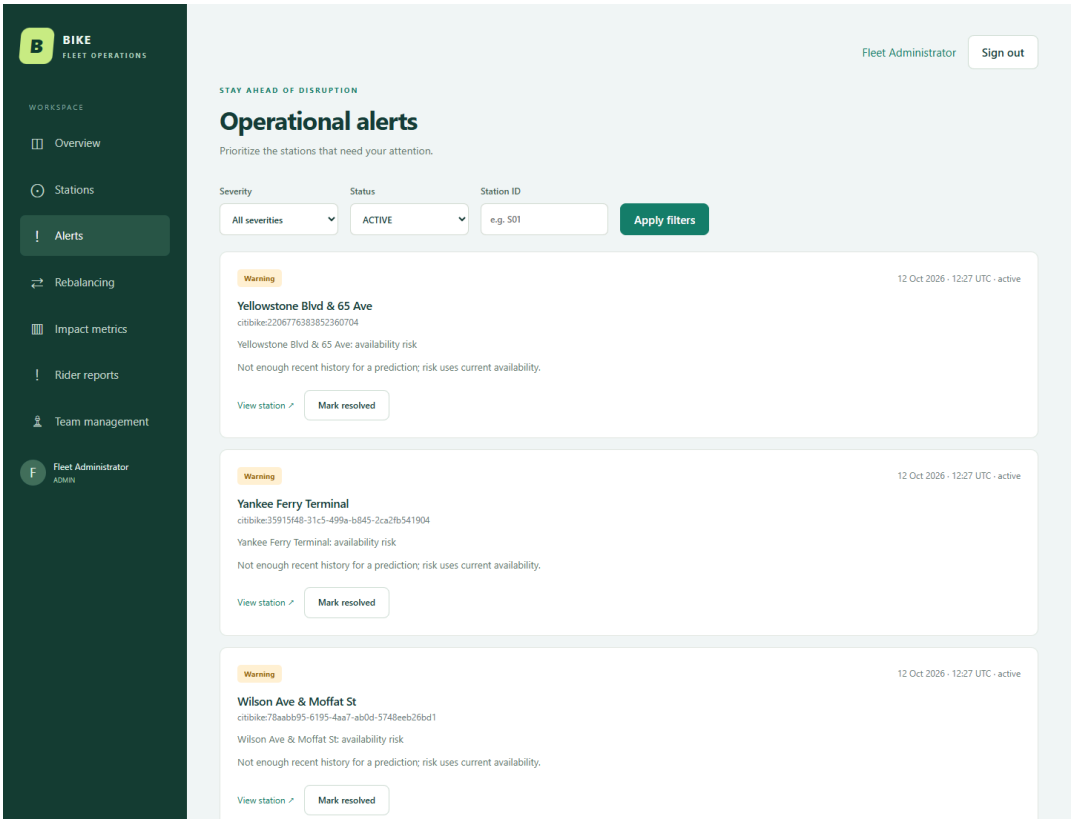
```
StationApi/Views/Workspace/Station.cshtml
001  @model JsonElement
002  @{ var moves = Model.GetProperty("recommendations").EnumerateArray(); var analysis =
       Model.GetProperty("analysis"); var station = analysis.GetProperty("station");
       ViewData["Title"] = station.StationName(); var history =
       station.GetProperty("history").EnumerateArray().OrderBy(x => x.Text("at")).ToList();
       }
003  <a class="back-link" asp-action="Stations">← All stations</a>
004  <header class="page-header"><div><span
       class="eyebrow">STATION</span><h1>@station.StationName()</h1><small
       class="muted">@station.Text("id")</small><p>Source reported:
       @station.Time("reportedAt") · Received by system:
       @station.Time("receivedAt")</p></div><span class="badge
       @analysis.Text("status").ToLowerInvariant()">@ViewText.Label(analysis.Text("status"))
       </span></header>
```

מסכים 06 | התראות תפעוליות

המסך מציג התראות עם חומרה, סטטוס, תחנה ותיאור. ניתן לסנן לפי חומרה, מצב ותחנה. קישור לפתרון מעביר למסך אישור, ולא שולח כתיבה באמצעות GET. שם התחנה משלים את המזהה הטכני.

פעולת הרשימה נבנית ב־EF כשאילתה עם תנאים אופציונליים ומוגבלת ל־500 רשומות. Resolve שומרת RESOLVED, זמן ומבצע ואז פוסלת מטמון. אם הספק ממשיך לדווח על בעיה, לאחר תקופת ההמתנה עשויה להיווצר התראה חדשה; זו רשומה חדשה ולא ביטול של פעולת המשתמש.

מסלול הפעולה: OperationsController.Alerts → WorkspaceController.Alerts.



מתוך קוד המקור

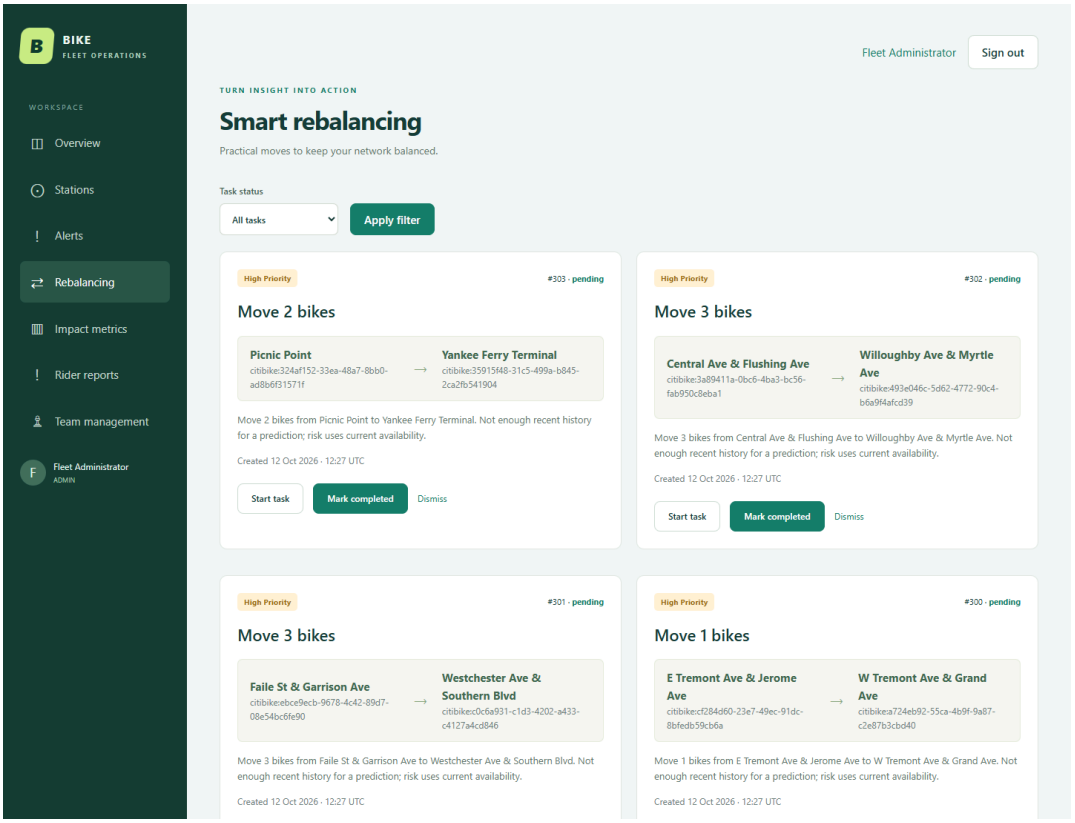
```
StationApi/Views/Workspace/Alerts.cshtml
001  @model JsonElement
002  @{ ViewData["Title"] = "Alerts"; }
003  <header class="page-header"><div><span class="eyebrow">STAY AHEAD OF
       DISRUPTION</span><h1>Operational alerts</h1><p>Prioritize the stations that need your
       attention.</p></div></header>
004  <form method="get" class="filters"><label>Severity<select name="severity"><option
       value="">All severities</option>@foreach(var s in new[]{"WARNING","CRITICAL"}) {
       <option selected="@(ViewBag.Severity == s)">@s</option>
       }</select></label><label>Status<select name="status"><option value="">All
       statuses</option>@foreach(var s in new[]{"ACTIVE","RESOLVED"}) { <option
       selected="@(ViewBag.Status == s)">@s</option> }</select></label><label>Station
       ID<input name="station" value="@ViewBag.Station" placeholder="e.g. S01"
       /></label><button>Apply filters</button></form>
```

מסכים 07 | משימות העברת אופניים

כרטיס משימה מציג כמות, מקור, יעד, סיבה, עדיפות וסטטוס. Start task מסמן תחילת טיפול, Mark completed מציין השלמה ו־Dismiss דוחה המלצה. מסנן Task status מאפשר לצפות גם בהיסטוריה של משימות סגורות.

לאחר שינוי מוצלח, Apply מפנה לרשימה עם הסטטוס שהוחזר מהשרת ו־updatedId. הכרטיס מודגש והרשימה ממוינת לפי UpdatedAt. משימה שהושלמה אינה מציגה שוב כפתור השלמה. הנתונים בזמן הצילום עשויים להכיל בעיקר משימות שנדחו בגלל מדידות ישנות.

מסלול הפעולה: WorkspaceController.Recommendations / Apply → OperationsController.Update.



מתוך קוד המקור

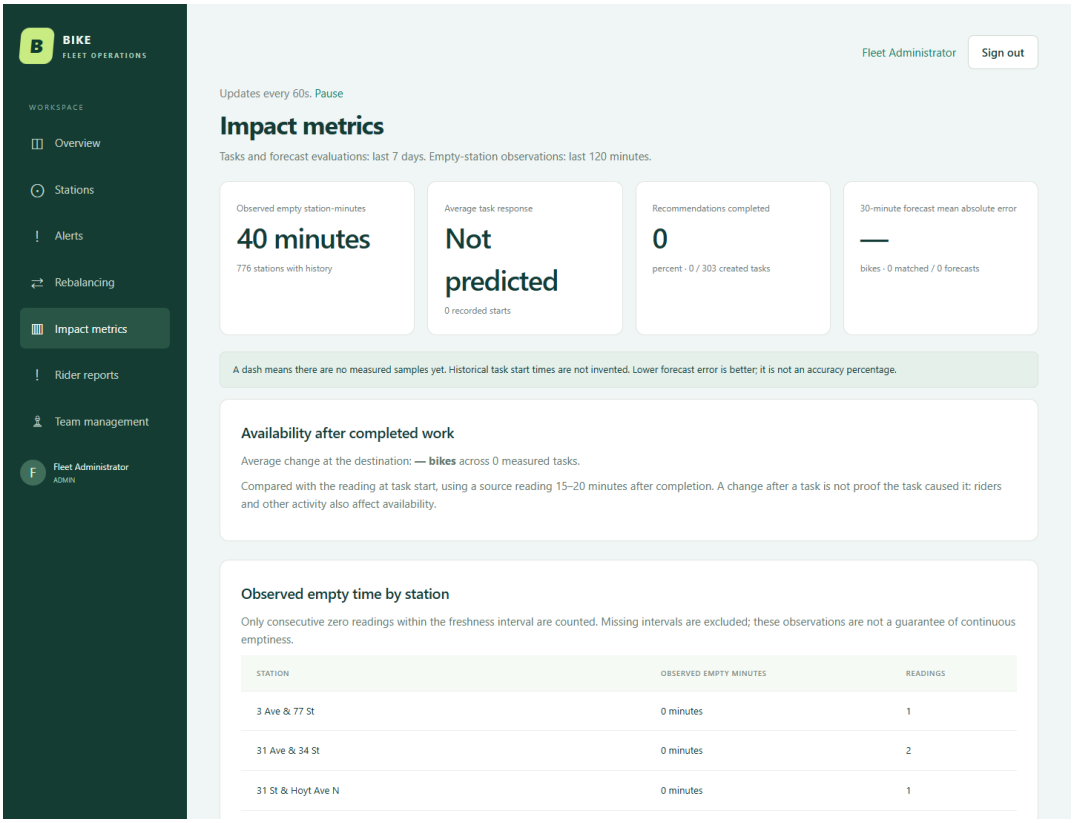
```
StationApi/Views/Workspace/Recommendations.cshtml
001  @model JsonElement
002  @{ ViewData["Title"] = "Rebalancing"; }
003  <header class="page-header"><div><span class="eyebrow">TURN INSIGHT INTO
       ACTION</span><h1>Smart rebalancing</h1><p>Practical moves to keep your network
       balanced.</p></div></header>
004  <form method="get" class="filters"><label>Task status<select name="status"><option
       value="">All tasks</option>@foreach(var s in
       new[]{"PENDING","IN_PROGRESS","COMPLETED","DISMISSED"}) { <option
       selected="@(ViewBag.Status == s)">@s</option> }</select></label><button>Apply
       filter</button></form>
```

מסכים 08 | מדדי השפעה

המסך מציג דקות תחנה ריקה, זמן תגובה ממוצע, אחוז השלמה ושגיאת תחזית ל־30 דקות. לכל מדד מוצגת משמעותו או כמות המדגמים. בהמשך יש שינוי בזמינות אחרי עבודה וטבלת זמן ריק לפי תחנה.

חלון המשימות והתחזיות הוא שבעה ימים; חלון קריאות ההיסטוריה נקבע בהגדרות. מקף פירושו שאין מספיק דגימות, לא ביצוע מושלם. Minutes משתמשת ב־DurationText, בעוד MAE נשאר מספר אופניים. צילום המסך אינו ניסוי מבוקר המוכיח השפעה סיבתית.

מסלול הפעולה: WorkspaceController.Metrics → MetricsController.Get → MetricsService.Summary.



מתוך קוד המקור

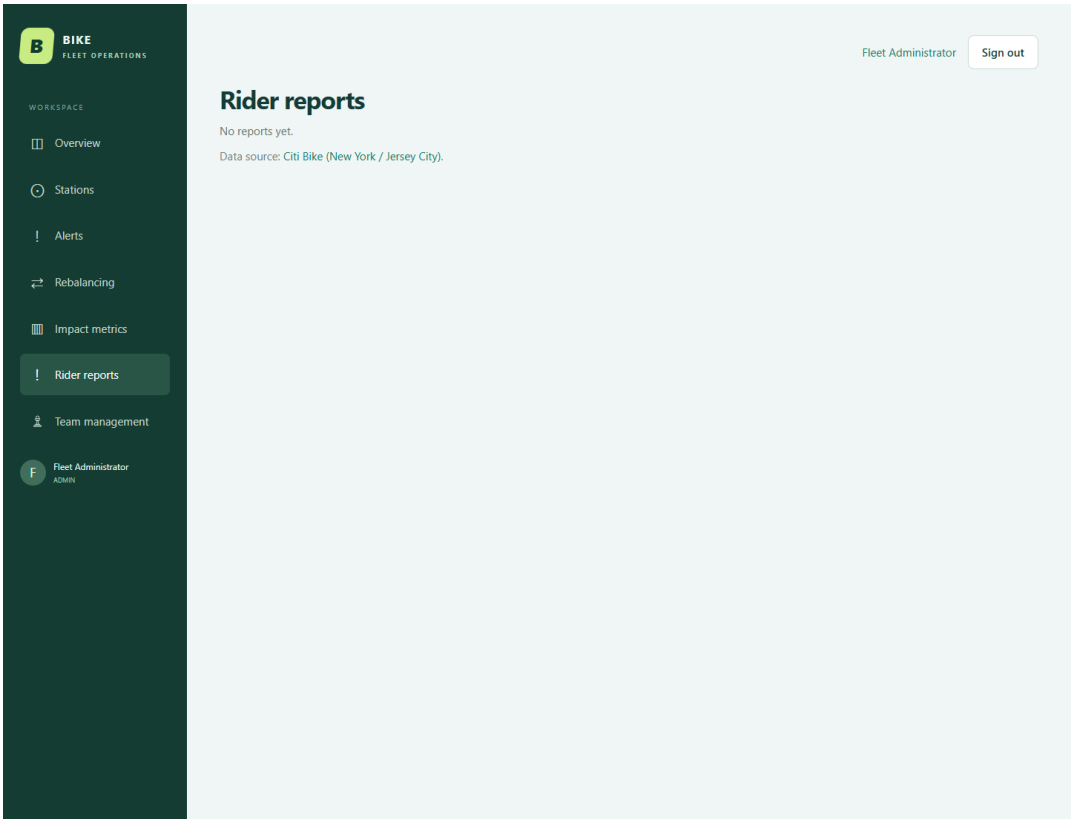
```
StationApi/Views/Workspace/Metrics.cshtml
001  @model JsonElement
002  @{ ViewData["Title"] = "Impact metrics"; }
003  <h1>Impact metrics</h1><p>Tasks and forecast evaluations: last 7 days. Empty-station
       observations: last @Model.Text("historyMinutes") minutes.</p>
004  <div class="metrics">
```

מסכים 09 | דיווחי רוכבים לצוות

מנהל ועובד יכולים לעיין בדיווחים שהתקבלו מרוכבים, לקרוא את התיאור ולפתור דיווח פתוח. המסך מאפשר להפריד בין מחסור אוטומטי בזמינות לבין בעיה שרוכב תיאר, כגון מתקן שאינו פועל. כאשר אין דיווחים, מוצג מצב ריק.

ResolveReport מבצעת POST לנתיב הפתרון ומחזירה למסך הרשימה. ה־API הוא שקובע הרשאות ושומר את האודיט. תמונה ריקה אינה סיבה להוסיף תקלה פיקטיבית לרשת; לצורכי בדיקה יש לסמן במפורש כל דיווח שנוצר ולסגור אותו לאחר השימוש.

מסלול הפעולה: RiderIssuesController → WorkspaceController.Reports / ResolveReport.



מתוך קוד המקור

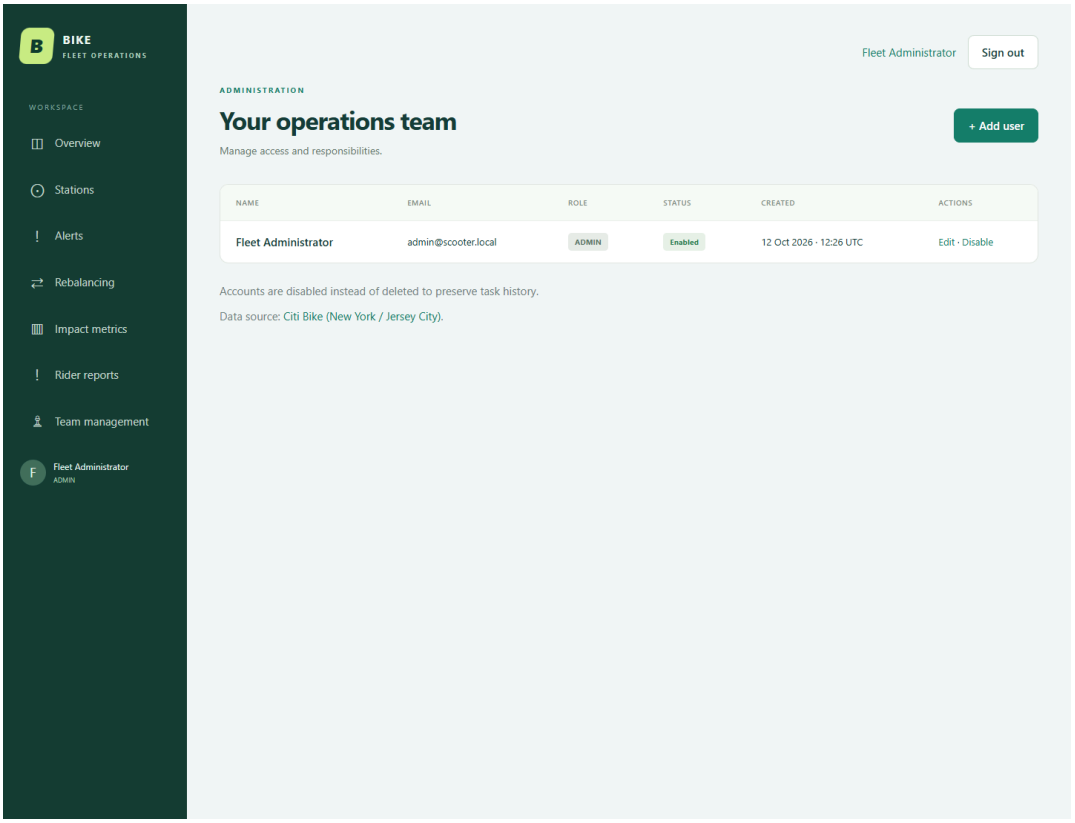
```
StationApi/Views/Workspace/Reports.cshtml
@model JsonElement
@{ ViewData["Title"] = "Rider reports"; }
<h1>Rider reports</h1>
@foreach(var issue in Model.EnumerateArray()) { <article
  class="alert-card"><strong>@issue.StationName("stationName", "stationId") ·
  @issue.Text("status")</strong><small
  class="muted">@issue.Text("stationId")</small><p>@issue.Text("description")</p><small
  >@issue.Time("createdAt")</small>
```

מסכים 10 | ניהול משתמשים

מסך המנהל מרכז שם, דוא״ל, תפקיד ומצב פעילות. ניתן לפתוח יצירה, עריכה או שינוי פעילות. אין להציג PasswordHash בטבלה. עובד ורוכב אינם מקבלים גישה לפעולות ניהול גם אם הם מנסים את הכתובת ישירות.

All משתמשת ב־AsNoTracking וממפה ל־UserView. שינוי מצב משתמש שומר את הרשומה במקום למחוק היסטוריה. קובץ המשתמשים המקורי ונתוני משתמשי התיעוד נשארים במסד; חשבונות התיעוד הושבתו בתום הצילום.

מסלול הפעולה: WorkspaceController.Users → UsersController.All.



מתוך קוד המקור

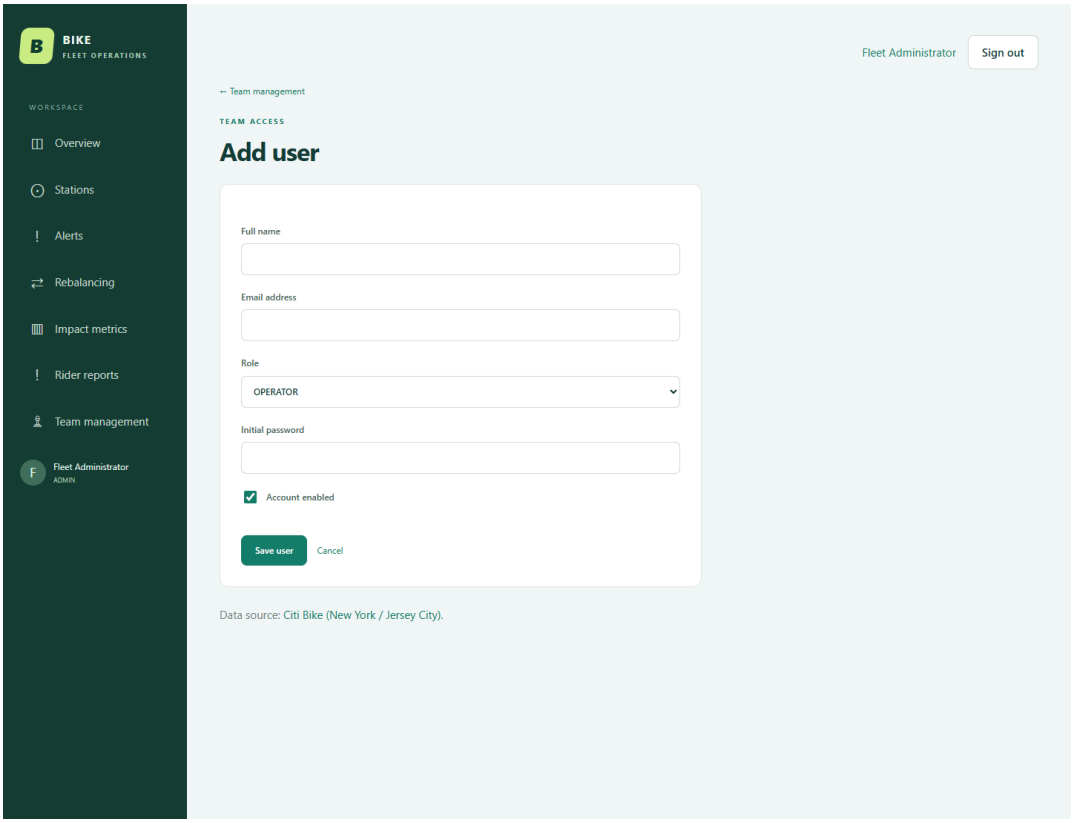
```
StationApi/Views/Workspace/Users.cshtml
001  @model JsonElement
002  @{ ViewData["Title"] = "Team management"; }
003  <header class="page-header"><div><span class="eyebrow">ADMINISTRATION</span><h1>Your
       operations team</h1><p>Manage access and responsibilities.</p></div><a class="button"
       asp-action="EditUser">+ Add user</a></header>
004  <section class="panel table-scroll"><table><thead><tr><th>Name</th><th>Email</th><th>Ro
       le</th><th>Status</th><th>Created</th><th>Actions</th></tr></thead><tbody>
```

מסכים 11 | יצירת חבר צוות

אותה תצוגה משמשת ליצירה ולעריכה. במצב יצירה אין id, הסיסמה נדרשת והמנהל בוחר תפקיד ופעילות. אחרי Save user השרת מאמת פרטים, בודק דוא״ל ייחודי ומגובב את הסיסמה.

המשתנה editing נגזר מהמזהה במודל. הוא קובע כותרת, שדה id והאם סיסמה היא חובה. הטופס אינו שולח בקשה להקמת חשבון במערכת חיצונית; הוא יוצר משתמש מקומי בפרויקט. בבדיקות התיעוד נוצרו חשבונות לדוגמה באישור המגישים.

מסלול הפעולה: WorkspaceController.SaveUser → UsersController.Create.



מתוך קוד המקור

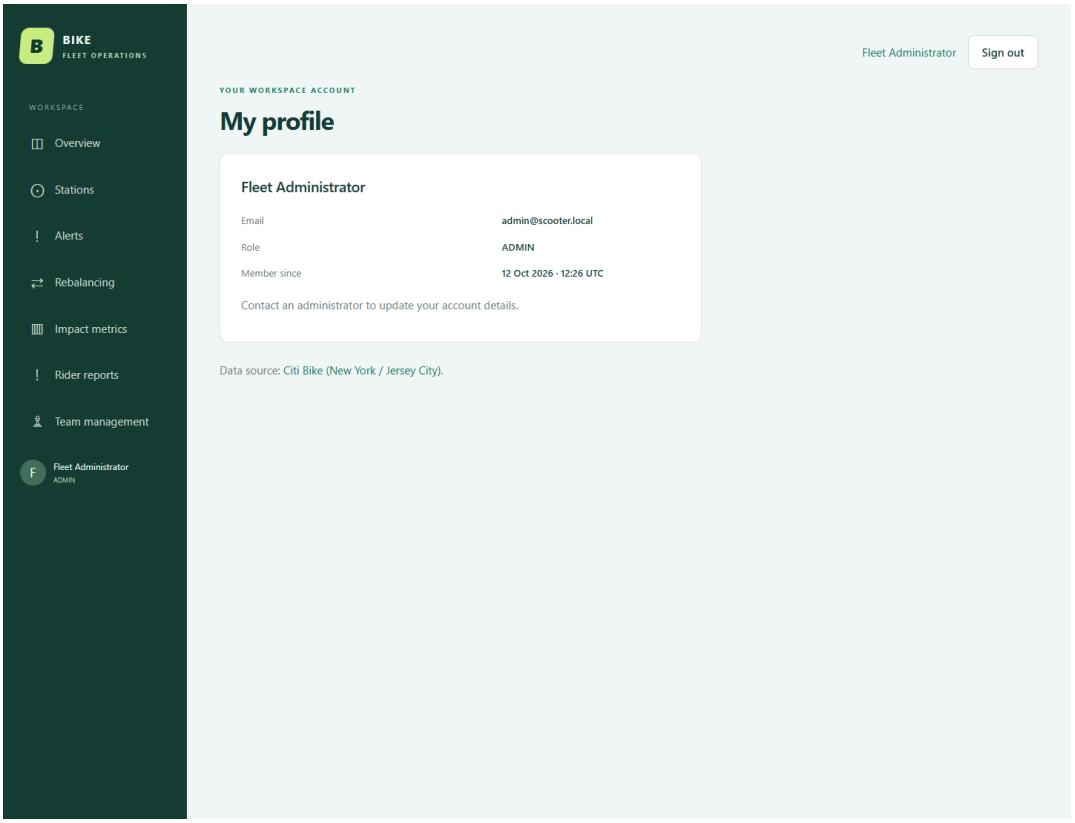
```
StationApi/Views/Workspace/EditUser.cshtml
001  @model JsonElement
002  @{ bool editing = Model.Number("id") > 0; ViewData["Title"] = editing ? "Edit user" :
       "Add user"; }
003  <a class="back-link" asp-action="Users">← Team management</a><header
       class="page-header"><div><span class="eyebrow">TEAM
       ACCESS</span><h1>@ViewData["Title"]</h1></div></header>
004  <section class="panel padded narrow">
```

מסכים 12 | פרופיל משתמש

הפרופיל מציג את זהות החשבון המחובר, תפקידו ופרטי החשבון הזמינים ב־UserView. תפקידו לאפשר למשתמש להבין באיזו זהות הוא עובד, בעיקר כאשר מדגימים תפקידים שונים בבחינה. התנתקות נמצאת במעטפת הדף.

Me משתמשת במזהה מתוך Claims ולא במזהה שנמסר בכתובת. אין כאן API לקריאת פרופיל שרירותי של אדם אחר. זהות התפקיד בצילום היא נתון הרצה; את כללי ההרשאות עצמם יש להסביר לפי ה־Controller וה־middleware.

מסלול הפעולה: AuthController.Me → WorkspaceController.Profile.



מתוך קוד המקור

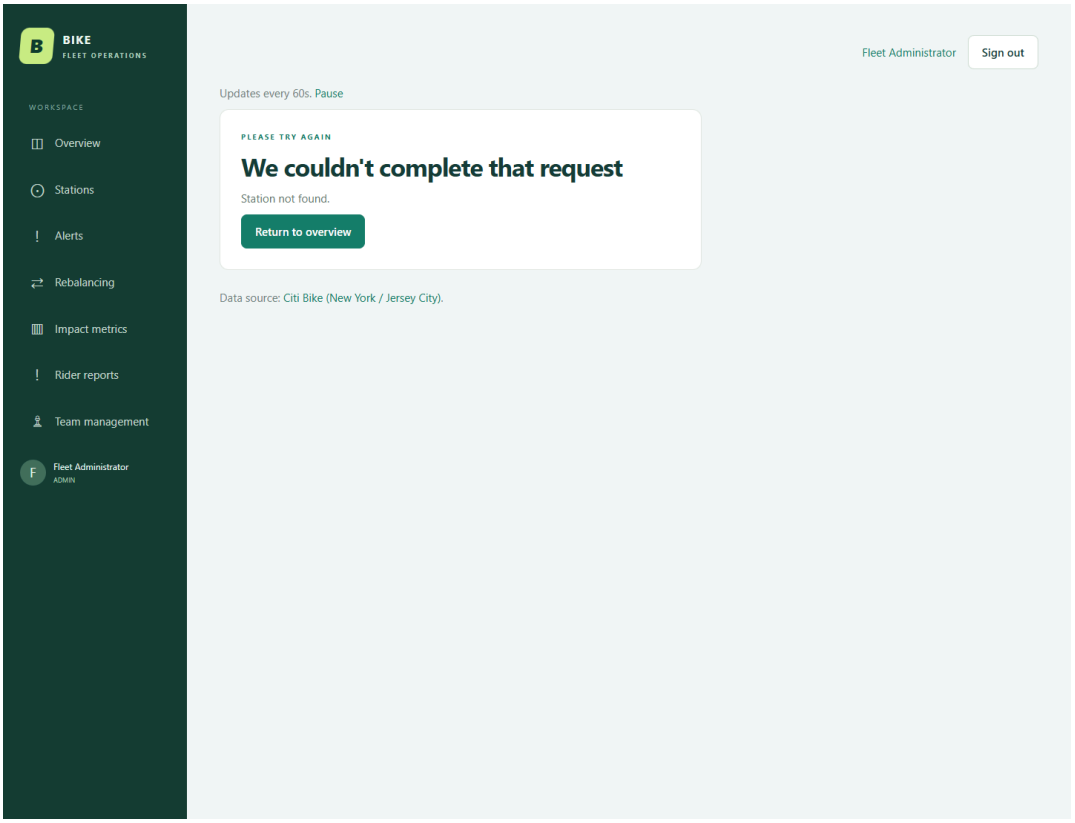
```
StationApi/Views/Workspace/Profile.cshtml
001  @model JsonElement
002  @{ ViewData["Title"] = "My profile"; }
003  <header class="page-header"><div><span class="eyebrow">YOUR WORKSPACE
       ACCOUNT</span><h1>My profile</h1></div></header>
004  <section class="panel padded narrow"><h2>@Model.Text("name")</h2><dl><dt>Email</dt><dd>
       @Model.Text("email")</dd><dt>Role</dt><dd>@Model.Text("role")</dd><dt>Member
       since</dt><dd>@Model.Time("createdAt")</dd></dl><p class="muted">Contact an
       administrator to update your account details.</p></section>
```

מסכים 13 | מסך שגיאה מבוקר

לצורך צילום זה נפתחה כתובת תחנה שאינה קיימת. ה־API מחזיר 404 וה־View מציגה הודעה במקום ערכי אפס או עמוד שנראה תקין. זו הדגמה מכוונת של שגיאת קלט, לא קריסה של מסד הנתונים.

OnActionExecutionAsync מטפלת ב־ApiFailure שהוחזרה מפעולת המסך ומגדירה את קוד התשובה. שגיאה בזמן אימות מוקדם או כשל תשתית כללי עשויים לעבור במסלול middleware אחר. חשוב להבחין בין 404, 403 ו־503 במקום לכנות את כולם שגיאת כניסה.

מסלול הפעולה: WorkspaceController.OnActionExecutionAsync → ApiFailure.



מתוך קוד המקור

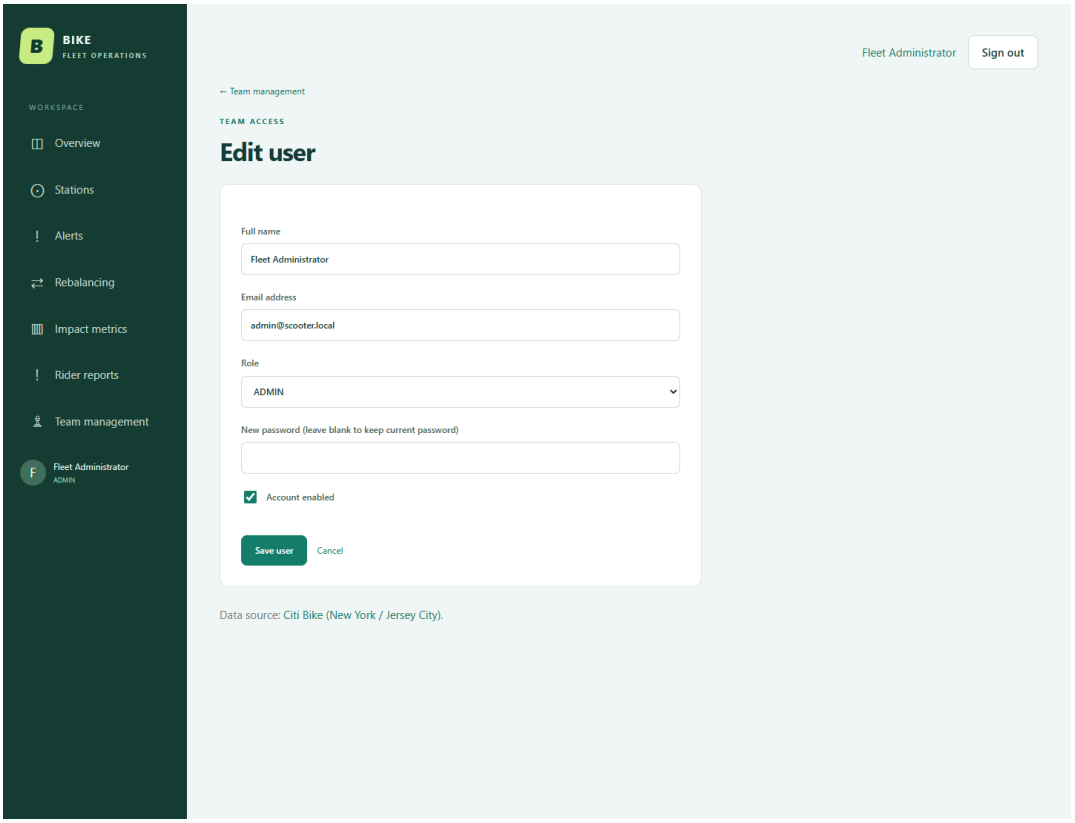
```
StationApi/Views/Workspace/Error.cshtml
001  @model string
002  @{ ViewData["Title"] = "Request unsuccessful"; }
003  <section class="panel padded narrow"><span class="eyebrow">PLEASE TRY
       AGAIN</span><h1>We couldn't complete that request</h1><p role="alert">@Model</p><a
       class="button" asp-action="Dashboard">Return to overview</a></section>
```

מסכים 14 | עריכת משתמש קיים

במצב עריכה נטענים הפרטים הקיימים וה־id נשלח בשדה מוסתר. סיסמה ריקה פירושה שמירת הסיסמה הקיימת. מנהל יכול לשנות שם, דוא״ל, תפקיד ופעילות בכפוף להגנות על חשבונו ועל המנהל הפעיל האחרון.

Update משתמשת בעסקה ברמת Serializable, בודקת כפילות דוא״ל, מגדילה SessionVersion ושומרת. שדה מוסתר אינו מנגנון הרשאה; ה־API בודק את המשתמש ואת התפקיד. צילום הטופס נעשה בלי לשמור שינוי לחשבון המנהל.

מסלול הפעולה: WorkspaceController.EditUser / SaveUser → UsersController.Update.



מתוך קוד המקור

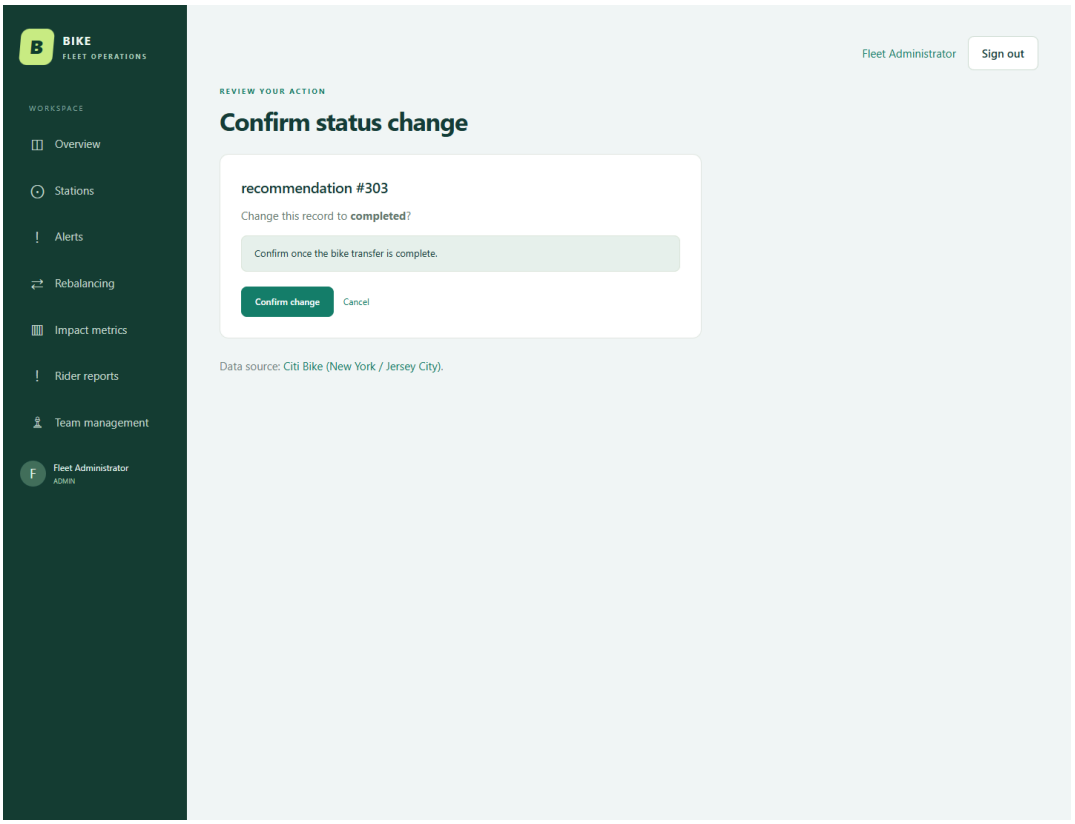
```
StationApi/Views/Workspace/EditUser.cshtml
001  @model JsonElement
002  @{ bool editing = Model.Number("id") > 0; ViewData["Title"] = editing ? "Edit user" :
       "Add user"; }
003  <a class="back-link" asp-action="Users">← Team management</a><header
       class="page-header"><div><span class="eyebrow">TEAM
       ACCESS</span><h1>@ViewData["Title"]</h1></div></header>
004  <section class="panel padded narrow">
```

מסכים 15 | אישור פעולה

מסך האישור מציג סוג רשומה, מזהה וסטטוס יעד. Confirm change מבצע POST, בעוד Cancel חוזר לרשימה. לפני השלמת העברה מופיעה תזכורת שהפעולה הפיזית צריכה להיות גמורה. פתיחת דף האישור אינה מסיימת משימה.

kind, id ו־status נשלחים בטופס. Apply בוחרת נתיב API מוגדר, ולכן לא מאפשרת כתובת חופשית מהלקוח. בדיקות ה־API עדיין נדרשות: משתמש יכול לשנות שדות hidden. לצורך התיעוד נפתח מסך האישור בלבד, בלי לסמן העברה אמיתית כמבוצעת.

מסלול הפעולה: WorkspaceController.Apply → WorkspaceController.Confirm.



מתוך קוד המקור

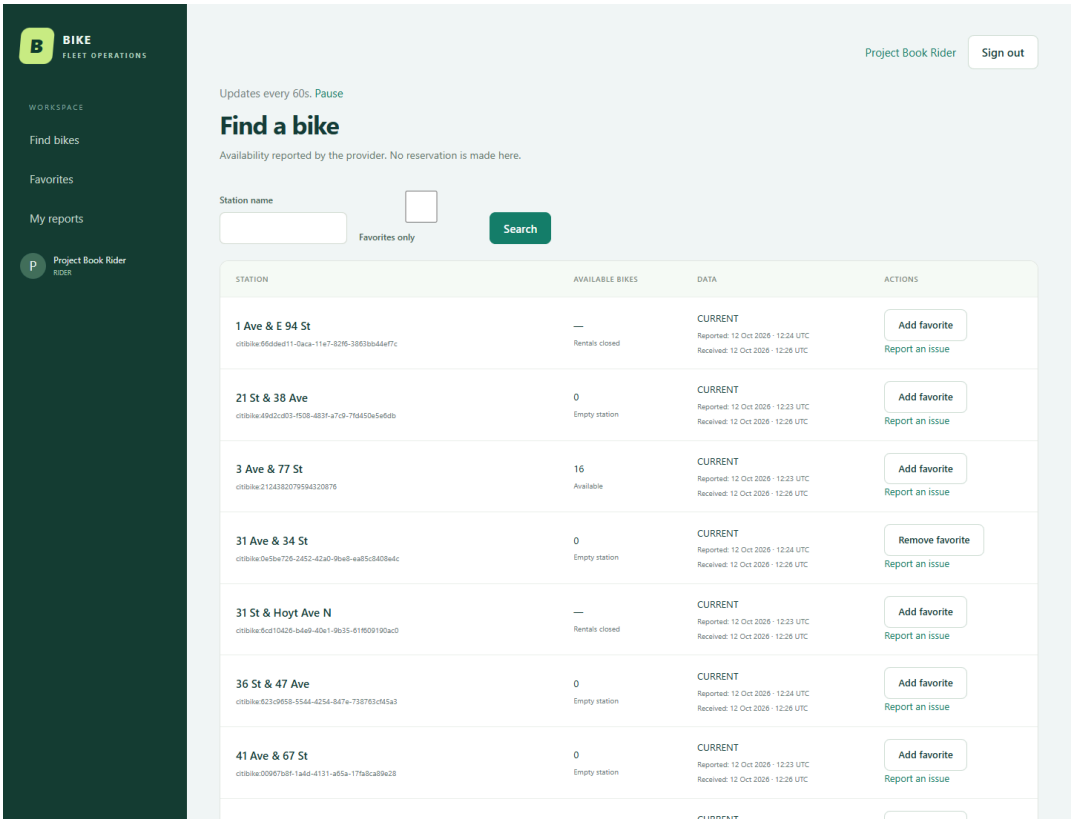
```
StationApi/Views/Workspace/Confirm.cshtml
001  @{ ViewData["Title"] = "Confirm action"; }
002  <header class="page-header"><div><span class="eyebrow">REVIEW YOUR
       ACTION</span><h1>Confirm status change</h1></div></header>
003  <section class="panel padded narrow"><h2>@ViewBag.Kind #@ViewBag.Id</h2><p>Change this
       record to <strong>@ViewText.Label((string)ViewBag.TargetStatus)</strong>?</p>
004  @if(ViewBag.TargetStatus == "COMPLETED") { <div class="notice">Confirm once the bike
       transfer is complete.</div> }
```

מסכים 16 | חיפוש אופניים לרוכב

המסך מציג זמינות פשוטה לרוכב, עם חיפוש בשם תחנה, מועדפים וקישור לדיווח. זמני המקור והקבלה מופיעים כדי שהרוכב יוכל להעריך את המידע. אין כאן הזמנת אופניים או התחייבות שכלי יישאר בתחנה עד ההגעה.

ה־API מסתיר כמות נוכחית כאשר המדידה ישנה או שהשכרה סגורה. הוא מוסיף favorite לפי רשומות המשתמש המחובר בלבד. RiderController מבצעת סינון לתצוגה; התפריט מצומצם לאזור הרוכבים ומונע בלבול עם תפעול.

מסלול הפעולה: RiderApiController.Stations → RiderController.Index.



מתוך קוד המקור

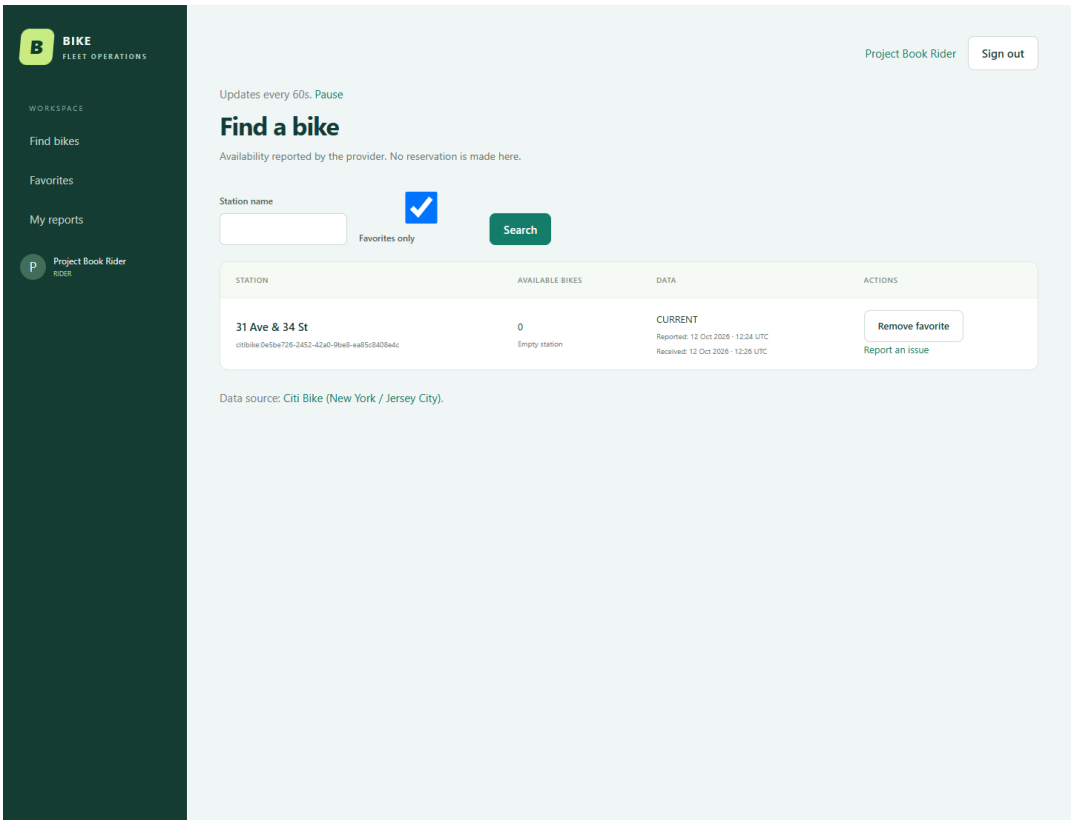
```
StationApi/Views/Rider/Index.cshtml
001  @model List<JsonElement>
002  @{ ViewData["Title"] = "Find a bike"; }
003  <header class="page-header"><div><h1>Find a bike</h1><p>Availability reported by the
       provider. No reservation is made here.</p></div></header>
004  <form method="get" class="filters"><label>Station name <input name="search"
       value="@ViewBag.Search" /></label><label><input type="checkbox" name="favorites"
       value="true" checked="@ViewBag.Favorites" /> Favorites only</label><button
       class="button">Search</button></form>
```

מסכים 17 | תחנות מועדפות

זהו מצב מסונן של מסך הרוכב ולא View נפרדת. סימון Favorites only מציג תחנות שסומנו על ידי החשבון. Add favorite ו־Remove favorite משנים את ההעדפה האישית ואינם משנים את מדידת התחנה או את העדפות המשתמשים האחרים.

המפתח המורכב UserId ו־StationId מונע אותה העדפה כפולה. Favorite בודקת שהתחנה קיימת לפני שמירה, ו־RemoveFavorite מוחקת רק את רשומת המשתמש הנוכחי. מועדף התיעוד הוסר לאחר הצילום. הנתונים האישיים אינם נכנסים למטמון התחנות המשותף.

מסלול הפעולה: RiderController.Favorite → RiderApiController.Favorite / RemoveFavorite.



מתוך קוד המקור

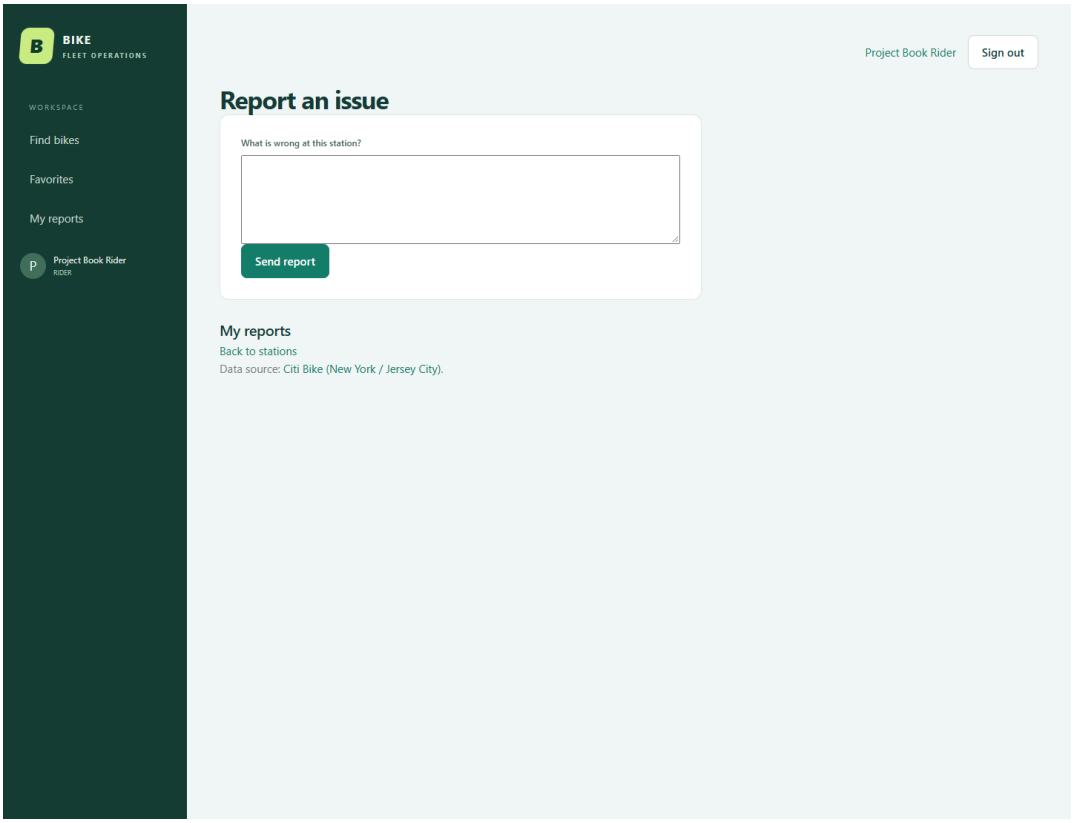
```
StationApi/Views/Rider/Index.cshtml
001  @model List<JsonElement>
002  @{ ViewData["Title"] = "Find a bike"; }
003  <header class="page-header"><div><h1>Find a bike</h1><p>Availability reported by the
       provider. No reservation is made here.</p></div></header>
004  <form method="get" class="filters"><label>Station name <input name="search"
       value="@ViewBag.Search" /></label><label><input type="checkbox" name="favorites"
       value="true" checked="@ViewBag.Favorites" /> Favorites only</label><button
       class="button">Search</button></form>
```

מסכים 18 | שליחת דיווח ומעקב אישי

כאשר מגיעים דרך תחנה, מופיע טופס תיאור התקלה עם מזהה התחנה. מתחת מוצגים הדיווחים האישיים והסטטוס שלהם. כניסה ל־My reports ללא תחנה מציגה היסטוריה בלי טופס יצירת דיווח לתחנה לא מוגדרת.

התיאור נדרש להיות באורך מתאים, והשרת בודק גם רווחים בלבד. לאחר הצלחה מוצגת הודעת אישור והדף נטען מחדש. בעת כשל מוצגת הודעה ורשימת הדיווחים נטענת מחדש. רוכב אינו יכול לבחור UserId של אדם אחר בגוף הבקשה.

מסלול הפעולה: RiderApiController.Report / Issues → RiderController.Report.



מתוך קוד המקור

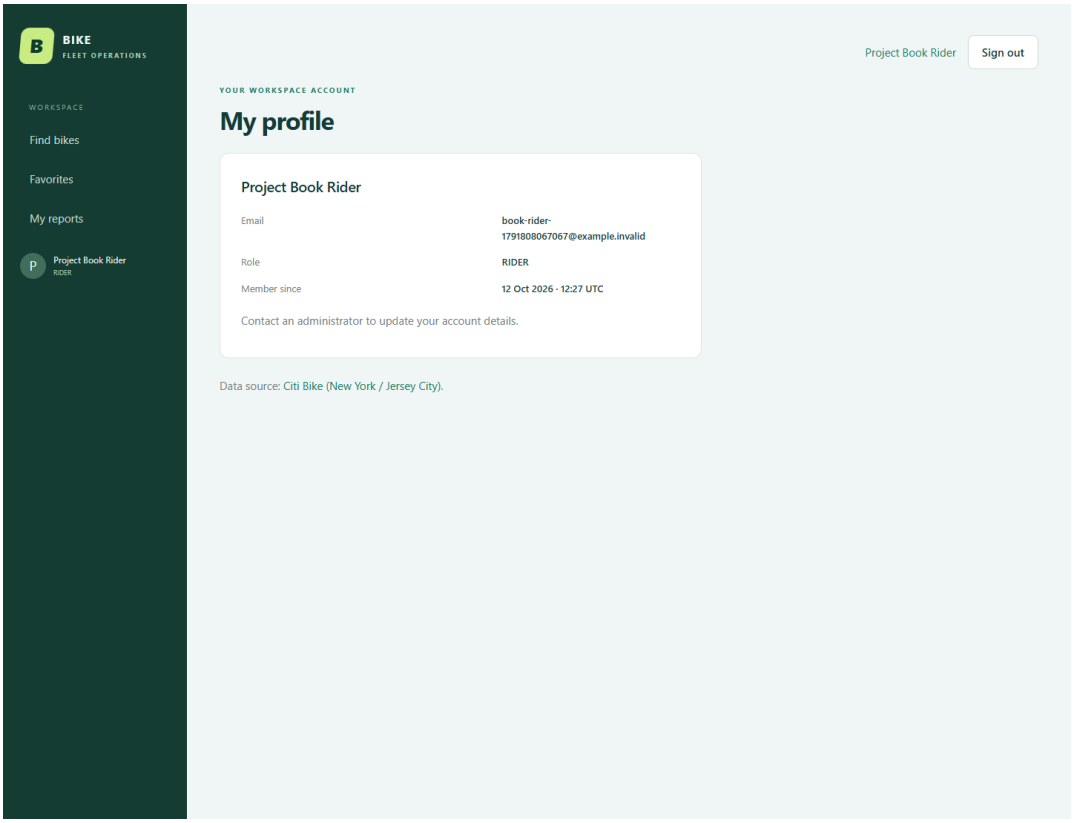
```
StationApi/Views/Rider/Report.cshtml
001  @model JsonElement
002  @{ ViewData["Title"] = "Issue reports"; }
003  <h1>Report an issue</h1>
004  @if(ViewBag.Error is string error) { <p class="notice warning">@error</p> }
```

מסכים 19 | זהות רוכב והרשאותיו

אותו דף פרופיל משמש גם רוכב, עם תפריט המתאים לתפקיד RIDER. זה מדגים שימוש חוזר ב־View בלי לשכפל עמוד כמעט זהה. פרטי החשבון המצולם הם של משתמש תיעוד, שהושבת בסיום.

במסנן Workspace קיימת החרגה של Profile ו־Logout מהפניית רוכבים לאזור Rider. בלעדיה לא היה ניתן להשתמש בפעולות משותפות אלו. ההחרגה אינה מעניקה לרוכב גישה לדשבורד תפעולי, לניהול משתמשים או להשלמת משימות.

מסלול הפעולה: AuthController.Me → WorkspaceController.Profile.



מתוך קוד המקור

StationApi/Views/Workspace/Profile.cshtml

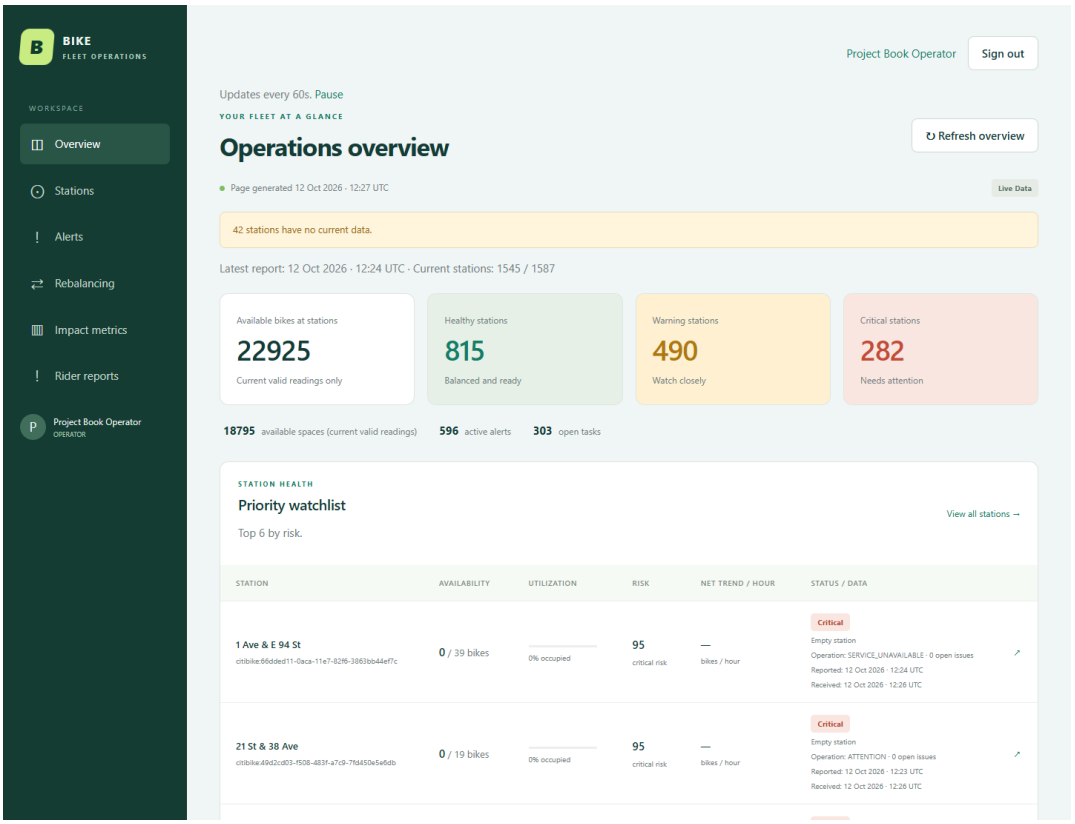
```
001  @model JsonElement
002  @{ ViewData["Title"] = "My profile"; }
003  <header class="page-header"><div><span class="eyebrow">YOUR WORKSPACE
       ACCOUNT</span><h1>My profile</h1></div></header>
004  <section class="panel padded narrow"><h2>@Model.Text("name")</h2><dl><dt>Email</dt><dd>
       @Model.Text("email")</dd><dt>Role</dt><dd>@Model.Text("role")</dd><dt>Member
       since</dt><dd>@Model.Time("createdAt")</dd></dl><p class="muted">Contact an
       administrator to update your account details.</p></section>
```

מסכים 20 | מבט עובד תפעול

עובד התפעול רואה את סקירת הצי ופעולות התפעול, אך אין לו קישור Team management. הרשאות OPERATOR נבדקות גם בשרת, ולכן היעדר הקישור אינו ההגנה היחידה.

הצילום המעודכן בוצע ב־12 באוקטובר 2026 לאחר איפוס סביבת Docker וקליטת נתוני GBFS מחדש. הדשבורד החזיר HTTP 200. צילום יחיד מעיד שהמסך זמין בזמן הבדיקה, ואינו הוכחה ליציבות ארוכת טווח.

מסלול הפעולה: FleetController.Dashboard → WorkspaceController.Dashboard.



מתוך קוד המקור

```
StationApi/Views/Workspace/Stations.cshtml
001  @model List<JsonElement>
002  @{ ViewData["Title"] = "Stations"; }
003  <header class="page-header"><div><span class="eyebrow">NETWORK
       VISIBILITY</span><h1>Bike stations</h1><p>Availability, trends and risk across your
       fleet.</p></div></header>
004  <form method="get" class="filters"><label>Find a station<input name="search"
       value="@ViewBag.Search" placeholder="Station name or ID"
       /></label><label>Status<select name="status"><option value="">All
       statuses</option>@foreach(var status in new[]{"HEALTHY","WARNING","CRITICAL"}) {
       <option value="@status" selected="@(ViewBag.Status ==
       status)">@ViewText.Label(status)</option> }</select></label><button
       type="submit">Apply filters</button><a asp-action="Stations">Reset</a></form>
```

36. תהליך שלם: מן הלחיצה לשמירה

הדגמת Rebalancing היא דרך טובה לקשור בין שכבות הפרויקט. המשתמש בוחר Mark completed בכרטיס. קישור GET פותח Confirm בלבד; אין כתיבה במסלול זה. לחיצה על Confirm change שולחת POST עם kind, id, status ו־token.

WorkspaceController.Apply בוחרת את endpoint של ההמלצה ושולחת PATCH באמצעות FleetApiClient. ה־API מאמת cookie, תפקיד ו־CSRF. Update מוודאת שהמשימה קיימת, שהמצב המבוקש מותר ושהתחנות שייכות למקור. בקשה שמשנה משימה סגורה נדחית, בעוד בקשה חוזרת לאותו מצב מטופלת באופן עקבי.

אחרי עדכון הרשומה ומדידות המשימה, SaveChangesAsync שומרת את השינויים. רק לאחר מכן נפסל המטמון ומוחזרת הרשומה. שכבת ה־Views קוראת את status מהתשובה, מציגה הודעת הצלחה ומפנה ל־Recommendations עם המסנן המתאים והמזהה להדגשה.

אם API מחזיר כשל, אין redirect הצלחה. גם כאשר נשמר COMPLETED, StationAnalysis עדיין נגזר מהספק. כך ניתן להבחין בין כשל שמירה, רשימה מסוננת שלא מציגה את המשימה, ומשימה שהושלמה אך לא הביאה לשיפור נצפה.

מתוך קוד המקור

```
StationApi/Controllers/WorkspaceController.cs
116           var saved = await api.Send(path, HttpMethod.Patch, kind == "user" ? new {
      enabled = status == "ENABLED" } : (object)new { status });
117           TempData["Success"] = kind == "recommendation"
118               ? $"Task #{id}: {Models.ViewText.Label(saved.GetProperty("status").GetStrin
      g()!)}."
119               : "Changes saved.";
120           if (kind == "alert" && saved.TryGetProperty("stationId", out var stationId))
      return RedirectToAction(nameof(Station), new { id = stationId.GetString() });
121           if (kind == "recommendation")
122           {
123               return RedirectToAction(nameof(Recommendations), new { status =
      saved.GetProperty("status").GetString(), updatedId = id });
124           }
125           return RedirectToAction(kind switch { "alert" => nameof(Alerts), "user" =>
      nameof(Users), _ => nameof(Recommendations) });
126       }
127       private static string Query(string key, string? value) =>
      string.IsNullOrWhiteSpace(value) ? "" : $"{key}={Uri.EscapeDataString(value)}";
128   }
```

37. בדיקות עסקיות דטרמיניסטיות

FleetTests היא תוכנית Console שמריצה assertions ומסיימת בכשל אם תנאי אינו מתקיים. Program בונה תחנות מלאכותיות עם זמן נתון ומפעילה את SmartFleetEngine. הנתונים המלאכותיים קיימים בבדיקות בלבד ואינם מוזרמים אל מאגרי הייצור.

נבדקים מחסור, עודף, תחנה מאוזנת, מידע חסר, זמן ישן או עתידי, קיבולת לא תקינה, מניעת המלצות לא אפשריות וחישוב מדדים. מבחני DurationText מכסים אפס, יחיד ורבים, שעה מדויקת, שעה עם שארית וערכים שאינם סופיים. בדיקות הרשמה מוודאות DataAnnotations וגיבוב סיסמה.

לאחר הניקוי עברו 66 בדיקות עסקיות בהרצה הרגילה. מספר זה אינו כולל את בדיקות המטמון, HTTP, Razor ו־Kafka שמופעלות בהמשך. האפשרות --gbfs מוסיפה בדיקה חיה מול המקור, ולכן מספר הבדיקות והנתונים יכולים להשתנות בין הפעלות.

יש להסביר לבוחן מה בדיוק נבדק: פונקציה טהורה מול fixture אינה בדיקת עומס, בדיקת UI אינה עסקה מול MySQL, ובניית solution אינה אימות התנהגות. הפרויקט משלב מספר סוגי ראיות במקום להציג מספר יחיד כהבטחת איכות מלאה.

מתוך קוד המקור

```
FleetTests/Program.cs
096  Check("Empty-time metric excludes gaps and nonzero samples",
       MetricsService.EmptyMinutes([
097      new(now.AddMinutes(-40), 0), new(now.AddMinutes(-35), 0), new(now.AddMinutes(-10),
       0),
098      new(now.AddMinutes(-5), 2), new(now, 0)], 10) == 5);
099  Check("Empty-time metric deduplicates source timestamps",
       MetricsService.EmptyMinutes([new(now.AddMinutes(-5), 0), new(now, 0), new(now, 0)],
       10) == 5);
100  Check("No readings do not invent empty time", MetricsService.EmptyMinutes([], 10) ==
       0);
101  foreach (var (value, expected) in new (double, string)[] { (0,"0 minutes"), (1,"1
       minute"), (45,"45 minutes"), (60,"1 hour"), (61,"1 hour 1 minute"), (120,"2 hours"),
       (306,"5 hours 6 minutes"), (.1,"1 minute"), (60.1,"1 hour 1 minute") })
102      Check($"Duration {value} => {expected}",
       StationApi.Models.DurationText.Format(value) == expected);
103  foreach (var value in new double?[] { null, -1, double.NaN, double.PositiveInfinity })
104      Check("Invalid duration is not displayed as a time",
       StationApi.Models.DurationText.Format(value) == "Not predicted");
105  Check("Single issue resolved restores healthy operation",
       StationStateService.Apply(healthy, 0, 0).OperationalStatus == "ACTIVE");
106  Check("Another open report keeps maintenance state", StationStateService.Apply(healthy,
       0, 1).OperationalStatus == "MAINTENANCE");
107  Check("Another open alert keeps station attention", StationStateService.Apply(healthy,
       1, 0).OperationalStatus == "ATTENTION");
108  Check("Resolution never overrides live critical availability",
       StationStateService.Apply(low, 0, 0).OperationalStatus == "ATTENTION");
```

38. בדיקות מטמון, HTTP ו־Kafka

CacheChecks מפעילה FleetDataService אמיתית עם תחליף Redis ומטעין נתונים מדומה. היא בודקת cache hit, TTL של 60 שניות, חישוב מחדש של טריות, הפרדת מקורות, fallback אחרי JSON פגום, פסילה עקב כתיבה, שינוי גרסת מעבד ונפילת Redis. גם מטמון תחנה בודדת נבדק.

HttpChecks משתמשת ב־HttpMessageHandler שמחזיר תגובות מתוכננות. היא בודקת ניסיון חוזר אחרי כשל זמני, feed ריק תקין, כשל שאינו הופך להצלחה וביטול. אלו מקרי גבול שניתן לבדוק בלי לחכות שספק אמיתי ייפול.

KafkaChecks משתמשת ב־DispatchProxy ל־IProducer. TaskCompletionSource מאפשרת להשאיר שליחות ממתינות ולוודא שאין flush מוקדם. נבדקים key, topic, token, התאמת JSON, כשל, ביטול ואצווה ריקה. הבנאי הפנימי של היצרן נועד להזרים תלות זו בלי broker אמיתי.

מבחנים אלה אינם מוכיחים חיבור רשת תקין ל־Redis או Kafka. הם מוודאים שהקוד מגיב נכון לחוזה של התלות. בדיקות אינטגרציה בסביבה פעילה משלימות אותם, אך לא הורצו מחדש באמצעות Docker כחלק מניקוי הקוד.

מתוך קוד המקור

```
FleetTests/KafkaChecks.cs
020          var sending = service.SendBatchAsync("status", items, item => item.StationId,
       cancellation.Token);
021          Check(stub.Messages.Count == 2 && !sending.IsCompleted && stub.Flushes == 0,
022              "Kafka batch waits for every delivery before flushing");
023          Check(stub.Messages.Select(x => x.Key).SequenceEqual(items.Select(x =>
       x.StationId))
024              && stub.Topics.All(x => x == "status") && stub.Tokens.All(x => x ==
       cancellation.Token),
025              "Kafka batch preserves topic, keys and cancellation token");
026          Check(stub.Messages.Select(x => x.Value).SequenceEqual(items.Select(item =>
       JsonSerializer.Serialize(item))),
027              "Kafka batch preserves existing JSON including missing and zero counts");
028          foreach (var pending in stub.Pending) pending.SetResult(new
       DeliveryResult<string, string>());
029          await sending;
030          Check(stub.Flushes == 1 && stub.LastTimeout == TimeSpan.FromSeconds(10), "Kafka
       batch flushes after successful delivery");
031
032          var failed = service.SendBatchAsync("status", items.Take(1), item =>
       item.StationId);
033          stub.Pending[^1].SetException(new InvalidOperationException("Simulated delivery
       failure"));
034          try { await failed; throw new Exception("Delivery failure was swallowed"); }
035          catch (InvalidOperationException) { Check(stub.Flushes == 1, "Kafka delivery
       failure propagates without a success flush"); }
```

39. בדיקות Razor ותרחישי קצה

WebChecks מפעילה שרת Kestrel מקומי שמרנדר את ה־Views האמיתיות, אך מחליפה את ה־API העסקי ב־fixture. נבדקות טעינת המסכים, היעדר script, token בטפסים, כמות לא ידועה לעומת אפס, זמני דיווח, רענון אוטומטי והשהייתו.

נבדקים גם דחיית API תפעולי לרוכב, דחיית ניהול משתמשים לעובד, ושינוי משימה שעובר דרך Apply ומציג את הסטטוס המעודכן. שגיאת שמירה מדומה צריכה להופיע ללא הודעת הצלחה. קישור ההרשמה נבדק בתוך חלונית הכניסה וההתנתקות נבדקת בראש התוכן.

לצורך ספר זה בוצעו גם כניסה דרך API וצילומים מהמערכת המקומית ב־8080. אלו ראיות נוספות לכך שהמסכים זמינים, אך לא בוצע ניסוי של העברת אופניים פיזית ולא סומנו משימות אמיתיות כהושלמו. חשבונות צילום נוצרו בהרשאה והושבתו.

בבדיקה המוקדמת נמצאו נתונים ישנים ושגיאת 503 זמנית. בהמשך אותרו השהיות Kafka ופיגור בעיבוד. לאחר איפוס מפורש של סביבת Docker, נצפו שוב מדידות עדכניות ומצבי Healthy, Warning ו־Critical. צילומי הספר עודכנו ב־12 באוקטובר 2026. תחנות שמקור המידע שלהן עדיין ישן נשארות מסומנות כאזהרה.

מתוך קוד המקור

```
FleetTests/WebChecks.cs
102             foreach (var status in new[] { "IN_PROGRESS", "COMPLETED" })
103             {
104                 fields["status"] = status;
105                 var changed = await client.PostAsync("/Workspace/Apply", new
      FormUrlEncodedContent(fields));
106                 var location = changed.Headers.Location?.ToString() ?? "";
107                 Assert(changed.StatusCode == HttpStatusCode.Redirect &&
      location.Contains("Recommendations") && location.Contains(status),
108                     $"Task update returns to its new status filter: {status}");
109                 var list = await client.GetStringAsync(location);
110                 Assert(list.Contains($"class=\"task-status\">{StationApi.Models.ViewTex
      t.Label(status)}") && list.Contains("updated-task"),
111                     $"Updated task is visible with saved status: {status}");
112                 if (status == "COMPLETED") Assert(!list.Contains("Mark completed"),
      "Completed task no longer offers completion");
113             }
114             Assert(dashboardHtml.Contains("account-actions") &&
      dashboardHtml.Contains("/Workspace/Logout"), "Sign out is available above page
      content");
115             var htmlLogin = await (await
      client.GetAsync("/Workspace/Login")).Content.ReadAsStringAsync();
116             Assert(htmlLogin.IndexOf("/Workspace/Register", StringComparison.Ordinal) <
      htmlLogin.IndexOf("</section>", htmlLogin.IndexOf("login-panel",
      StringComparison.Ordinal), StringComparison.Ordinal), "Registration link is inside
      the sign-in panel");
```

40. הפעלה, הגדרות וסודות

הפעלה מקומית מלאה נעשית דרך docker compose up -d --build. Compose מגדיר broker, MySQL, MongoDB, Redis, processor, ingestion ו־api. הפורטים הציבוריים קשורים ל־127.0.0.1. אתר היישום זמין כברירת מחדל ב־8080, והכתובות בין הקונטיינרים משתמשות בשמות השירותים ברשת הפנימית.

env. מחזיק סיסמת MySQL ופרטי seed, ו־setup.ps1 יוצר ערכים מקומיים כשצריך. אין לפרסם קובץ זה. MYSQL_ROOT_PASSWORD אינו אמור להיכנס לצילום או לספר. משתני DataSource בקליטה וב־API צריכים להתאים, בעיקר התחילית והמקור.

Volumes שומרים את MySQL, MongoDB ומפתחות Data Protection. מחיקת volume עלולה למחוק מידע או לפסול cookies. עדכון קוד במחשב אינו מעדכן image שכבר רץ: נדרשת בנייה מחדש אצל המשתמש. הספר אינו מבצע שינוי ב־Docker כדי להשיג צילום יפה יותר.

לפני הגשה מומלץ לבדוק בריאות שירותים, קליטה טרייה, כניסת תפקידים ותרחיש כתיבה בטוח. smoke-test.ps1 יוצר משתמש בדיקה ודיווחים מסומנים, בודק הרשאות ומנקה באמצעות השבתה וסגירה. הוא אינו משלים העברה אמיתית.

41. ניקוי הקוד והחלטות תחזוקה

הניקוי האחרון התמקד בקוד שאינו תורם להתנהגות. הוסרו שתי מחלקות ללא שימוש, מתודה שאינה נקראת, משתנים מיותרים והפניות לחבילות שלא שימשו את השירות. קוד שליחה חוזר בשלושת שירותי הקליטה הועבר ל־SendBatchAsync ביצרן הקיים.

קוד C# בייצור ירד מ־2,884 ל־2,716 שורות, ומ־58 ל־56 קבצים. הספירה כוללת שורות ריקות והערות, ואינה כוללת bin/obj, Razor או CSS. בדיקות נוספו בנפרד: חמישה קובצי בדיקות במקום ארבעה. לא נמחקו בדיקות כדי להציג צמצום גדול יותר.

ה־Controllers בעלי הנתיבים הקיימים נשמרו. גם migrations, חוזי JSON ושמות שדות אחסון נשארו. תיקון מוסכמות שמות אינו מצדיק שבירת מסד או לקוח חיצוני. camelCase מתאים למשתנים ולפרמטרים, PascalCase למחלקות ומתודות, ומיפוי מפורש נחוץ כאשר שם חיצוני שונה.

היעד ההנדסי הוא פחות כפילות ויותר הבנה. קובץ קצר בעל אחריות ברורה יכול להיות עדיף על מיזוג הכול לקובץ ארוך. מנגד, helper שלא משרת צורך אמיתי ניתן להסרה. ההחלטה מתקבלת לפי שימושים ותלות, לא לפי מספר קבצים בלבד.

מתוך קוד המקור

```
BikeFleetOperationPlatform/Services/KafkaProducerService.cs
016      public async Task SendBatchAsync<T>(string topic, IEnumerable<T> items, Func<T,
       string> keySelector,
017          CancellationToken token = default)
018      {
019          var sends = items.Select(item => SendToKafkaAsync(topic, new Message<string,
       string>
020          {
021              Key = keySelector(item),
022              Value = JsonSerializer.Serialize(item)
023          }, token));
024          await Task.WhenAll(sends);
025          _producer.Flush(TimeSpan.FromSeconds(10));
026      }
027
```

42. מגבלות והמשך פיתוח

המערכת היא בסיס תפעולי לימודי עם נתונים חיצוניים אמיתיים. היא אינה מערכת השכרה, ניווט משאיות או ניהול מנעולים. התחזיות ליניאריות, המרחק בהמלצות יחסי, ואין הוכחה מבוקרת שהמלצות משפרות זמינות בפועל. אלו גבולות שיש להסביר בגלוי בבחינה.

בצילום המערכת, המדידות שהיו זמינות במסד לא עמדו בסף הטריות. האתר הציג אזהרה במקום זמינות שגויה. כדי לאבחן סיבת תקלה יש לבדוק בנפרד מקור HTTP, שליחה ל־Kafka, צריכה ושמירה, אך עבודת התיעוד לא שינתה תשתית ולא הפעילה מחדש קונטיינרים.

להמשך אפשר לשקול pagination ברשימות, source health ייעודי, תור שגיאות, ניטור זמני קליטה ובדיקות אינטגרציה אוטומטיות. לתחזיות אפשר להוסיף תכונות זמן ומזג אוויר רק לאחר איסוף נתונים ובדיקת תועלת מול מודל הבסיס. לפריסה מרובת שרתים יש לתכנן תיאום כתיבות שאינו מסתמך על SemaphoreSlim מקומי.

הצעות אלו אינן חלק מהמימוש המתועד. הספר מבחין בין הקיים, הנבדק והעתידי כדי שהמגישים יוכלו להציג בביטחון את המערכת שבנו, בלי להתחייב ליכולות שאינן קיימות.

43. הכנה להצגה ולשאלות בבחינה

הצגה מומלצת מתחילה בבעיה: תחנה ריקה או מלאה ופיזור לא מאוזן. לאחר מכן מציגים את תרשים הזרימה, מסך Overview, תחנה בודדת ומשימת העברה. מסבירים מראש שהנתונים תלויים בספק ושצילום בזמן מידע ישן ידגים את מנגנון האמינות במקום תחזית.

שאלה: למה יש גם MySQL וגם MongoDB? תשובה: ישויות תפעול ומטא־דאטה נשמרות יחסית, בעוד מצב והיסטוריית מדידות נשמרים כמסמכים; ה־API מאחד ביניהם. זו בחירת הפרויקט, לא טענה שאי אפשר לבנות פתרון עם מסד יחיד.

שאלה: למה צריך Kafka? תשובה: הפרדה בין קליטה לעיבוד, שמירת מיקום צריכה וחזרה על הודעות לאחר כשל. למה Redis? כדי להקטין חישוב חוזר, עם תוקף קצר ו־fallback. למה אין JavaScript? הממשק מרונדר בשרת ומשתמש בטפסים, הפניות ורענון HTML.

שאלה: למה משימה הושלמה ותחנה עדיין Warning? תשובה: מצב המשימה מתעד עבודה, ואילו סיכון התחנה נגזר מהמדידה. שאלה: האם MAE הוא אחוז דיוק? לא; זו שגיאה ממוצעת באופניים. כדאי לדעת להראות את הפונקציה הרלוונטית בקוד ולא להסתפק בשם הטכנולוגיה.

44. מילון הישויות התפעוליות

FleetUser: Id מזהה מקומי; Name ו־Email פרטי זיהוי; PasswordHash תוצר גיבוב; Role תפקיד; Enabled הרשאת פעילות; SessionVersion גרסת ההתחברות; CreatedAt זמן יצירת החשבון.

FleetAlert: StationId התחנה הרלוונטית; Severity חומרה; Title ו־Description הסבר; Status מצב; CreatedAt, ResolvedAt ו־ResolvedBy תיעוד מחזור החיים. StationName הוא שדה NotMapped להעשרת תצוגה ואינו עמודה חדשה.

Recommendation: מזהי מקור ויעד, Quantity להעברה, Priority, Reason, Status, זמני יצירה ועדכון ומבצע העדכון. SourceStationName ו־DestinationStationName מיועדים לתצוגה ומסומנים NotMapped. כמות ההמלצה אינה כמות שהמערכת מעבירה פיזית.

RiderFavorite: זוג UserId ו־StationId. RiderIssue: מזהה דיווח, משתמש, תחנה, תיאור, מצב ותיעוד סגירה. אין בפרויקט ניהול מלאי של אופניים בודדים לפי מספר שלדה; יחידת המידע העיקרית היא התחנה.

TaskMeasurement: קשר להמלצה, זמני התחלה וסיום, כמות ביעד לפני ואחרי וזמן תצפית. ForecastSample: תחנה, זמן הנפקה, זמן יעד, כמות חזויה, כמות בפועל וזמן מדידה. שדות nullable מסמנים תצפית שטרם התקבלה, לא תוצאה של אפס.

מתוך קוד המקור

```
StationApi/Models/Operations.cs
064  public class TaskMeasurement
065  {
066      public long RecommendationId { get; set; }
067      public DateTime? StartedAt { get; set; }
068      public DateTime? CompletedAt { get; set; }
069      public int? DestinationBefore { get; set; }
070      public int? DestinationAfter { get; set; }
071      public DateTime? ObservedAt { get; set; }
072  }
073  public class ForecastSample
074  {
075      public long Id { get; set; }
076      [MaxLength(255)] public string StationId { get; set; } = "";
077      public DateTime IssuedAt { get; set; }
078      public DateTime DueAt { get; set; }
079      public double PredictedCount { get; set; }
080      public int? ActualCount { get; set; }
081      public DateTime? ObservedAt { get; set; }
082  }
```

45. חוזי DTO ומיפוי שמות

DTO הוא אובייקט העברת נתונים, אך בפרויקט חלק ממחלקות המטא־דאטה משמשות גם ישויות EF. StationInformationDto כולל StationId, Name, Lat, Lon ו־VehicleTypesDto כולל מזהה סוג, גורם Capacity. צורה וסוג הנעה. אותם רעיונות מופיעים בשירותים שונים כדי לשמר את חוזי התקשורת והאחסון הקיימים.

מאפייני C# נכתבים ב־PascalCase: StationId, NumBikesAvailable, NumVehiclesAvailable, NumDocksAvailable, IsRenting, IsReturning ו־JsonPropertyName ממפה לשמות GBFS LastReported. והודעות קיימות; Column ו־ToTable משמרים את שמות העמודות והטבלאות במסד. משתנים מקומיים ופרמטרים נשארים ב־camelCase. כך הקוד קריא בלי לשבור נתונים קיימים.

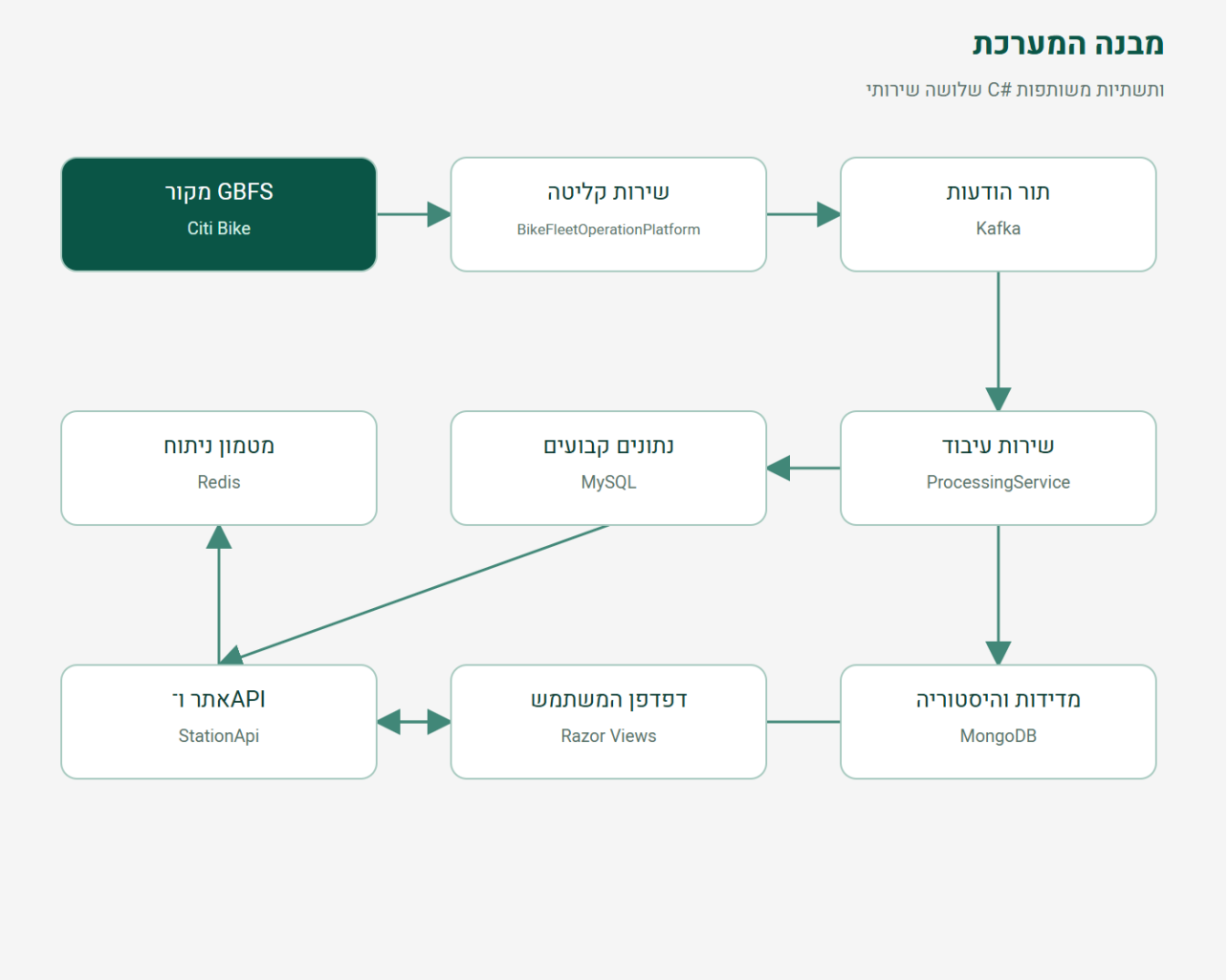
StationInput ו־StationAnalysis הם records של שכבת הניתוח, לא אותה ישות EF. Reading הוא זוג זמן וכמות; MoveSuggestion הוא תוצר חישובי לפני שמירה כהמלצה תפעולית. ההבחנה מאפשרת למנוע הניתוח לעבוד בלי DbContext.

AllDataDto ו־GetFilterDto משרתים את הנתיב הוותיק. RegisterRequest, LoginRequest, UserRequest, StatusRequest ו־RiderIssueRequest מגדירים קלט לפעולות. UserView מגדיר פלט בטוח לחשבון. אין למחוק טיפוס רק בגלל שמו: צריך לבדוק את הצרכנים ואת חוזה JSON שלו.

מתוך קוד המקור

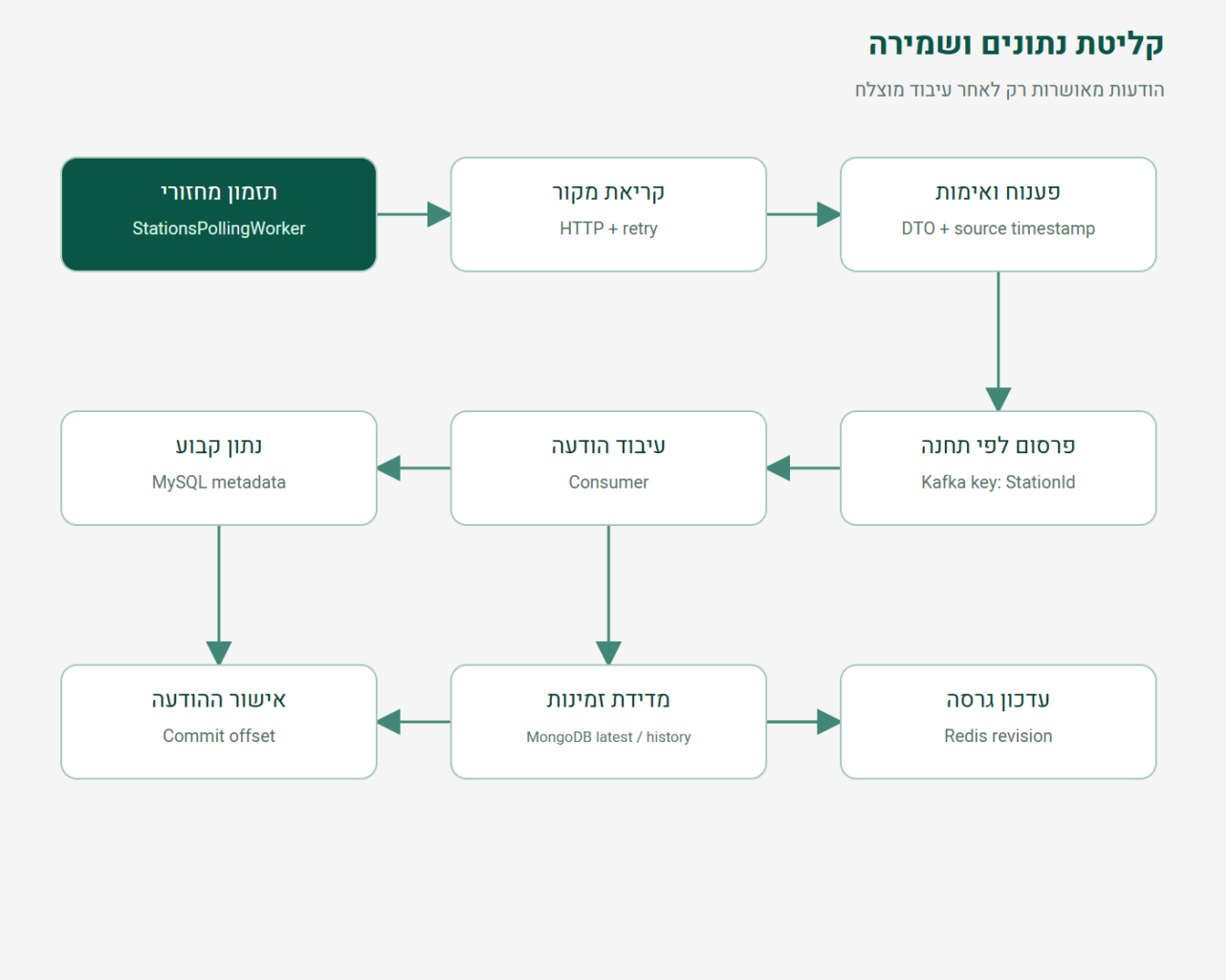
```
BikeFleetOperationPlatform/Dtos/StationStatusDto.cs
001  using System.ComponentModel.DataAnnotations;
002
003  namespace BikeFleetOperationPlatform.Dtos;
004
005  public class StationStatusDto
006  {
007      [Required]
008      public string StationId { get; set; } = string.Empty;
009      [Range(0,int.MaxValue)]
010      public int? NumBikesAvailable { get; set; }
011      [Range(0, int.MaxValue)]
012      [System.Text.Json.Serialization.JsonPropertyName("num_vehicles_available")]
013      public int? NumVehiclesAvailable { get; set; }
014      [Range(0, int.MaxValue)]
015      [System.Text.Json.Serialization.JsonPropertyName("num_docks_available")]
016      public int? NumDocksAvailable { get; set; }
017      [System.Text.Json.Serialization.JsonConverter(typeof(BikeFleetOperationPlatform.
       Services.GbfsFlagConverter))]
018      public int? IsRenting { get; set; }
019      [System.Text.Json.Serialization.JsonConverter(typeof(BikeFleetOperationPlatform.
       Services.GbfsFlagConverter))]
020      public int? IsReturning { get; set; }
021      public long? LastReported { get; set; }
022      public DateTime? ReceivedAt { get; set; }
023  }
```

תרשים 1 | מבנה המערכת



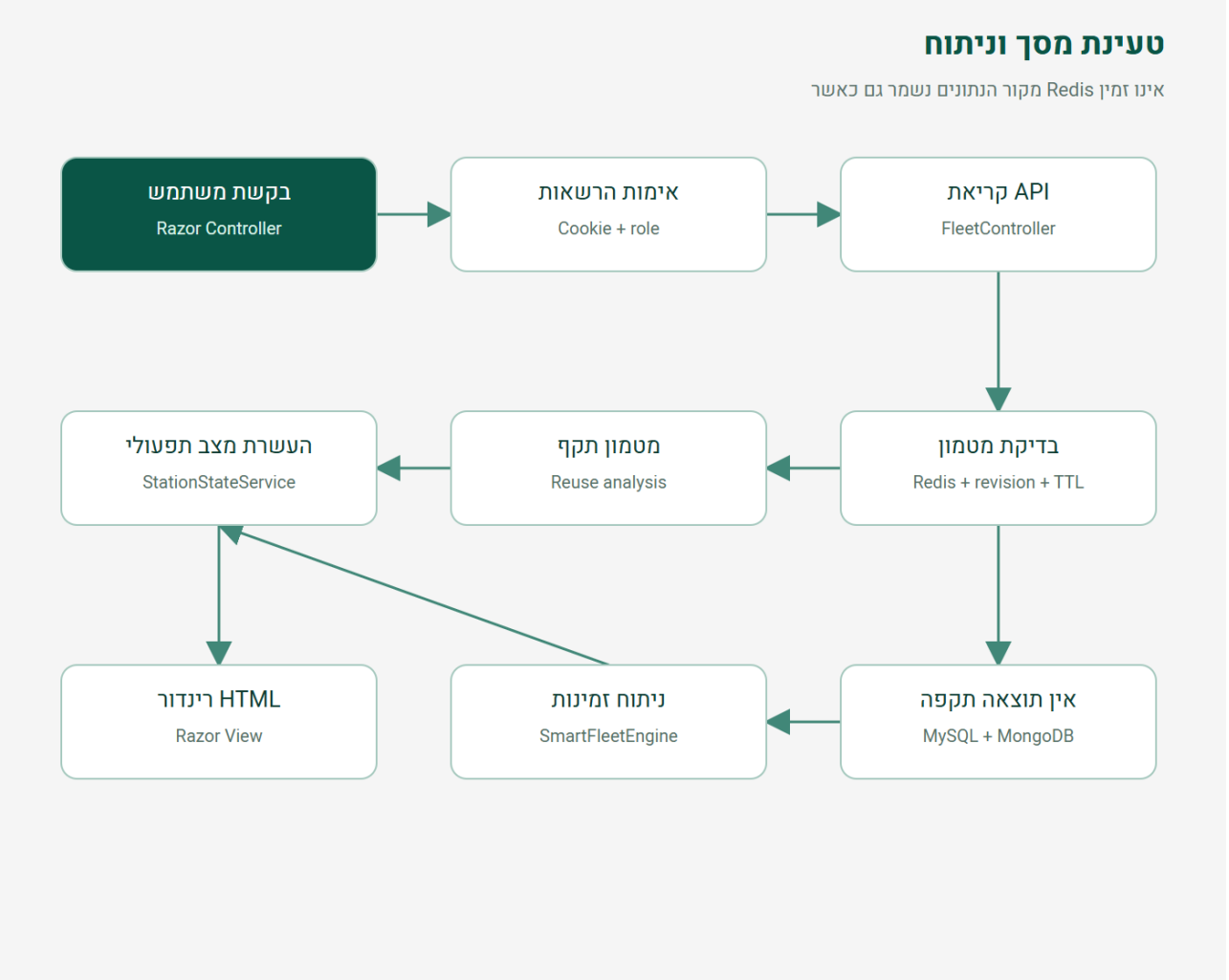
החצים מציגים את התלות והעברת המידע בין הרכיבים. שמות הרכיבים תואמים לקוד הפרויקט.

תרשים 2 | קליטת נתונים ושמירה



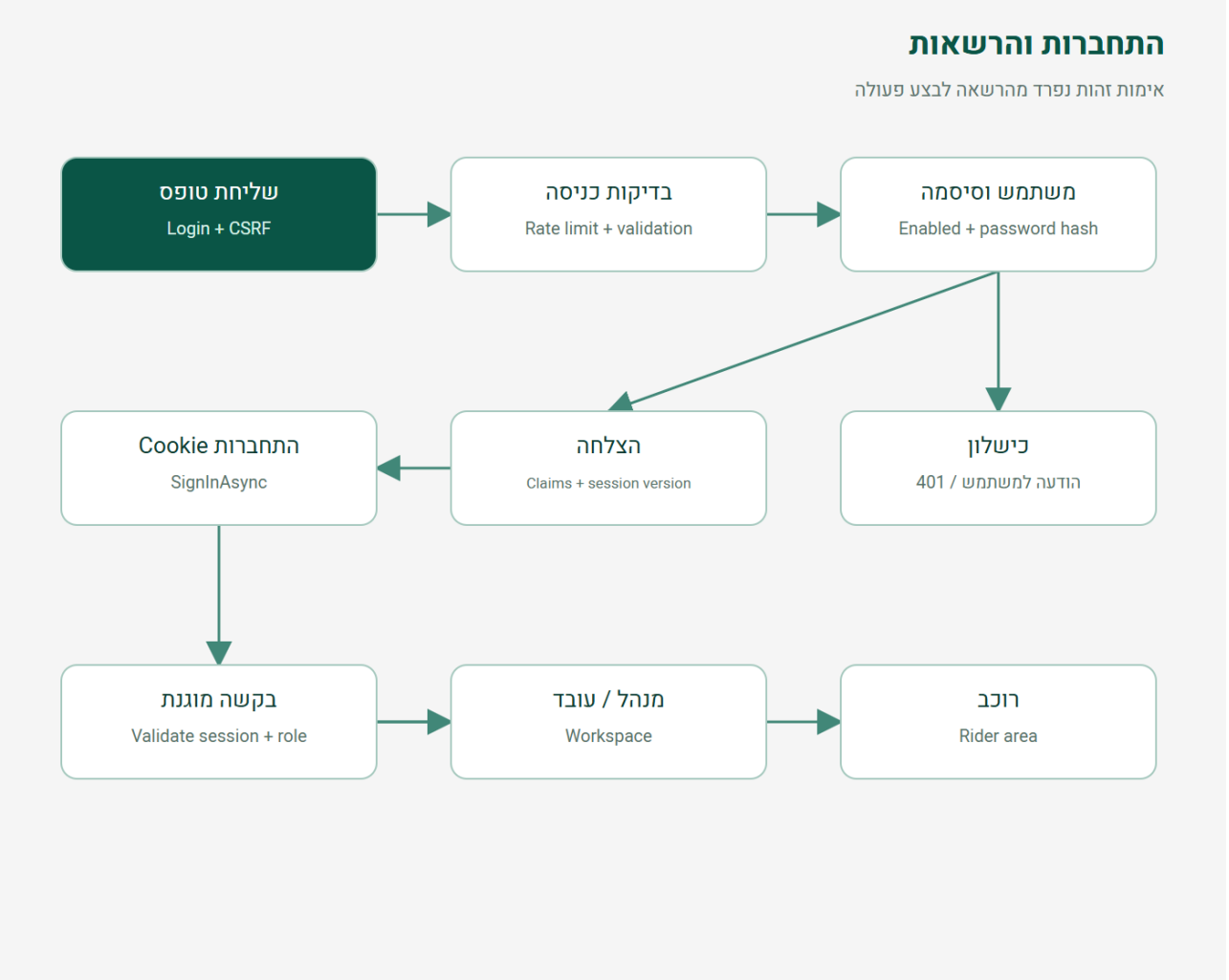
HTTP נכשל: ניסיונות חוזרים מוגבלים. כשל תשתית בעיבוד: ניסיון חוזר לאותה הודעה. JSON פגום נרשם ונדחה.

תרשים 3 | טעינת מסך וניתוח



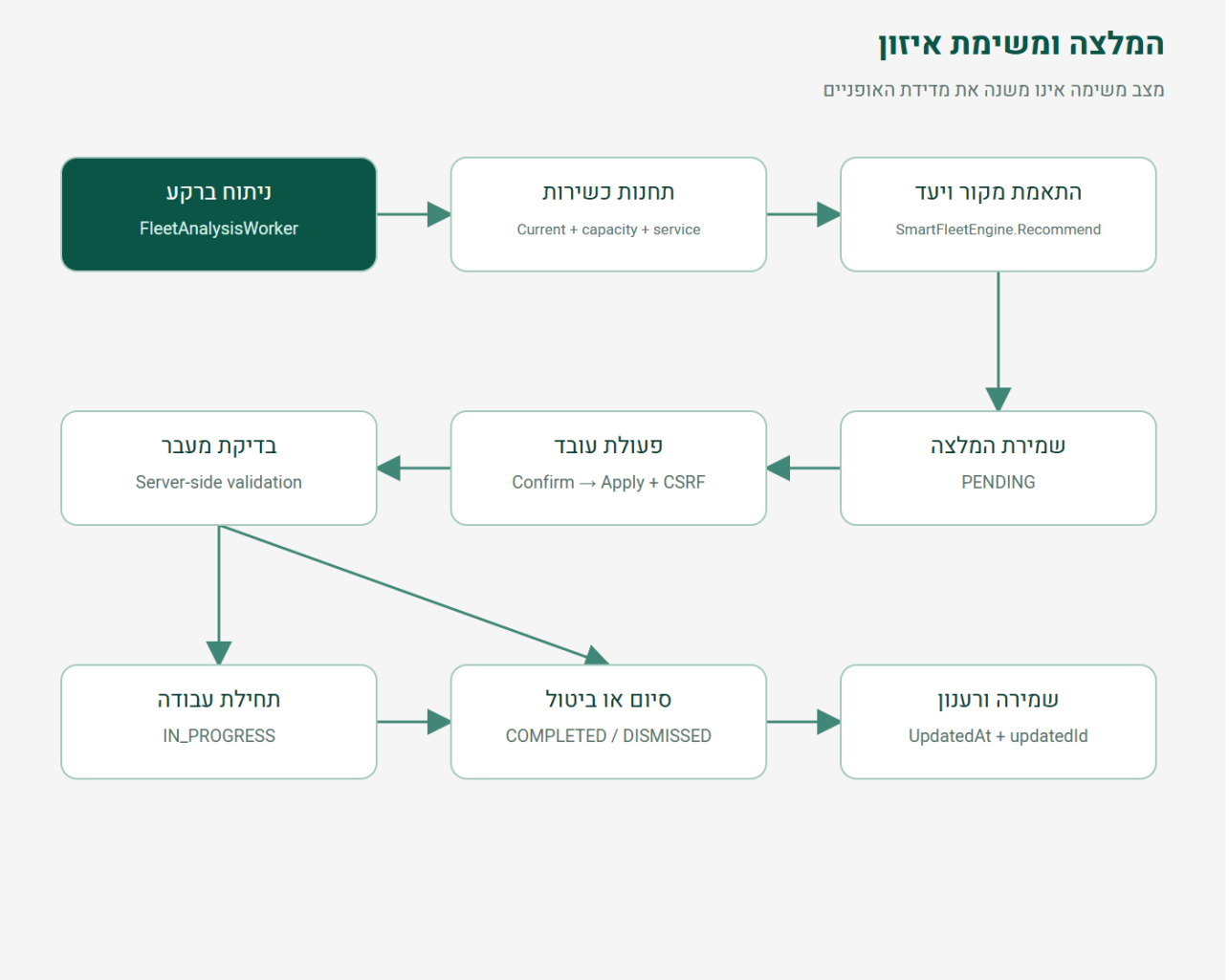
השליפה לפרטי תחנה ממוקדת במזהה שלה. מטמון הניתוח אינו מחליף את זמני המדידה המקוריים.

תרשים 4 | התחברות והרשאות



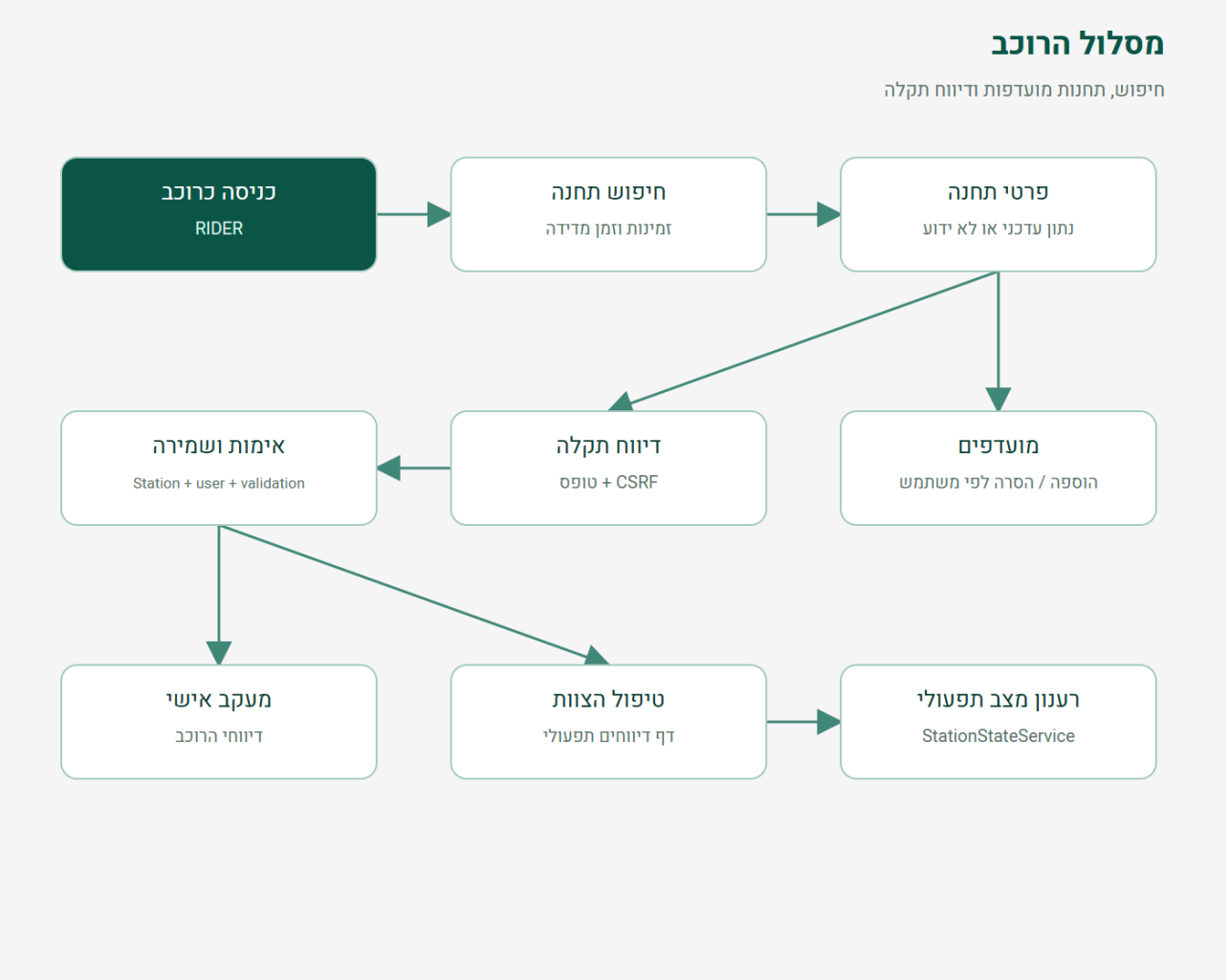
הרשמה ציבורית יוצרת RIDER בלבד. התנתקות מבטלת את Cookie ההתחברות; בקשות שינוי מחייבות CSRF.

תרשים 5 | המלצה ומשימת איזון



אפשר להשלים או לדחות גם ישירות מ־PENDING. מצב סופי אינו חוזר לפעיל; הבקשה החוזרת לאותו מצב אינה יוצרת מעבר חדש.

תרשים 6 | מסלול הרוכב



מועדפים משויכים למשתמש. סגירת דיווח אינה הופכת זמינות ישנה לעדכנית ואינה משנה נתוני GBFS.

נספח א | קובצי ממשק, תצורה וכלי עבודה

Views/Workspace מכילה Login, Register, Dashboard, Stations, Station, Alerts, Recommendations, Metrics, Reports, Users, EditUser, Profile, Confirm ו־Error. כל מסך מופיע בפרק הצילומים עם קישור למסלול הפעולה שלו. Views/Rider מכילה Index ו־Favorites; Report היא תצוגה מסוננת של Index ולא קובץ נפרד.

Views/Shared מכילה _StationTable, _Layout ו־_ViewStart. Recommendations קובעת את המעטפת, ו־_ViewImports מייבאת namespaces ו־Tag Helpers. site.css מגדירה את פריסת התפריט, הטבלאות, הכרטיסים, המצבים וההתאמות למסך קטן.

appsettings.json בכל שירות מכיל הגדרות ברירת מחדל: חיבורים, מקור, polling או אפשרויות ניתוח. appsettings.Development.json ו־launchSettings.json משלימים פיתוח מקומי. קובצי csproj קובעים framework וחבילות; Dockerfile בכל שירות בונה ומפרסם אותו. docker-compose.yml מחבר את השירותים, הרשת, משתני הסביבה וה־volumes.

BikeFleet.sln ו־BikeFleet.slnx מתארים את הפתרון; BikeFleet.code-workspace והגדרות .vscode מסייעים לעבודה בעורך. .editorconfig מגדיר מוסכמות קוד. index.html הוא דף הפניה מקומי פשוט ואינו אתר ה־Razor. StationApi.http הוא אוסף בקשות דוגמה, ולא endpoint עצמאי.

scripts/setup.ps1 מכין הגדרות ראשוניות; scripts/smoke-test.ps1 מבצע תרחישי בדיקה מול אתר פעיל. README, SMART_FLEET, IMPLEMENTATION_REPORT ו־CODE_CLEANUP מתעדים הפעלה, לוגיקה ואימות. תוצרי bin/obj, קובצי משתמש של IDE ו־.env אינם חלק מההסבר העסקי ואין לפרסם את הסודות שבהם.

נספח ב | מונחים ותשובות קצרות

API: ממשק HTTP שמקבל בקשות ומחזיר JSON וקוד מצב. Controller: נקודת טיפול בבקשה, לא מסד נתונים. View: תבנית Razor שמייצרת HTML. DTO: מבנה להעברת נתונים. Entity: אובייקט שממופה למאגר יחסי. Record: טיפוס המתאים לערכים ותוצרים חישוביים.

Dependency Injection: העברת תלויות דרך container במקום בנייה מפוזרת. Scoped: מופע לאורך scope של בקשה או מחזור. Singleton: מופע יחיד בתהליך. HostedService: עבודה ברקע תחת חיי ה־host. CancellationToken: איתות לביטול פעולה אסינכרונית.

Topic: ערוץ הודעות Kafka. Partition: חלוקה של topic. Offset: מיקום הודעה במחיצה. Consumer group: קבוצת צרכנים עם מיקום צריכה. Commit: אישור מיקום לאחר טיפול. Idempotency: חזרה על פעולה בלי ליצור תוצאה כפולה לא רצויה.

Cache-aside: קריאה מהמטמון, ובחוסר קריאה ממקור ושמירה זמנית. TTL: זמן חיים. Invalidation: פסילת תוצאה ישנה. Nullable: יכולת לייצג היעדר ערך. Upsert: עדכון רשומה קיימת או יצירת חדשה. AsNoTracking: קריאה ב־EF ללא מעקב שינוי ישויות.

Claims: פרטי זהות והרשאה בתוך principal. CSRF: ניצול cookie של משתמש לפעולה מאתר זר; token מסייע למנוע זאת. Hash: ייצוג חד־כיווני לסיסמה, לא הצפנה שנפתחת כדי להשוות. MAE: ממוצע שגיאה מוחלטת. Rebalancing: העברת כלים כדי לשפר את הפיזור בין תחנות.

נספח ג | מקורות וגרסת התיעוד

המקור הראשי לספר הוא קוד הפרויקט המקומי לאחר ניקוי הקוד. קטעי הקוד הועתקו מהקבצים המצוינים לידם. צילומי המסך נלקחו מהאתר המקומי ב־12 באוקטובר 2026 לאחר איפוס התשתיות וקליטת מדידות עדכניות, ולא נוצרו כהדמיות. הקליטה הראשונית נמשכה במהלך הצילום; לכן מספר התחנות עשוי להשתנות בין המסכים.

מקורות רשמיים לקריאה נוספת:

Citi Bike System Data: https://citibikenyc.com/system-data

מסמך הספק מפרסם feed זמינות בפורמט GBFS. הפרויקט משתמש בזמינות תחנות, ולא בנתוני נסיעות אישיים מהורדות היסטוריות.

ASP.NET Core MVC Views: https://learn.microsoft.com/en-us/aspnet/core/mvc/views/overview?view=aspnetcore-8.0

התיעוד מסביר ש־Views הן תבניות HTML עם Razor וש־partials מאפשרות שימוש חוזר בתצוגה. הספר מיישם מושגים אלה על הקבצים הקיימים בפרויקט.

Apache Kafka Documentation: https://kafka.apache.org/documentation/

תיעוד כללי למושגי הודעות וצרכנים; הגדרות הפריסה הספציפיות נמצאות בקובץ Compose של הפרויקט.

הספר אינו כולל סיסמאות, tokens או גיבובי סיסמה. דוגמאות מספריות מסומנות ככאלה. בדיקות מקומיות שעברו מתועדות בנפרד מאימות של תשתיות אמיתיות. לפני הגשה יש לקרוא את המסמך ולוודא שהמגישים מסוגלים להסביר את הבחירות ואת המגבלות במילים שלהם.